SOST

Proof of Irreversible Convergence

ConvergenceX • PoPC • Metals Reserve (tokenized, allowlisted)

Technical Whitepaper v4.0

> **Living Document Notice:** This whitepaper reflects the genesis design of SOST Protocol. SOST is a dynamic project under continuous development, with ongoing improvements in narrative, DeFi mechanics, and security. Operational parameters (non-consensus) may be updated or adapted over time. Any changes will be made in the interest of the project's long-term health and the benefit of all SOST holders. Consensus rules (Section 10) remain immutable at genesis. Updated versions will be published with full errata documentation.
>
> **CURRENT (since V15, block 25,000):** the coinbase is split 50% miner / 50% DTD. The 25% Gold Funding Vault allocation described in this paragraph is HISTORICAL (blocks 0–24,999); the vault receives no new emission and its frozen balance funds the DTD Jackpot (Section 4.2).

SOST is a Proof-of-Work protocol where each mined block allocates 25% of issuance to an on-chain Gold Funding Vault (Phase 1: gold-first, tokenized XAUt/PAXG) by consensus. Conversions to XAUT/PAXG are operational actions executed with public attestations and auditable trails. Purchased precious-metal tokens are deposited into a separate Heritage Reserve on Ethereum mainnet. The Heritage Reserve is sealed by default. Reserve assets are intended to remain perpetual and untouched while SOST exists. During Phase I (before block 10,000), the protocol developer executes reserve operations operationally. After V6 activation (block 10,000), reserve custody transitions to the protocol itself under the 5-defense Gold Vault model, requiring ≥95% miner supermajority signaling and satisfying all five constitutional defenses for any non-routine reserve movement.

ALLOWLIST (v2 — conservative scope) In mainnet v2, the reserve is limited to tokenized precious metals with public audits and verifiable liquidity. Initial allowlist: PAXG (Paxos Gold) and XAUt (Tether Gold). Expanding to other metals (silver/platinum/palladium/rhodium/iridium) is a roadmap item, not a promise. An asset may be added only if it satisfies: (1) verifiable custody/audits, (2) sufficient liquidity, (3) acceptable issuer risk and public documentation.

A mathematically grounded Proof-of-Work based on verifiable dynamical-system certificates.

No ICO • No Premine • No VCs • No Ongoing Parameter Governance

The protocol is permissionless by design. Third-party exchanges, custodians, or issuers may apply their own KYC/AML requirements.

Mainnet v2 Genesis: 2026-03-15 18:00:00 UTC Note: mainnet v2 is a clean regenesis following security and custody improvements.

Table of Contents

Abstract

The Problem

ConvergenceX: Proof of Irreversible Convergence

Monetary Policy

Metals Reserve Protocol (Gold-first)

Proof of Personal Custody (PoPC)

Governance Model

Economic Model

Security Analysis

The Constitution

Roadmap

Technical Specifications

Conclusion Legal Disclaimer Appendix A — Canonical Constants (Normative) Appendix B — Serialization (Normative) Appendix C — PRNG Specification (Normative) Appendix D — Test Vectors (Normative) Appendix G — Emergency Reserve Authorization (V6 5-Defense Model) (Normative) Appendix M — Removed: Adaptive Mode (Historical, Non-Normative)

Historical Appendices (Non-Normative): Appendix F — Errata v3.3 Appendix H — Errata v3.4 Appendix I — Errata v3.5

1. Abstract

SOST is a Proof-of-Work cryptocurrency built on ConvergenceX, a consensus mechanism that replaces brute-force hash guessing with a mandatory sequential convergence process. Each mining attempt requires ~8 GB RAM (4 GB dataset + 4 GB scratchpad), 100,000 strictly sequential rounds of integer-only gradient descent over a deterministic random linear system, memory-hard scratchpad reads at every round, and a stability basin verification that produces a deterministic certificate. Node validation requires only ~500 MB RAM — no dataset or scratchpad is needed to verify blocks.

Every block splits the coinbase reward by consensus:

50% + remainder to the miner,

25% to a Gold Funding Vault (Gold-first),

25% to a PoPC Pool powering Proof of Personal Custody.

Purchased precious-metal tokens are held on Ethereum mainnet in a Heritage Reserve (sealed by default). During Phase I (before block 10,000), the protocol developer may adjust future purchase execution (including the default 50/50 XAUt/PAXG split) operationally. After V6 activation (block 10,000), all reserve adjustments require protocol-level governance under the 5-defense Gold Vault model. Previously purchased reserve

assets are intended to remain perpetual and cannot be moved except under Emergency Catastrophe authorization (≥95% miner supermajority + all five constitutional defenses satisfied).

PoPC audit randomness derives exclusively from ConvergenceX block entropy: (block_id, commit, checkpoints_root). No external oracle is required for consensus-critical validation; off-chain data is used only for operational PoPC policy and settlement.

No ongoing parameter governance exists. Monetary rules, emission schedules, coinbase splits, and constitutional constraints are immutable at genesis. Only narrow emergency authorization signaling exists, requiring ≥95% miner supermajority under the V6 5-defense Gold Vault model. During Phase I (before block 10,000), operational actions (vault execution, reporting, emergency rotations) are executed by the protocol developer, constitutionally constrained, and publicly auditable. After V6 activation, these functions transition to automated on-chain governance.

2. The Problem 2.1 Proof-of-Work: A Standard Worth Studying

Bitcoin's SHA-256 Proof-of-Work (2009) remains an engineering landmark: minimal, rigorous, and resilient under real adversarial pressure. Its design established the core insight that verifiable computation can secure decentralized consensus at global scale.

At the same time, the dominant pattern across PoW systems remains a lottery-style threshold check:

nonce → hash → compare to target

This structure—brilliant in its simplicity—also enables:

massive parallelization (benefiting specialized pipelines),

rapid hardware specialization,

mining centralization driven by economies of scale.

SOST does not attempt to "replace" Bitcoin's virtues. It explores a different design space: structured work that produces verifiable certificates and public entropy as a byproduct.

2.2 Memory-Hard Alternatives

RandomX and Ethash advanced ASIC-resistance via memory-hardness and reduced specialized advantage. Yet the proof object remains primarily a hash threshold. Computation is structured, but it does not yield a convergence certificate nor a reusable entropy artifact beyond the block hash.

2.3 Crypto Has Limited Structural Reserves

Many systems discuss reserves or backing, but few embed structural accumulation as a constitutional side effect of mining. SOST implements a consensus-enforced allocation to an on-chain vault (Gold Funding Vault), with an operational conversion pipeline into a separate Heritage Reserve (sealed) on Ethereum mainnet, with an emergency catastrophe procedure for exceptional circumstances.

3. ConvergenceX: Proof of Irreversible Convergence

3.0 Design Roots (Motivation)

Before any technical specification, it is worth stating the principle that ConvergenceX encodes into consensus: trial and error — the oldest and most universal method of progress known to humanity. Since the first tool was shaped by hand, every meaningful advance has followed the same pattern: attempt, fail, measure the failure, correct, attempt again. This iterative refinement is not merely a technique; it is the only known path toward anything resembling perfection — a destination that, by its nature, can be approached but never reached.

This asymptotic quality is not a defect. It is a structural feature of the universe itself. The cosmic microwave background — the residual radiation from the Big Bang — is not perfectly uniform. It contains tiny anisotropies, density fluctuations of roughly one part in 100,000. These imperfections were not errors in the universe's initial conditions; they were the seeds of all structure. Without those minute deviations from uniformity, matter would never have clumped into galaxies, stars would never have ignited, planets would never have formed, and life would never have emerged. The universe required imperfection to produce complexity. A perfectly smooth cosmos would be a perfectly sterile one.

The same principle operates at every scale. Biological evolution depends on imperfect DNA replication — mutations are errors, yet they are the sole engine of adaptation. Scientific knowledge advances through falsification: every disproven hypothesis narrows the residual error and brings the model closer to reality, but never arrives at a final truth. Engineering iterates through prototypes, each one correcting the failures of its predecessor.

ConvergenceX translates this universal principle into a consensus mechanism. A miner does not guess randomly until lucky. A miner begins with an approximate solution, measures the error (the residual), applies a correction (gradient descent), and repeats — 100,000 times, sequentially, each round depending on the one before it. The proof-of-work is not a lottery ticket. It is a recorded journey of iterative refinement: a trajectory from disorder toward convergence, verified by a mathematical certificate that proves the solution reached a genuine basin of attraction — stable, reproducible, and irreversible. The block is valid not because the miner was fortunate, but because the miner demonstrated disciplined, sequential improvement under strict deterministic rules. In this sense, ConvergenceX is not an invention. It is a formalization of the oldest principle in nature: progress through corrected error.

ConvergenceX is born from a very concrete observation: over the last decade, computing has rediscovered that solving linear systems and performing matrix–vector multiplications is not only "computation," but dynamics—a convergence process that can be verified. Modern analog computing (especially the in-memory computing line using resistive memories / RRAM) shows that a matrix can be represented as a physical structure and that certain operations can emerge almost "by the laws of the circuit," while precision is recovered through iterative refinement, bit-slicing techniques, and block partitioning for scalability. ConvergenceX carries that intuition into consensus: it replaces the "pure randomness" of the hash lottery with a mandatory sequential trajectory (deterministic iterations), where each round depends on the previous one, memory is mixed in (scratchpad) to penalize shortcuts, and the final result is accom-

panied by a verifiable certificate (checkpoints + stability-basin proof). In essence, it is a PoW inspired by the idea that the hardest work to counterfeit is not “getting lucky,” but demonstrating convergence under strict rules, reproducibly and identically on any machine (integer-only arithmetic, no floating point)—a reinterpretation of the spirit of the Antikythera Mechanism, but with modern standards of verifiability and adversarial consensus.

A simple mental model is an arrows-to-a-target process. Imagine a target board (the true solution) and an archer that is extremely fast but not perfectly precise. The first volley lands “somewhere near” the center — not exact, but close. Then the archer looks at the error (how far the arrows are from the bullseye) and fires corrected volleys that progressively shrink that distance. After a few rounds, the arrows cluster tightly around the bullseye. That is the essence of iterative refinement: quick approximate step → measure error → correct → repeat. In RRAM systems, the “fast volley” is the analog in-memory matrix operation, and the “correction rounds” are deterministic refinement loops that drive the residual down until the desired precision is reached.

ConvergenceX transfers this intuition into consensus. Instead of a pure hash lottery, a miner must execute a mandatory, strictly sequential convergence journey: derive a deterministic per-tip problem, run fixed rounds of integer-only updates with memory-hard mixing, commit intermediate checkpoints, and finally prove the result lies in a stable basin of attraction (a verifiable “this really converges” certificate). In the same arrows analogy, the block is valid only if the miner can show not just one lucky arrow, but an entire trajectory of corrections that consistently drives the solution toward the target — under rules that every node can reproduce bit-for-bit. The goal is not to claim “analog hardware inside the chain,” but to adopt the convergence principle: work that is hardest to fake is not random guessing, but demonstrating irreversible convergence under strict, deterministic constraints.

3.1 Core Concept

Instead of “guessing until the hash is small enough”, a miner must complete a deterministic convergence journey:

generate a per-tip deterministic problem (block_key),

execute 100,000 sequential rounds with memory-hard mixing,

commit checkpoints into a Merkle root,

verify stability basin constraints,

then compare a single commitment hash to the difficulty target.

You cannot win without completing the journey. Round r+1 depends on round r. Intra-attempt parallelism is structurally eliminated.

3.2 Algorithm — Detailed Specification (Consensus-Critical)

All arithmetic is integer-only (32-bit signed clamping where needed). No floating point exists in consensus paths.

Phase 1: Problem Generation (per tip) block_key = SHA256(prev_hash || “BLOCK_KEY”)

Matrix M (32×32): raw = PRNG(SHA256(MAGIC || “M” || block_key), 32*32 bytes) M[i][j] = (raw[k] % 255) - 127

Vector b (32): raw = PRNG(SHA256(MAGIC || “B” || block_key), 32*4 bytes) b[i] = int32_le(raw[i*4..i*4+3])

A(x) = M^T(Mx) + λx with λ = 100

block_key depends only on prev_hash (reduces grinding surface). Header fields remain bound via header_core inside seed and commit.

Phase 2: Iterative Convergence (100,000 rounds) header_core = prev_hash||merkle_root||timestamp LE)||powDiffQ(u32 LE)

seed = SHA256(MAGIC||“SEED”||header_core||block_key||nonce||extra_nonce) x0 = PRNG(SHA256(MAGIC||“X0”||seed), 32*4 bytes) x0[i] = asr_i32(int32_le(x0[i]), 4) state = SHA256(MAGIC||“ST”||seed)

for r=1..100000: 1) integer gradient step using A(x) 2) 4 scratchpad reads (state-dependent indices) 3) mix scratchpad into x (XOR/add + shifts) 4) state = SHA256(state || m0..m3 || x[j0..j3] || r) 5) checkpoint every 6250 rounds (16 total) Phase 3: Checkpoint Merkle Tree (16 leaves)

Each leaf binds {state_hash, x_hash, round, residual(L1)} into checkpoints_root.

Phase 4: Stability Basin Verification (certificate)

Deterministic perturbation probes (k) and refinement steps (steps) enforce:

local non-explosion, and

global contraction for sufficiently large perturbations.

cASERT adaptively selects (k, steps, scale) for liveness, with an asymmetric policy: it never relaxes under fast conditions.

Phase 5: Commitment & Target Check COMMITMENT (binds all proof components):

commit = SHA256(MAGIC || “COMMIT” || header_core || // binds to all header fields (anti-grinding) seed || // binds to nonce + block context state || // binds to full 100k-round computation x_bytes || // binds to final solution vector checkpoints_root || // binds to convergence trajectory stability_metric_u64_le // binds to basin verification)

VALID iff commit <= target(powDiffQ) // Direct comparison (big-endian, 32 bytes). // No second hash: commit already binds the full proof via SHA256.

The commitment hash is a single SHA-256 that binds together every component of the proof: the initial conditions (seed), the full computation trace (state), the final solution (x_bytes), the convergence trajectory (checkpoints_root), and the stability certificate (stability_metric). Any modification to any component changes the commitment, invalidating the proof.

3.3 Stability Basin Verification — Mathematical Certificate

The stability basin check is what transforms ConvergenceX from “structured computation” into a “mathematical certificate.” It answers the question: **is this solution a**

genuine attractor, or did the algorithm land here by accident?

Concept A stability basin (or basin of attraction) is a region of the solution space where all nearby starting points converge to the same solution under the dynamics of the system. If `x_final` is in a stability basin, then small perturbations to the starting conditions will still lead to the same (or very nearby) solution after additional refinement steps.

ConvergenceX tests this by launching k deterministic perturbation probes around the final solution and verifying two independent criteria:

Criterion 1: Local Non-Explosion

```
LOCAL NON-EXPLOSION (per probe, per step):

  For each probe i = 0..k-1:
    delta[i] = deterministic perturbation vector (scale ∈ [-S, +S])
    x_ref  = x_final
    x_pert = x_final + delta[i]
    d_prev = L1_distance(x_ref, x_pert)   // = d0

    For step t = 1..g (gradient refinement):
      x_ref  = gradient_step(x_ref,  M, b, λ, stability_lr_shift)
      x_pert = gradient_step(x_pert, M, b, λ, stability_lr_shift)
      d_now  = L1_distance(x_ref, x_pert)

      REJECT if d_now > d_prev + margin_eff
      // Distance must never grow faster than allowed margin

      d_prev = d_now
```

This ensures that the perturbed trajectory does not diverge from the reference trajectory at any step. If it does, the solution is near an unstable region and the proof is invalid.

Criterion 2: Global Contraction

```
GLOBAL CONTRACTION (for large perturbations only):

  d0 = initial L1 distance between x_ref and x_pert
  d_final = L1 distance after g refinement steps

  IF d0 > large_threshold:
    REJECT if d_final * c_den > d0 * c_num + margin_eff

  Where:
    large_threshold = (n * scale) / 2
    c_num = 7, c_den = 10    // contraction ratio 0.7
```

```
    margin_eff = margin * m_num / m_den
```

For perturbations that are large enough to meaningfully test the basin structure, the final distance must be strictly smaller than the initial distance (with a contraction factor of 0.7). This proves that the dynamics are contractive — nearby points get pulled toward the solution, which is the mathematical definition of a stable attractor.

Why Two Criteria Local non-explosion catches solutions near saddle points or ridges where small perturbations in specific directions cause rapid divergence. Global contraction catches solutions in flat regions where the dynamics neither converge nor diverge — technically stable but not in a genuine basin.

Together, they provide a rigorous certificate that `x_final` is a true attractor of the regularized linear system.

Deterministic Perturbation Probes

```
PERTURBATION GENERATION (consensus-critical, miner-unbiasable):

  stctx = SHA256(MAGIC || "STCTX" || header_core || seed || checkpoints_root)

  For probe i:
    raw = PRNG(SHA256(MAGIC || "PERT" || stctx || i || n), n bytes)
    delta[j] = (raw[j] % (2*scale + 1)) - scale

  Properties:
    - Deterministic: same block → same perturbations
    - Unbiasable: stctx depends on full proof, not just nonce
    - Uniform: perturbation components in [-scale, +scale]
```

The stability context `stctx` is derived from `header_core`, `seed`, and `checkpoints_root`. This means the perturbation vectors depend on the entire proof trajectory — a miner cannot choose a nonce that produces "easy" perturbations without first completing the full 100,000-round convergence.

cASERT Equalizer Parameters The intensity of perturbation probes adapts to network conditions through the cASERT equalizer (Section 3.12). The equalizer is the structural layer of difficulty control and is consensus-critical. It uses 43 profiles total. Active range height-gated: Pre-V12 (#5,750..#7,349): 21 active E7-H13, 22 reserved H14-H35. V12 (#7,350..#11,999): 28 active E7-H20, 15 reserved H21-H35. V13 (#12,000+, current): 43 active E7-H35, 0 reserved. CASERT_H_MIN=-7. H13 pre-V12 ceiling, H20 V12 ceiling, H35 current ceiling (V13). H10-H35 margin=115. B0 is baseline. This completes the cASERT equalizer calibration.

Profile table (struct: {scale, steps, k, margin}) — HISTORICAL draft values, superseded; the current consensus table is in Section 3.12 (E7{1,1,1,200} ... B0{1,4,4,165} ... H35{2,20,20,115}):

Profile	Scale	Steps	k	Margin
E7(-7)	1	2	3	320
E6(-6)	1	2	3	300
E5(-5)	1	2	3	290
E4(-4)	1	2	3	280
E3(-3)	1	3	3	240
E2(-2)	1	4	3	225
E1(-1)	1	4	4	205
B0(0)	1	4	4	185
H1(+1)	1	5	4	170
H2(+2)	1	5	5	160
H3(+3)	1	6	5	150
H4(+4)	1	6	6	145
H5(+5)	2	5	5	140
H6(+6)	2	6	5	135
H7(+7)	2	6	6	130
H8(+8)	2	7	6	125
H9(+9)	2	7	7	120
H10(+10)	2	7	8	115
H11(+11)	2	8	8	110
H12(+12)	2	8	9	105
H13(+13)	2	9	9	100
H14-H20	2	9-12	10-13	115
H21-H35	2	13-20	13-20	115

Anti-Stall Decay (zone-based, targeting B0):

Hardening profiles decay toward B0 (natural destination): H13–H7: drop 1 level every 600s (10 min/level) H6–H4: drop 1 level every 900s (15 min/level) H3–H1: drop 1 level every 1200s (20 min/level) B0 : natural destination, no further decay toward easing Easing (E1–E7) only after 6+ additional hours at B0 (1800s/level). Relief valve: HISTORICAL E7 after 605s (blocks 5,750-6,549); current V12 triangular cascade (block 7,350+). When mining resumes, decay stops instantly. Validators recompute the profile with the block's own timestamp, so the cascade and anti-stall apply identically at validation (the declared profile must match).

At B0 (baseline), 4 independent probes each run 4 gradient steps with perturbation scale 1 and margin 165 (block 5,750+; was 185 pre-V8). When the chain is behind schedule, cASERT bitsQ handles difficulty reduction and the Equalizer never hardens (lag <= 0 -> at most B0); easing below B0 (to E7) comes only from the V12 cascade on a slow block or from anti-stall. When the chain runs ahead, profiles harden through H1-H35 (current ceiling since #12,000; HISTORICAL H13 / H20), progressively tightening stability requirements. This asymmetric policy ensures that relaxation is only available when the chain genuinely needs help, never when an attacker is dominating hashrate.

3.4 Anti-Grinding and Proof Binding

ConvergenceX implements multiple layers of anti-grinding protection to prevent miners from exploiting degrees of freedom in block construction.

Block Key Derivation

```
BLOCK KEY (per-tip, anti-grinding):

  block_key = SHA256(prev_hash || "BLOCK_KEY")

  This determines:
    - Matrix M (32×32 random integers)
    - Target vector b (32 random integers)
    - Scratchpad access pattern (via state evolution)
    - Stability perturbation vectors (via stctx)
```

The block key depends only on `prev_hash`. This means that all attempts for a given chain tip share the same linear system — the problem is stable and predictable once the tip is known. Changing the nonce changes the initial state `x0` and the convergence path, but the problem itself (M, b) remains the same.

This is a deliberate design to minimize grinding surface. If `block_key` included `merkle_root`, `timestamp`, or `powDiffQ`, miners could iterate over header field combinations to search for a problem with a "favorable" convergence landscape — effectively multiplying the search space in a way that benefits sophisticated miners disproportionately. With `prev_hash` only, the problem is fixed per tip and cannot be manipulated without chain reorganization.

Full header binding is achieved by injecting `header_core` into the *attempt seed* and the final `commit`, so any change to `merkle_root`, `timestamp`, or `powDiffQ` forces a full recomputation. This is a deliberate design choice to minimize *problem grinding*. Header field grinding is neutralized because `header_core` is bound into `seed` and `commit`:

```
HEADER CORE (binds all header fields into the proof):

 header_core = prev_hash(32) || merkle_root(32) || timestamp(u32 LE) || powDiffQ(u32 |
  // Total: 72 bytes, canonical little-endian serialization

  header_core is passed to ConvergenceX as an input parameter.
  It is mixed into the commitment hash, ensuring that any change to
  merkle_root, timestamp, or powDiffQ invalidates the proof —
  even though the problem (M, b) remains the same.
```

To get a different problem, the miner must change `prev_hash` — which requires reorganizing the chain. The other header fields are consensus-constrained: - `merkle_root`: determined by transaction selection (changes invalidate commit, not problem) - `timestamp`: constrained by MTP and future drift rules - `powDiffQ`: determined by cASERT bitsQ

Stability Context Binding

```
STABILITY CONTEXT (anti proof-reuse):

  stctx = SHA256(MAGIC || "STCTX" || header_core || seed || checkpoints_root)

   This binds the stability verification to:
     - The specific block being mined (header_core)
     - The specific nonce being tried (seed)
     - The full convergence trajectory (checkpoints_root)
```

A miner cannot reuse a stability proof from one block/nonce combination on another. The perturbation vectors are different for every attempt, making each stability certificate unique to the exact proof it certifies.

Seed Derivation

```
SEED (per-attempt uniqueness):

  seed = SHA256(MAGIC || "SEED" || header_core || block_key || nonce || extra_nonce)

   Derived values:
     x0 = initial solution vector
     All downstream computation depends on seed
```

The seed combines `header_core` (binding all header fields), the block key (shared across attempts), and the nonce/extra_nonce (unique per attempt). This ensures that each attempt starts from a different initial condition and traverses a different convergence path through the shared problem landscape. Any change to `merkle_root`, `timestamp`, or `powDiffQ` changes the seed and forces a full recomputation.

3.5 Checkpoint Merkle Tree — Structure and SPV Applications

Tree Structure ConvergenceX produces 16 checkpoints during the 100,000-round convergence loop, one every 6,250 rounds (100,000 / 6,250 = 16 exactly). Each checkpoint captures a snapshot of the computation state:

```
CHECKPOINT SCHEMA:

  checkpoint = {
    round:      uint32,     // round number (6250, 12500, ..., 100000)
   state_hash: bytes32,   // SHA256 chain of all state updates to this point
    x_hash:     bytes32,    // SHA256 of solution vector x at this round
    residual:   uint64     // ||A(x) - b||₁ (convergence quality metric)
  }

LEAF CONSTRUCTION:

  leaf = SHA256("CP" || state_hash || x_hash || round_u32_le || residual_u64_le)
```

```
TREE PROPERTIES:
  - 16 leaves → 4 levels → root in 15 hash operations
  - Proof for any single checkpoint: 4 sibling hashes (128 bytes)
  - Odd leaf count: last leaf duplicated (standard Merkle padding)
```

SPV Verification Use Cases **Light client block validation:** A node in BASIC sync mode can verify the `checkpoints_root` without recomputing any gradient steps. It trusts the root (validated by full nodes at chain tip) and can spot-check individual checkpoints when operating in ADAPTIVE mode.

Convergence quality audit: By requesting checkpoints at rounds 6,250 and 100,000 from a full node, a light client can verify that the residual decreased — confirming genuine convergence occurred. The Merkle proof ensures the checkpoint data is authentic.

Entropy verification: The `checkpoints_root` is one of three components in the PoPC entropy triple. A PoPC verifier can confirm that the entropy source is authentic by validating the Merkle root against the block header, without needing to recompute the full proof.

3.6 Integer-Only Consensus Arithmetic

Every computation in the ConvergenceX consensus-critical path uses integer arithmetic exclusively. No floating-point operations exist in any consensus code path. This is a deliberate design decision with concrete implications.

Why Zero Floating-Point IEEE 754 floating-point arithmetic is not deterministic across all hardware. Different CPU architectures, compiler optimizations, FPU modes, and instruction orderings can produce slightly different results for the same computation. In a consensus system, even a single bit of divergence causes a chain split.

Bitcoin avoids this by using integer arithmetic for difficulty adjustments and emission calculations. ConvergenceX extends this principle to the entire proof-of-work computation, which includes matrix-vector multiplication, gradient descent, residual computation, and stability verification — operations that traditionally use floating-point.

Implementation

```
INTEGER ARITHMETIC PRIMITIVES (consensus-critical):

  clamp_i32(x):
    return max(-2^31, min(2^31 - 1, x))

  u32(x):  return x & 0xFFFFFFFF
  u64(x):  return x & 0xFFFFFFFFFFFFFFFF

  sat_u64_add(a, b):
    return min(a + b, 2^64 - 1)
```

```
  safe_residual_l1(Ax, b, n):
    acc = 0
    For i in 0..n-1:
      acc += abs(clamp_i32(Ax[i] - b[i]))
      if acc > 2^63 - 1: return 2^63 - 1
    return acc

  l1_dist_sat_u64(a, b):
    acc = 0
    For i in 0..n-1:
      d = abs(clamp_i32(a[i]) - clamp_i32(b[i]))
      acc = sat_u64_add(acc, d)
      if acc == U64_MAX: return U64_MAX
    return acc

  asr_i32(x, s):
    if s >= 31: return (x < 0) ? -1 : 0
    ux = uint32(x)
    shifted = ux >> s
    if x < 0:
      mask = 0xFFFFFFFF << (32 - s)
      shifted |= mask
    return int32(shifted)
```

Emission Schedule (Integer-Only Exponentiation) The emission decay uses fixed-point arithmetic with explicit floor division:

```
FIXED-POINT EMISSION (consensus-critical):

  Q_NUM = 7,788,007,830,714,049
  Q_DEN = 10,000,000,000,000,000

  _qmul_floor(a, b):
    return (a * b) // Q_DEN

  _qpow_floor(base, exp):
    result = Q_DEN
    x = base
    while exp > 0:
      if exp & 1: result = _qmul_floor(result, x)
      x = _qmul_floor(x, x)
      exp >>= 1
    return result

  subsidy(height):
    epoch = height // 131,553
    qpow = _qpow_floor(Q_NUM, epoch)
    return (R0 * qpow) // Q_DEN
```

This produces bit-identical results on any hardware, any language, any platform — the fundamental requirement for consensus arithmetic.

3.7 ASIC Resistance — Design and Analysis

The Memory-Hard Foundation Each ConvergenceX attempt performs 100,000 rounds, each reading 4 pseudo-random 32-bit words from a 4 GB scratchpad. The access indices are derived from the evolving state hash, which changes every round. This creates 400,000 pseudo-random reads across 1,073,741,824 possible 4-byte positions per attempt.

```
MEMORY ACCESS ANALYSIS (per attempt):

  Scratchpad size:     4,096 MB = 4,294,967,296 bytes
  Addressable words:   1,073,741,824 (32-bit words)
  Reads per round:     4
  Rounds per attempt:  100,000
  Total reads:         400,000

  Access pattern:      Pseudo-random, state-dependent
  Predictability:     Zero — each index depends on state from prior round
  Locality:          None during mining — uniform distribution across 4 GB
                (L3 cache reuse applies to dataset/scratchpad CONSTRUCTION,
                  which is sequential. During MINING, accesses are pseudo-
random.
                CPU advantage comes from ALU operations and dataset generation,
                        not from access locality during mining.)

  Required bandwidth:  400,000 × 4 bytes = 1.6 MB read per attempt
  Working set:      The access pattern is state-dependent and unpredictable;
                           the design targets a high penalty for time-
memory tradeoffs
                and requires practical residency of the full scratchpad for
                 competitive mining. A dedicated TMTO analysis is planned.
```

Why Sequential Dependency Defeats ASICs The critical constraint is not the memory reads themselves — it is the sequential dependency chain:

```
SEQUENTIAL DEPENDENCY (unbreakable):

  Round r produces:
   state[r] = SHA256(state[r-1] || scratchpad_data || solution_data || r)

  Round r+1 needs:
    state[r] to derive scratchpad indices idx0..idx3
    x[r] to compute gradient step

  Therefore:
```

```
  Round r+1 CANNOT BEGIN until round r COMPLETES.
  Intra-attempt parallelism is eliminated by state dependency.
  Specialized hardware may still improve energy per attempt,
  which is addressed via memory-hardness and sequential verification.
```

An ASIC for SHA-256 can compute billions of independent hashes in parallel because each hash is stateless. An ASIC for ConvergenceX can only execute one round at a time per attempt, because each round reads from the solution of the previous round. The only parallelism available is running multiple independent attempts (different nonces) simultaneously — which is exactly what a CPU with multiple cores already does.

ASIC advantage estimate: A custom ASIC could potentially optimize the SHA-256 operations within each round and achieve tighter memory bus timing. The design is intended to minimize ASIC advantage; preliminary estimates suggest low multiples over modern CPUs, compared to >1000× advantage for SHA-256 ASICs. **Formal, reproducible benchmarks** will be published separately (methodology, hardware baselines, energy per attempt, and RAM latency profiles).

Scratchpad Construction (Epoch-Based) The scratchpad is deterministic, epoch-specific, and constructed via a sequential SHA-256 chain. It is built once per epoch and reused across all blocks within that epoch:

```
SCRATCHPAD CONSTRUCTION (deterministic, epoch-based):

  epoch_key(epoch=0):
    SHA256(MAGIC || "EPOCH" || epoch_u32)

  epoch_key(epoch>0):
    SHA256(MAGIC || "EPOCH" || epoch_u32 || anchor_block_id)

  build(seed_key, size_mb):
    h = SHA256(MAGIC || "SCR" || seed_key)
    counter = 0
    For each 32-byte position in scratchpad:
      h = SHA256(h || counter_u32)
      write h to scratchpad
      counter += 1

  Properties:
    - Deterministic: same epoch → same scratchpad on all nodes
    - Sequential: each 32-byte chunk depends on the previous one
    - Epoch-specific: new epoch → new scratchpad → new access patterns
    - Anchor-dependent (epoch>0): scratchpad for epoch N depends on
    the block that ended epoch N-1, which is unknown until that block is mined
```

Epoch anchor innovation: For epoch > 0, the scratchpad key depends on the `block_id` of the last block of the previous epoch. Since `block_id` commits to the full header (including timestamp), it is sufficient for uniqueness without separately including `anchor_timestamp` — which would create a timestamp-grinding vector. This

means the scratchpad for a new epoch cannot be pre-computed — miners must wait for the epoch transition block to be mined before building the new scratchpad.

Dataset Cache (Per-Block, v2.0) In addition to the epoch-based scratchpad, ConvergenceX v2.0 introduces a persistent 4 GB dataset that is generated once per block from prev_hash and cached across all nonce attempts for the same tip:

```
DATASET (v2.0, per-block):

  CXDataset.generate(prev_hash):
    Deterministic 4GB (512M uint64_t entries) derived from prev_hash.
    Cached and reused across all nonce attempts for this block.
    Regenerated only when prev_hash changes (new tip).

  Properties:
       - Per-block: derived from prev_hash (NOT from block_hash —
no circularity)
    - Cached: generated once per tip, amortized across all mining attempts
     - Memory-hard: 4 GB residency required
```

Per-Block Program (v2.0) Each block generates a unique 256-operation program from the block_key (which is derived from prev_hash). This program executes during each round, mixing its output into the scratchpad values. The program makes ASIC optimization impractical because the operation sequence changes every block:

```
PROGRAM (v2.0, per-block):

  CXProgram.generate(block_key):
    256 operations derived deterministically from block_key
    Operations: MUL, XOR, ADD, ROT, AND, OR, NOT, SUB
    Each operation has a unique immediate value

  During each round r:
    prog_state = program.execute(dataset[r % dataset_size], r)
    Mix prog_state into scratchpad values (XOR into m0, m1)

  Properties:
    - Derived from block_key (= SHA256(prev_hash || "BLOCK_KEY"))
    - NOT derived from block_hash (no circularity)
    - Changes every block → no fixed pattern to optimize in hardware
```

The v2.0 architecture layers three memory-hard components: the epoch scratchpad (traditional sequential reads), the per-block dataset (4GB cached), and the per-block program (variable ALU operations). Together they create a mining workload that changes its character every block while maintaining deterministic reproducibility for validation.

3.8 ConvergenceX as Entropy Engine

Every valid block produces a unique, unpredictable, publicly verifiable entropy triple:

```
ENTROPY TRIPLE (per block):

  E = (block_id, commit, checkpoints_root)

  full_header = MAGIC(10) || "HDR2"(4) || header_core(72) ||
            checkpoints_root(32) || nonce(u32 LE) || extra_nonce(u32 LE)
    - Does NOT include commit (commit is appended separately in block_id)

  block_id = SHA256(full_header || "ID" || commit)
    - Depends on: all header fields + full ConvergenceX proof
    - commit is NOT part of full_header; it is concatenated after "ID"

 commit = SHA256(MAGIC || "COMMIT" || header_core || seed || state || x || cp_root || m
    - Depends on: all 100,000 rounds of computation

  checkpoints_root = Merkle root of 16 checkpoint leaves
    - Depends on: state evolution at 16 intermediate points

PROPERTIES:
  - Unpredictable: no party can know any component before the block is mined
  - Unbiasable: choosing a different nonce changes all three values
  - Publicly verifiable: any node can validate against the block header
  - Information-rich: three independent 32-byte values per block
```

This entropy powers the PoPC audit system (Section 6.3). The same proof-of-work that secures the chain produces the randomness that governs custody audits — no external oracle, no trusted randomness beacon, and no additional computation required.

3.9 Comparative Analysis: ConvergenceX vs SHA-256 vs RandomX vs Ethash

Bitcoin's SHA-256 remains the canonical PoW baseline—simple, robust, and historically proven. SOST targets different properties: structured work + certificate + entropy triple + memory-hardness.

Dimension	Bitcoin (SHA-256)	Monero (RandomX)	Ethereum Classic (Ethash)	SOST (ConvergenceX)
Year introduced	2009	2019	2015 (PoS migration 2022)	2026
Proof nature	Hash below target	Hash below target	Hash below target	Convergence certificate + hash below target

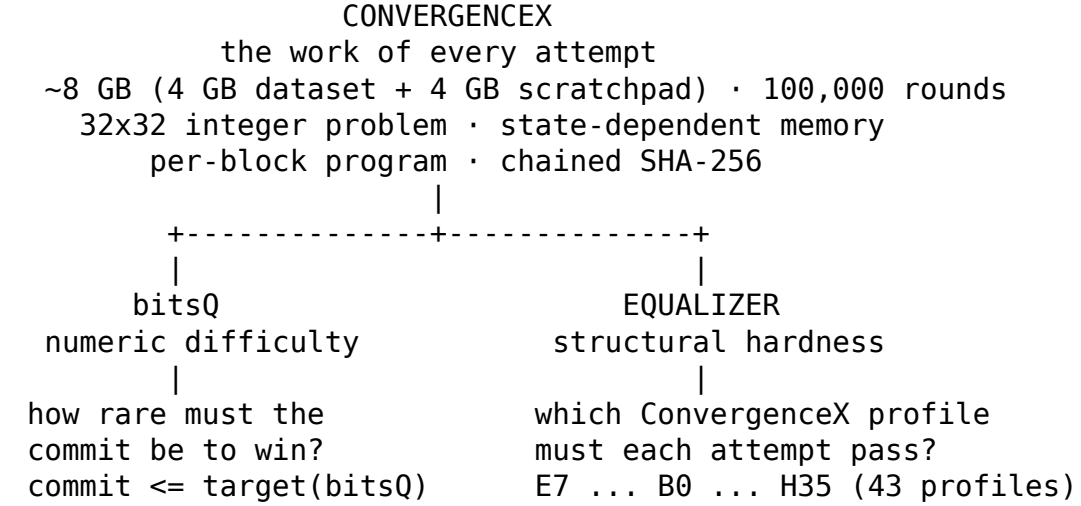
Dimension	Bitcoin (SHA-256)	Monero (RandomX)	Ethereum Classic (Ethash)	SOST (ConvergenceX)
Core operation	Double SHA-256	Random VM program execution	DAG read + Keccak hash	Gradient descent + scratchpad + stability check
RAM required (mining)	~0 MB	2,048 MB (2 GB)	1-4 GB (growing DAG)	~8,192 MB (4 GB dataset + 4 GB scratchpad)
RAM required (node)	~0 MB	~256 MB	~256 MB	~500 MB (no dataset/scratchpad)
ASIC advantage	>1,000×	~3-5× estimated	~2-5× (Ethash ASICs exist)	Designed for low multiples (hypothesis; formal benchmark pending)
Difficulty adjustment	Every 2,016 blocks (~2 weeks)	Every block (simple ratio)	Every block (bomb + ratio)	Every block (cASERT bitsQ Q16.16; V1: 48h half-life, V2: 24h half-life from block 1450; V5 (block 5175+): avg288-based with dynamic cap)
Mathematical certificate	None	None	None	Stability basin proof (k probes × g steps)
Useful entropy output	Block hash only	Block hash only	Block hash only	Entropy triple (block_id + commit + cp_root)
Consensus arithmetic	Integer (nBits, subsidy)	Integer + float (VM)	Integer	100% integer (zero floating-point)

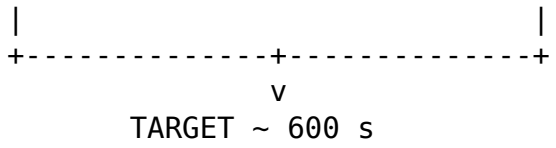
3.10 Consensus Parameters

Parameter Value RAM (mining total) ~8,192 MB (4,096 MB scratchpad + 4,096 MB dataset) RAM (node validation) ~500 MB (no scratchpad/dataset needed) Rounds per attempt 100,000 sequential State dimension 32 Learning rate shift 18 Regularization λ 100 Checkpoint interval 6,250 (16 checkpoints) Difficulty encoding SOSTCompact Q16.16 Difficulty algorithm cASERT bitsQ (V1: 48h half-life, V2: 24h half-life from block 1450; V5 (block 5175+): avg288-based with dynamic cap) Per-block delta cap V1: 6.25% (1/16), V2: 12.5% (1/8) from block 1450; V5 (block 5270+): dynamic cap — ±15s dead band, 0%/0.5%/1.0%/2.0%/~3.03% (prev/33) max based on deviation; median removed from consensus at block 5,270, avg288 is the sole signal Equalizer profiles 43 total (table since block 5,750). Pre-V12 (#5,750..#7,349): 21 active E7-H13. V12 (#7,350..#11,999): 28 active E7-H20. V13 (#12,000+, current): all 43 active E7-H35, ceiling H35. cASERT model unified: bitsQ (numeric) + Equalizer (structural) + anti-stall CASERT_H_MIN -7 CASERT_H_MAX 35 (ceiling H35 since V13 #12,000; HISTORICAL: H13 pre-V12, H20 V12 #7,350..#11,999; H10-H35 margin=115) Equalizer selection direct lag map (CASERT_DIRECT_LAG_HEIGHT = 5,323): lag = (h-1) - floor((prev.time - genesis)/600), positive = ahead; target = B0 if lag <= 0, else min(lag, ceiling); up immediate, down at most max(1, elapsed/600) levels/block cASERT decay anti-stall after 3,600s (60 min) without a block, first drop immediate, then H>=7 600s/level, H6-H4 900s/level, H3-H1 1200s/level to B0; easing E1-E7 after 6+ additional hours at B0 (1800s/level) Relief valve current: V12 triangular cascade (block 7,350+): elapsed >= 540s, n = min(1+(elapsed-540)/60, 7), drop n(n+1)/2 = 1..28 levels, floor E7. HISTORICAL: E7 at 605s (blocks 5,750-6,549); H1@630s (blocks 5,635-5,749) Block target spacing 600s Stability baseline (B0) scale=1, steps=4, k=4, margin=165

3.10.1 One Proof of Work, Two Dials — ConvergenceX + bitsQ + Equalizer (in simple terms)

SOST keeps blocks at about 600 seconds with two independent controls acting on the same Proof of Work. One decides how rare a winning result must be; the other decides how demanding each attempt is. Every node recomputes both and rejects a block that declares anything else.





- ConvergenceX decides WHAT work a miner runs and produces a 32-byte commit.
- bitsQ decides HOW EXCEPTIONAL the commit must be: commit <= target(bitsQ). Blocks too fast -> bitsQ up, target down, harder. Too slow -> bitsQ down, easier.
- The Equalizer does NOT touch the target. It selects the profile whose stability parameters (stab_scale / stab_steps / stab_k / stab_margin) every attempt must satisfy.

The idea that matters: SOST separates the difficulty of HITTING a winning result (bitsQ) from the COST OF ONE ATTEMPT (Equalizer). Bitcoin has only the first: one SHA-256d attempt always costs the same and only the target moves. This is a richer control architecture; it does not by itself make SOST “better” than Bitcoin.

Current bitsQ rule (block 5,270+): avg = mean of the last 288 intervals (each clamped to 1 s .. 1 day); dev = avg - 600. |dev| <= 15 s: no change; <= 60 s: <= 0.5%/block; <= 120 s: <= 1%; <= 240 s: <= 2%; > 240 s: <= ~3.03% (prev/33). Inside the cap the step is proportional: prev x (|dev| - 15) / 2400. Slingshot (7,350+): >20 min -6.5%, >30 min -12.5%, >1 h -25%, >2 h -37.5%, >3 h -50% in the same block.

Current Equalizer rule (block 5,323+): lag = (h-1) - floor((prev.time - genesis)/600), positive = ahead. lag <= 0 -> at most B0 (never harden when behind); lag > 0 -> H(lag) up to H35, up immediately, down at most max(1, elapsed/600) levels per block. Easing profiles E1..E7 come only from the V12 cascade (block already took >= 540 s: drop 1,3,6,10,15,21,28 levels at 540..>=900 s) or from anti-stall (60 min without a block).

Worked example: 1. The chain is 4 blocks ahead of schedule -> lag = 4 -> profile H4. Attempts must pass stricter stability parameters; bitsQ is unchanged. 2. The next block has already taken 660 s -> cascade drops 6 levels -> E2 for this block only: attempts become cheaper and the chain catches up. 3. The last 288 blocks averaged 640 s -> dev = +40 s; proportional step 25/2400 ~ 1.04%, capped at 0.5% -> target slightly easier.

In simple terms: ConvergenceX is the exam, bitsQ is the pass mark, and the Equalizer is how long and hard each exam paper is.

Historical designs (the five-signal PID/EWMA equalizer used before block 5,323, the H13/H20 ceilings of earlier versions, and the pre-genesis L1-L6 level concept) are NOT current rules and are kept only in the historical appendices.

3.11 cASERT bitsQ Difficulty Adjustment (Consensus-Critical)

cASERT bitsQ compares observed timing to expected schedule and adjusts powDiffQ deterministically. Redesigned as a unified system with Equalizer profiles (structural layer) and anti-stall.

Miner lag-adjust (non-consensus): Every ~6 seconds (block 5,750+; previously 30s), the miner re-queries the node for the current profile. If the profile changed (live lag

dropped or lag cap forced a lower profile), the miner restarts search immediately. Reduces wasted mining at suboptimal profiles to ~6 seconds. Old miners remain compatible — miner-side optimization only.

slow chain → decrease powDiffQ → larger target → easier fast chain → increase powDiffQ → smaller target → harder

Historical: V1 (blocks <1450): 48h half-life exponential, 6.25% cap; V2 (blocks >=1450): 24h half-life exponential, 12.5% cap. Current: V5 (block 5175+): avg288-based — compares average of last 288 block intervals against 600s target. Replaces anchor-based exponential for current blocks (historical fork descriptions preserved). Dynamic cap (block 5270+): ±15s dead band, 0%/0.5%/1.0%/2.0%/~3.03% (prev/33) max based on deviation. Median removed from consensus at block 5,270; avg288 is the sole signal. HISTORICAL: blocks 5,175-5,259 used a ±30s dead band with a flat 12.5% cap; blocks 5,260-5,269 a ±30s dead band with median288 override and 0.5/1.5/2.5/3.0% steps. Slingshot (V12, block 7,350+): on the current block elapsed time: >1200s -6.5%, >1800s -12.5%, >3600s -25%, >7200s -37.5%, >10800s -50%.

CONVERGENCEX DIFFICULTY CONTROL = bitsQ + Equalizer. bitsQ regulates the numeric difficulty of the target; the Equalizer regulates the structural cost profile of each ConvergenceX attempt under the cASERT rules. Both are recomputed and verified by every node.

3.12 cASERT Equalizer (Consensus-Critical)

The cASERT equalizer is consensus-critical. It adjusts ConvergenceX stability parameters according to the equalizer profile. Block validation verifies the equalizer profile. Both bitsQ and equalizer profile are consensus-critical.

The Equalizer uses 43 profiles from E7(-7) through H35(+35): Active range is height-gated: Pre-V12 (#5,750..#7,349): 21 active E7..H13, 22 reserved H14..H35. V12 (#7,350..#11,999): 28 active E7..H20, 15 reserved H21..H35. V13 (#12,000+, current): 43 active E7..H35, 0 reserved. CASERT_H_MIN=-7. Ceiling H35 since V13 (HISTORICAL: H13 pre-V12 / H20 V12). H10-H35 margin=115. B0 is baseline. This completes the cASERT equalizer calibration. It adjusts stability verification intensity based on how far ahead the chain is relative to the ideal schedule.

PROFILE TABLE (struct: {scale, steps, k, margin}) — block 5,750+: E7(-7): scale=1, steps=1, k=1, margin=200 E6(-6): scale=1, steps=2, k=1, margin=195 E5(-5): scale=1, steps=2, k=2, margin=190 E4(-4): scale=1, steps=3, k=2, margin=185 E3(-3): scale=1, steps=3, k=3, margin=180 E2(-2): scale=1, steps=4, k=3, margin=175 E1(-1): scale=1, steps=4, k=4, margin=170 B0(0): scale=1, steps=4, k=4, margin=165 H1(+1): scale=1, steps=5, k=4, margin=160 H2(+2): scale=1, steps=5, k=5, margin=155 H3(+3): scale=1, steps=6, k=5, margin=150 H4(+4): scale=1, steps=6, k=6, margin=145 H5(+5): scale=2, steps=5, k=5, margin=140 H6(+6): scale=2, steps=6, k=5, margin=135 H7(+7): scale=2, steps=6, k=6, margin=130 H8(+8): scale=2, steps=7, k=6, margin=125 H9(+9): scale=2, steps=7, k=7, margin=120 H10(+10): scale=2, steps=8, k=7, margin=115 H11(+11): scale=2, steps=8, k=8, margin=115 H12(+12): scale=2, steps=9, k=8, margin=115 H13(+13): scale=2, steps=9, k=9, margin=115 (pre-V12 ceiling, #5,750..#7,349) H14-H20: scale=2, steps=10-13, k=9-12, margin=115 (activated at V12 #7,350+;

H20 = V12 ceiling) H21-H35: scale=2, steps=13-20, k=13-20, margin=115 (active since V13 #12,000+; H35 = current ceiling)

SIGNAL COMPUTATION — CURRENT (direct lag map, block 5,323+): lag = (h - 1) - floor((prev.time - GENESIS_TIME) / 600) // positive = AHEAD target = B0 if lag <= 0, else min(lag, ceiling) up (hardening) immediate; down (easing) at most max(1, elapsed/600) levels per block V12 triangular cascade (block 7,350+): elapsed >= 540s -> drop 1,3,6,10,15,21,28 levels (540..>=900s), floor E7. Validator recomputes; declared profile must equal it.

SIGNAL COMPUTATION — HISTORICAL (genesis design, opposite sign convention): expected_time = GENESIS_TIME + height * TARGET_SPACING elapsed = latest_block_time - GENESIS_TIME expected_h = elapsed / TARGET_SPACING lag = expected_h - current_height ahead = max(0, -lag)

DECAY ANTI-STALL (zone-based, targeting B0; activates after 3,600s / 60 min without a block since block 4,300 — 7,200s HISTORICAL before 4,300; first drop immediate): Hardening profiles decay toward B0 (natural destination): H13–H7: drop 1 level every 600s (10 min/level) H6–H4: drop 1 level every 900s (15 min/level) H3–H1: drop 1 level every 1200s (20 min/level) B0 : natural destination, no further decay toward easing Easing (E1–E7) only after 6+ additional hours at B0 (1800s/level). Relief valve: HISTORICAL E7 after 605s (blocks 5,750-6,549); current V12 triangular cascade (block 7,350+).

Anti-stall is evaluated against the elapsed time since the previous block (the miner's clock while mining; the block's own timestamp at validation). This prevents a permanent stall if profiles become too high for any miner to solve.

DETERMINISM: cASERT derives its signal from on-chain block data (height, timestamp) and consensus constants. Timestamp manipulation is bounded by MTP and the future-drift limit (30s since block 12,000). Validators recompute the profile from the block's timestamp, so every node reaches the same result.

ASYMMETRIC POLICY: When the chain is ahead of schedule, cASERT hardens through profiles H1-H35 (H35 ceiling since #12,000; HISTORICAL H13 pre-V12 / H20 V12), progressively tightening stability requirements (scale, steps, k, and margin all adjust per profile). An attacker with high hashrate mining blocks quickly pushes the ahead count up, facing increasingly strict stability requirements. The attacker gains nothing. cASERT eases below B0 (down to E7) through the V12 triangular cascade on a slow block, or through anti-stall (60 min without a block, then E1-E7 after 6+ further hours at B0) — while cASERT bitsQ (with slingshot relief) handles difficulty reduction to maintain liveness.

3.13 Node Synchronization and Verification Policy

FULL: recompute full ConvergenceX proof for each block. ADAPTIVE (default): FULL on recent tail + sampling; BASIC deep history. BASIC: header/difficulty/time/target checks only; MUST NOT accept tip blocks.

3.13.1 Fast Sync — Hard Checkpoints and Assumevalid Anchor

New nodes joining the network can skip expensive ConvergenceX recomputation for trusted historical blocks. Structural, semantic, and economic validation always runs —

only the expensive CX recompute is conditionally skipped.

Trust model (two independent mechanisms):

1. Hard checkpoints: A block is trusted ONLY if its height AND block hash match a hardcoded checkpoint exactly. Lower height alone is never sufficient to grant trust.
2. Assumevalid anchor: Similar to Bitcoin Core's -assumevalid flag. If a known block hash exists on the active chain, ancestors of that branch can skip expensive CX recomputation. If the anchor is not on the active chain (e.g. during a chain split), no fast trust is granted.

Verification always performed (all blocks, all modes): - Header structure and magic bytes - Timestamp validation (MTP monotonicity + future drift) - cASERT bitsQ difficulty compliance - SOSTCompact target threshold (commit <= target) - Coinbase split correctness (50/25/25) - Constitutional address verification - Transaction structure and semantic rules - UTXO set updates (ConnectBlock)

Verification skipped (trusted historical blocks only): - Full 100,000-round gradient descent recomputation - 4GB scratchpad + 4GB dataset reconstruction (miner-only memory) - Stability basin re-verification

Any node operator can force full verification with –full-verify. Checkpoints and the assumevalid anchor are committed in source code, reviewed by the development team, and updated with each release. A block must have been fully verified by at least one trusted node before being added as a checkpoint or set as the assumevalid anchor.

This mechanism does not change consensus rules. A block that passes fast sync verification would also pass full verification.

3.14 Worked Example — How a Block Is Mined and Verified (ConvergenceX + cASERT)

This section provides a technical but simple walkthrough of how a SOST block is constructed and validated, using real-looking values as an example.

3.14.1 Inputs (example)

prev_hash: 32 bytes merkle_root: 32 bytes timestamp ts = 1,771,700,109 MedianTimePast MTP = 1,771,697,922 powDiffQ = 350,629 → powDiffQ / 65,536 ≈ 5.35 bits cASERT equalizer selected stability parameters (H3, moderately ahead): scale=1, steps=6, k=5, margin=150

3.14.2 Timestamp Rules (MTP + Future Drift)

Rule A: ts > MTP Here: 1,771,700,109 > 1,771,697,922 → OK (difference 2,187s). Rule B: ts <= now + future-drift limit → prevents "future blocks". Current: 30s (block 12,000+); HISTORICAL: 60s (6,550-11,999), 600s (before 6,550).

3.14.3 cASERT bitsQ Difficulty (powDiffQ → target)

Difficulty is encoded as Q16.16 log-space: bits ≈ powDiffQ / 65,536 ≈ 5.35

Validity requires: commit <= target(powDiffQ) (big-endian integer comparison)

cASERT bitsQ update intuition: if blocks slow → reduce powDiffQ → easier if blocks fast → increase powDiffQ → harder then clamp per-block change and global bounds.

3.14.4 cASERT Equalizer — What the stability parameters mean

scale: each perturbation component in [-scale, +scale] (scale 1 for E7-H4, scale 2 for H5-H35) (“how far we poke”) k: number of probes (more probes = stronger certificate; B0 k=4) steps: refinement steps per probe (B0 steps=4) margin: tolerated bounded growth in local distance (B0 margin=165) stab_shift=20: conservative refinement step size (smaller updates)

3.14.5 One mining attempt (simple pipeline)

header_core = prev_hash || merkle_root || ts || powDiffQ block_key = SHA256(prev_hash || “BLOCK_KEY”) seed = SHA256(MAGIC||“SEED”||header_core||block_key||nonce||extra_nonce) run 100k sequential rounds + scratch mixing + checkpoints compute checkpoints_root stability basin verification using cASERT equalizer params commit = SHA256(... all bound components ...) accept iff commit <= target(powDiffQ) block_id = SHA256(full_header || “ID” || commit)

3.14.6 What a validator does

BASIC: checks time, cASERT bitsQ powDiffQ, commit<=target, and linkage. FULL: recomputes ConvergenceX proof and compares commit/cp_root/metric/block_id.

3.15 SOST vs Bitcoin vs Ethereum — Security and Incentives (simple comparison)

Two questions are often mixed up: how a protocol is designed, and how decentralized its real network is. Bitcoin and Ethereum are remarkable for both. SOST shares the design principles of permissionless consensus; it does not yet have their maturity.

BITCOIN — WORK. SHA-256d Proof of Work on specialised ASICs. A block about every 10 minutes; difficulty retargets every 2,016 blocks. The block finder (usually a pool) takes the whole subsidy (3.125 BTC since the 2024 halving) plus fees. Full nodes are not paid, but each one verifies every rule on its own: miners order history, nodes decide validity.

ETHEREUM — STAKE. Proof of Stake since 2022. A solo validator deposits 32 ETH. 12-second slots, 32 slots = one epoch (6.4 min). Validators attest about once per epoch and are occasionally chosen to propose a block. Rewards: issuance, plus priority fees and MEV for proposers; the base fee is burned. Explicit finality after about two epochs. Signing conflicting blocks or votes is slashed; being offline costs only small penalties.

SOST — WORK + PARTICIPATION. ConvergenceX Proof of Work on ordinary CPUs and RAM, a block about every 600 s, bitsQ and the Equalizer adjusted every block. Since V15 the block reward (subsidy + fees) is split 50% to the finder and 50% to the DTD draw. A separate DTD Jackpot every 288 blocks rewards miners who also run a bound node.

Worked examples — the same day, three ways:

BITCOIN A miner with 1% of the hashrate finds ~1% of ~144 daily blocks ~ 1.4 blocks/day, each worth 3.125 BTC + fees. Running a node earns nothing, but lets you reject any invalid block, even from the largest pool.

ETHEREUM A 32 ETH validator attests ~once per 6.4 minutes and sometimes proposes a block (larger reward). The yearly rate is a few percent and moves with the total amount staked. Offline: small penalties. Equivocation: slashed.

SOST Epoch-0 subsidy = 7.85100863 SOST. Miner A finds a block with no fees:
A (finder) 3.92550432 SOST (the odd stock goes to the miner)
DTD half 3.92550431 SOST
The DTD half goes to ONE eligible recent miner drawn uniformly - it may be A, or B, a much smaller miner. Every 288 blocks the DTD Jackpot (100 SOST base, up to 500 with rollover, paid from the frozen historical reserve, no new coins) goes to a miner with >= 3 PoW blocks in the last 2,016, an active NODE_BIND and the heartbeat history; the draw is weighted by PoW blocks.

	Bitcoin	Ethereum	SOST
Security	PoW (SHA-256d)	PoS	PoW (ConvergenceX)
What you commit	ASICs + energy	32 ETH (solo validator)	CPU + RAM + energy; no deposit
Block producer	whoever finds the PoW	validator selected per slot	whoever finds the PoW
Votes / slashing	no / no	yes / yes	no / no
Direct reward	100% subsidy + fees	issuance; proposer fees/MEV	50% of subsidy + fees
Extra reward	-	occasional proposal	DTD every block; Jackpot every 288 blocks
Are nodes paid?	no	only as validators	no; a bound node is a Jackpot condition, never a multiplier
Sybil resistance	hashrate	staked ETH	real PoW blocks (SbPoW-signed identities)
Coins locked to take part	no	yes	no
Finality	probabilistic	explicit (~2 epochs)	probabilistic
Difficulty adjustment	every 2,016 blocks	n/a	every block (bitsQ + Equalizer)

	Bitcoin	Ethereum	SOST
Long-term security budget	fees after last halving	issuance + fees	fees only after the cap (no tail emission); under current rules half of fees also go to DTD

Why SOST does not pay nodes simply for existing: paying every node invites a Sybil attack (one person, 10,000 cheap servers). Jackpot V2 uses the node as a DOOR and real PoW as the WEIGHT:

```
1 miner · 100 nodes · 100 PoW blocks  -> weight 100
1 miner ·   1 node  · 100 PoW blocks  -> weight 100
1 miner ·   5 nodes ·   2 PoW blocks  -> not eligible (minimum 3)
```

Normal DTD today: mined in the last 288 blocks; addresses that mined any of the previous 6 blocks are skipped (with a fallback if that would leave nobody); uniform draw among the eligible. Anti-dominance (>= 10% of the last 288 blocks) switches on only when at least 11 distinct miners are active. Jackpot V2 has no cooldown and no anti-dominance.

The honest test of decentralization is “what can you switch off?”. Bitcoin and Ethereum keep producing blocks without their websites, foundations, largest pools, staking services or any single client. SOST today still depends on project-run infrastructure (seed nodes, public explorer and RPC, release distribution), has few independent operators, and two addresses produced the last 288 blocks. The goal is that switching off everything run by SOSTCore changes nothing for the chain.

In one sentence each: Bitcoin is minimalist decentralized money; Ethereum is decentralized computation; SOST aims to be Proof-of-Work money on a Bitcoin-like base (UTXO, 10-minute blocks) with ConvergenceX for broad CPU mining, per-block difficulty control and DTD incentives that reward work and node participation, without becoming a staking chain. ASIC resistance by memory hardness is a design goal that must survive years of adversaries; it is not a proven fact. Final decentralization is not written in C++: it comes from independent people running the code without needing its authors.

4. Monetary Policy

4.1 Feigenbaum Emission Schedule (Consensus-Critical)

Epoch-constant subsidy with exponential decay:

TARGET_SPACING = 600 seconds BLOCKS_PER_EPOCH = 131,553

Decay factor q = exp(-1/4) implemented as fixed-point: Q_NUM = 7,788,007,830,714,049 Q_DEN = 10,000,000,000,000,000

Initial reward: R0_STOCKS = 785,100,863 (7.85100863 SOST)

Consensus rule: epoch = height // BLOCKS_PER_EPOCH reward = floor(R0 * q^epoch) // fixed-point exponentiation by squaring

4.1.1 Hard Cap by Construction (Consensus-Critical) SOST enforces a maximum supply by construction, not by tracking cumulative supply on-chain.

Let B = BLOCKS_PER_EPOCH and 0<q<1. Since floor(x) ≤ x:

TotalIssued ≤ B * Σ_{e≥0}(R0 * q^e) = B * R0 / (1 - q) ≤ SUPPLY_MAX_STOCKS

because R0 is chosen from SUPPLY_MAX_STOCKS and q exactly as defined.

No other minting path exists. Any block that mints more than subsidy(height) is invalid. Any shortfall due to flooring remains permanently unissued.

Hard cap: 4,669,201 SOST enforced at consensus level. Subsidy drops to zero when cap is reached; miners earn fees only.

4.2 Coinbase Split (Consensus-Critical)

CURRENT (V15, block 25,000+): total_reward = subsidy(height) + fees dtd = total_reward // 2 miner = total_reward - dtd (the miner keeps the odd stock) The DTD half is drawn every block among eligible recent miners (Section 3.15); if nobody is eligible it accumulates as pending. The Gold Funding Vault and the PoPC Pool receive 0% of new emission. Their balances are frozen by consensus (rule S13): no signed transaction can spend them; only the keyless TX_TYPE_JACKPOT transaction, rebuilt by every validator, pays the DTD Jackpot (100 SOST base, 500 SOST cap with rollover, every 288 blocks; Jackpot V2 eligibility from block 30,000).

HISTORICAL (blocks 0–24,999; from block 7,100 draw blocks paid 50% miner / 50% lottery): q = reward // 4 gold = q popc = q miner = reward - gold - popc

Invariant: miner + gold + popc == reward

4.3 Coinbase Transaction Format (Normative)

HISTORICAL (pre-V15) format shown below; since block 25,000 the coinbase pays the miner and the DTD outputs only (no vault or PoPC output).

Exactly 3 outputs in canonical order: miner FUNDING_VAULT_ADDR POPC_POOL_ADDR

Amounts must match the split exactly. Any deviation → invalid block.

4.4 Supply Projections

HISTORICAL PROJECTION (genesis design, superseded at block 25,000 — the vaults stopped accumulating at ~33,000 SOST each and are now frozen for the DTD Jackpot):

VAULT ACCUMULATION (PoPC Pool & Gold Funding Vault each receive 25%):

Epoch 0 (years 0-2.502): 258,210 SOST per vault Year 1 alone: 103,160 SOST per vault

TOTAL SUPPLY BREAKDOWN: Miners: 2,334,600 SOST (50%) Gold Funding Vault: 1,167,300 SOST (25%) PoPC Pool: 1,167,300 SOST (25%) Maximum: 4,669,201 SOST (hard cap)

~95% of total supply mined in ~12 epochs (~30 years)

4.5 Comparison: SOST Emission vs Bitcoin Emission (Respectful)

Property	Bitcoin	SOST
Max supply	21,000,000 BTC	4,669,201 SOST
Initial reward	50 BTC/block	7.851 SOST/block
Decay method	Halving every 210,000 blocks	Smooth 9.03% annual decay
Decay events	Discrete cliff (reward drops 50% overnight)	Continuous (no cliff events)
Mining allocation	100% to miner	50% miner / 50% DTD since block 25,000 (HISTORICAL: 50% miner, 25% gold, 25% PoPC)
Smallest unit	1 satoshi (10^{-8} BTC)	1 stock (10^{-8} SOST)
~95% emission	~2036 (year 28)	~2056 (year 30)
Supply cap enforcement	Consensus rule	Subsidy-by-height only (upper-bounded by construction)

5. Metals Reserve Protocol (Gold-first)

5.1 How It Works — Two-Vault Architecture

SOST uses a two-vault architecture to separate **operational liquidity** from **permanent reserve storage**.

TWO-VAULT ARCHITECTURE

FUNDING VAULT (operational, on SOST chain): - Hardcoded destination in genesis consensus rules (coinbase output). - Receives exactly 25% of each block subsidy by consensus (q = reward // 4). - Consensus-enforced: any block without the correct vault output is invalid. - Accumulates SOST and periodically converts it into tokenized gold exposure (XAUT/PAXG) via publicly auditable operational execution (Section 5.3).

HERITAGE RESERVE (sealed, Ethereum mainnet): - The Heritage Reserve is an Ethereum mainnet smart contract that holds allowlisted precious-metal tokens (default: XAUt and PAXG). - Default state is Heritage (sealed): no outbound transfers occur in normal operation. - Reserve assets are intended to remain perpetual and untouched while SOST exists, except under the Emergency Catastrophe procedure (Section 5.2 + Appendix G).

FLOW: Block reward (coinbase) → [Gold Funding Vault (Gold-first) (SOST)] → sell SOST (TWAP) → buy XAUT/PAXG → [Heritage Reserve (XAUT/PAXG)] → HERITAGE / SEALED — Perpetual by default; movable only via Emergency Catastrophe.

WHY TWO VAULTS: - Gold Funding Vault needs operational flexibility (sell SOST, buy gold tokens). - Heritage Reserve needs maximum immutability (Heritage sealed). - Splitting them achieves both without mixing “hot” and “cold” functions.

ASSET POLICY: Default split: The conversion pipeline targets 50/50 acquisition of XAUt and PAXG by default. During Phase I (before block 10,000), the protocol developer may adjust this split operationally; adjustments apply only to future purchases and must be disclosed in batch reports. Previously purchased reserve assets are not rebalanced except under Emergency Catastrophe.

5.2 Constitutional Rules — Metals Reserve (tokenized, allowlisted)

The following rules are **constitutional** (project-level commitments) and are enforced either by SOST consensus (Gold Funding Vault inflow) or by immutable smart contracts and public auditability (Heritage Reserve behavior and reporting).

Metals Reserve (tokenized, allowlisted) Constitution (immutable at genesis)

G1 (Consensus Allocation): Every valid block MUST pay the Gold Funding Vault (Gold-first) exactly q = reward // 4. Any block that fails to do so is invalid.

G2 (Two-Vault Separation): Gold Funding Vault is operational; Heritage Reserve is sealed storage. All purchased XAUT/PAXG MUST be deposited into the Heritage Reserve.

G3 (Heritage Reserve): The Heritage Reserve is deployed on Ethereum mainnet and is sealed by default. No outbound transfers occur in normal operation. Reserve assets are intended to remain perpetual and untouched while SOST exists, except under Emergency Catastrophe authorization.

G4 (Purchase Start Delay + Market Readiness): Purchases begin no earlier than 8 months after public launch and only once market readiness is met. Market readiness requires at least one verifiable execution venue capable of selling SOST and acquiring XAUt/PAXG under the TWAP policy with bounded slippage. A venue may be a CEX, OTC desk, or a DEX with demonstrated liquidity, provided execution produces a publicly auditable batch trail.

G5 (Batching / Anti-Momentum): Purchases are executed in batches and MUST follow a rule-based TWAP execution policy (Section 5.3.1), not discretionary market-timing.

G6 (Allowlist + Staging Assets): Reserve assets are limited to allowlisted precious-metal tokens (default: XAUt and/or PAXG). USDC and USDT may be used strictly as temporary staging assets during execution (≤72 hours) and are not Reserve assets.

G7 (Default 50/50; Future Purchases Only): Default acquisition targets 50/50 XAUt/PAXG. During Phase I (before block 10,000), the protocol developer may adjust the split operationally; after V6 activation, adjustments require protocol-level governance. All adjustments apply only to future purchases and must be disclosed in batch reports. Previously purchased reserve assets are not rebalanced except under Emergency Catastrophe.

G8 (Issuer Risk Policy): During Phase I, the protocol developer may redirect future purchases away from a threatened issuer without consensus. After V6 activation (block 10,000), redirection requires protocol-level governance. Previously purchased reserve assets remain sealed unless Emergency Catastrophe is authorized.

G9 (Emergency Catastrophe — V6 5-Defense Gold Vault Model): The Gold Vault must be able to change form, but not purpose. Reserve movements are permitted only under a narrowly defined catastrophe procedure intended for issuer failure/migration events or existential reserve-safety events. After V6 activation (block 10,000), emergency reserve operations require ALL five constitutional defenses to be satisfied simultaneously:

Defense 1 (Purpose Restriction): actionType must be E1 (ROTATE) or E2 (LIQUIDATE→BUY→REDISTRIBUTE). No general-purpose spending. Vault SOST is never burned. Defense 2 (Dual Destination Whitelist): Target addresses are hardcoded in the contract. Adding a new valid destination requires a hard fork of the Heritage Reserve contract — no vote or key can modify the whitelist. Defense 3 (Hard Per-Spend Cap): No single transaction may move more than 2% of the vault balance at the time of execution. Defense 4 (Asymmetric Rate Limiting): GV1 (routine rotation): max 5% of balance per 30-day window. GV3 (non-routine migration): 30-day cooldown between operations. Defense 5 (95% Supermajority): Non-routine reserve migration (GV3) requires ≥95% miner supermajority signaling (W=144, YES ≥ 137).

GV2 (operational small spend ≤10%) is eliminated. Vault SOST is never burned.

Signaling window (default): Approval is measured over a window of 24 hours, equivalent to W = 144 blocks with a target of 600s per block (10 minutes). Supermajority (95%): The emergency shall be considered approved if, within that window, at least 95% of blocks include the support signal (YES ≥ ceil(0.95 × W)). With the default value W = 144, this requires YES ≥ 137 signaled blocks.

Emergency actions are limited to exactly two action types: (E1) ROTATE: rotate reserve holdings to other allowlisted precious-metal tokens (including XAUt↔PAXG). (E2) LIQUIDATE→BUY→REDISTRIBUTE: liquidate precious-metal tokens to acquire SOST and redistribute it to PoPC Pool (50%) and Gold Funding Vault (50%), with a fully auditable trail. No supply is destroyed — SOST supply is immutable by construction.

During Phase I (before block 10,000), the protocol developer may pause purchases or adjust future purchase policy operationally without consensus, disclosed via batch reports. After V6 activation, such adjustments require protocol-level governance.

G10 (Transparency): Both vault addresses, all batch reports, and all relevant transaction proofs MUST be publicly published in a reproducible format.

5.3 Automated Vault Execution Architecture

The Gold Funding Vault (Gold-first) conversion pipeline is designed to be operationally automated and fully auditable, without requiring a cross-chain bridge or wrapped SOST (wSOST). No trustless cross-chain mechanism is used; instead, automation is achieved through a deterministic execution stack with public attestations at every stage.

VAULT EXECUTION PIPELINE (no bridge, no wSOST)

WATCHER (SOST chain): Monitors the Gold Funding Vault (Gold-first) balance continuously. Detects when accumulated SOST exceeds a batch threshold.

BATCHER: Creates a signed execution order when threshold is met.

EXECUTOR (CEX / OTC / DEX where feasible): Sells SOST in TWAP clips per policy (Section 5.3.1). Purchases XAUT and/or PAXG with proceeds. Deposits purchased tokens into the Heritage Reserve (sealed).

ATTESTOR: Publishes a public batch record.

TRUST MODEL: Consensus-level allocation to the Gold Funding Vault: trustless (hard-coded by validation rules). Conversions and purchases are operational and auditable. During Phase I (before block 10,000), the protocol developer may adjust execution parameters (including the default 50/50 split) and may pause purchases. After V6 activation, adjustments require protocol-level governance. Movements of previously purchased reserve assets require Emergency Catastrophe authorization (Appendix G). No bridge, no wSOST, no smart contract dependency between chains beyond the Heritage Reserve itself.

(Subject to progressive decentralization per C16.)

5.3.1 Execution Policy: TWAP Batch Sales (Operational, Non-Consensus) SOST's Gold Funding Vault accumulates SOST by consensus (25% of coinbase). Conversions from SOST into tokenized gold exposure (XAUT/PAXG) are operational actions executed by the protocol developer (Phase I) or the automated execution stack (post-V6), with the explicit goal of minimizing market impact while maximizing auditability and predictability.

TWAP (Time-Weighted Average Price) means executing a daily sell budget as many small, evenly spaced limit orders over time (e.g., across 24 hours), so the realized execution price approximates the market's average price over that window and minimizes market impact.

Objective. Convert a portion of the Gold Funding Vault's SOST into XAUT/PAXG with: - Minimal market impact (avoid large, price-disruptive sells), - Maximum traceability (verifiable logs and transaction proofs), - Rule-based execution (avoid discretionary "sell whenever we want" behavior).

TWAP Policy (WP-friendly) - Sell at most **X% of daily market volume** (typical range: 1%–3%). - Split the daily sell budget into **N clips** (e.g., 24–96 micro-orders) distributed through the day. - Use **limit orders** with explicit slippage tolerance (no blind market selling).

Recommended starting parameters (tunable operationally) - Daily sell budget: - `DailySell = min(VaultBalance × 0.25%, DailyVolume × 2%)` - Clips: - 48 clips/day (one every 30 minutes) or 96 clips/day (one every 15 minutes) - Max slippage per clip: - 0.5%–1.0% (if exceeded, the clip is canceled or postponed) - Cooldown rule: - If intraday price drawdown exceeds a threshold (example: −8% over 24h), reduce the sell budget (e.g., cut in half) or pause execution for 24 hours.

Anti-crash layer (liquidity-aware throttling) - If spreads widen or order-book liquidity degrades, sell less. - If price drops rapidly, the policy does not accelerate selling; it throttles or pauses. - If price rises strongly, the system may sell slightly more, but only within strict ceilings tied to volume and vault-balance limits.

Simple example (illustrative) “Today the exchange reports 500,000 SOST of daily volume. Our policy allows selling at most 2% of daily volume: 10,000 SOST. Instead of selling 10,000 SOST at once, the executor sells 100 clips of 100 SOST distributed throughout the day using limit orders, enforcing a 1% max slippage rule per clip.”

Audit and transparency (mandatory) Each batch record MUST include: - Execution window (day/time range), - Total SOST sold and number of clips, - Venue(s) used (exchange or OTC counterparties), - Realized TWAP/VWAP and total slippage, - Purchase proof for XAUT/PAXG, - Transaction hash proving deposit into the Heritage Reserve.

This framework exists to prevent the common criticism that a project “dumps whenever it wants.” Execution is rate-limited, rule-based, and publicly auditable.

5.4 Heritage Reserve and Dissolution

Heritage (default state, perpetual) The Heritage Reserve remains sealed by default. No outbound transfers occur in normal operation. The gold tokens remain a permanent, publicly verifiable reserve record. “Decades of collective work — converted to gold exposure — preserved.”

Dissolution (network lifecycle only) Trigger: 90 consecutive days with zero blocks (dead-man switch). A 60-day public notice window begins. The chain is declared terminated operationally (“project wind-down”).

Reserve assets may only be accessed under: (a) Emergency Catastrophe: ≥95% miner supermajority signaling + all five constitutional defenses (V6 5-defense Gold Vault model) (b) Formal Dissolution: During Phase I, declared by the protocol developer when chain is irreversibly dead. After V6 activation (block 10,000), dissolution is triggered automatically by the protocol’s dead-man switch, subject to public attestation and maximum 90-day notice period (c) Reserve Lifecycle transitions: planned custody migrations per Section 5.5, with 30-day notice

Dissolution enables access to the reserve for orderly wind-down. During Phase I, the protocol developer is responsible for distributing reserve assets transparently after the 60-day public notice window expires. After V6 activation, distribution is executed via the protocol’s automated governance mechanism.

5.4.1 Liveness Monitoring and Dissolution Process (Operational, Non-Consensus)

The protocol distinguishes between **consensus rules (immutable)** and **operational procedures (auditable)**. The “dead-man switch” is an operational process designed to provide transparency and an orderly wind-down if the chain becomes inactive.

Definitions - Chain liveness: blocks continue to be produced on the SOST network. - Chain death: 90 consecutive days with **zero blocks** (not “low activity,” literally no blocks).

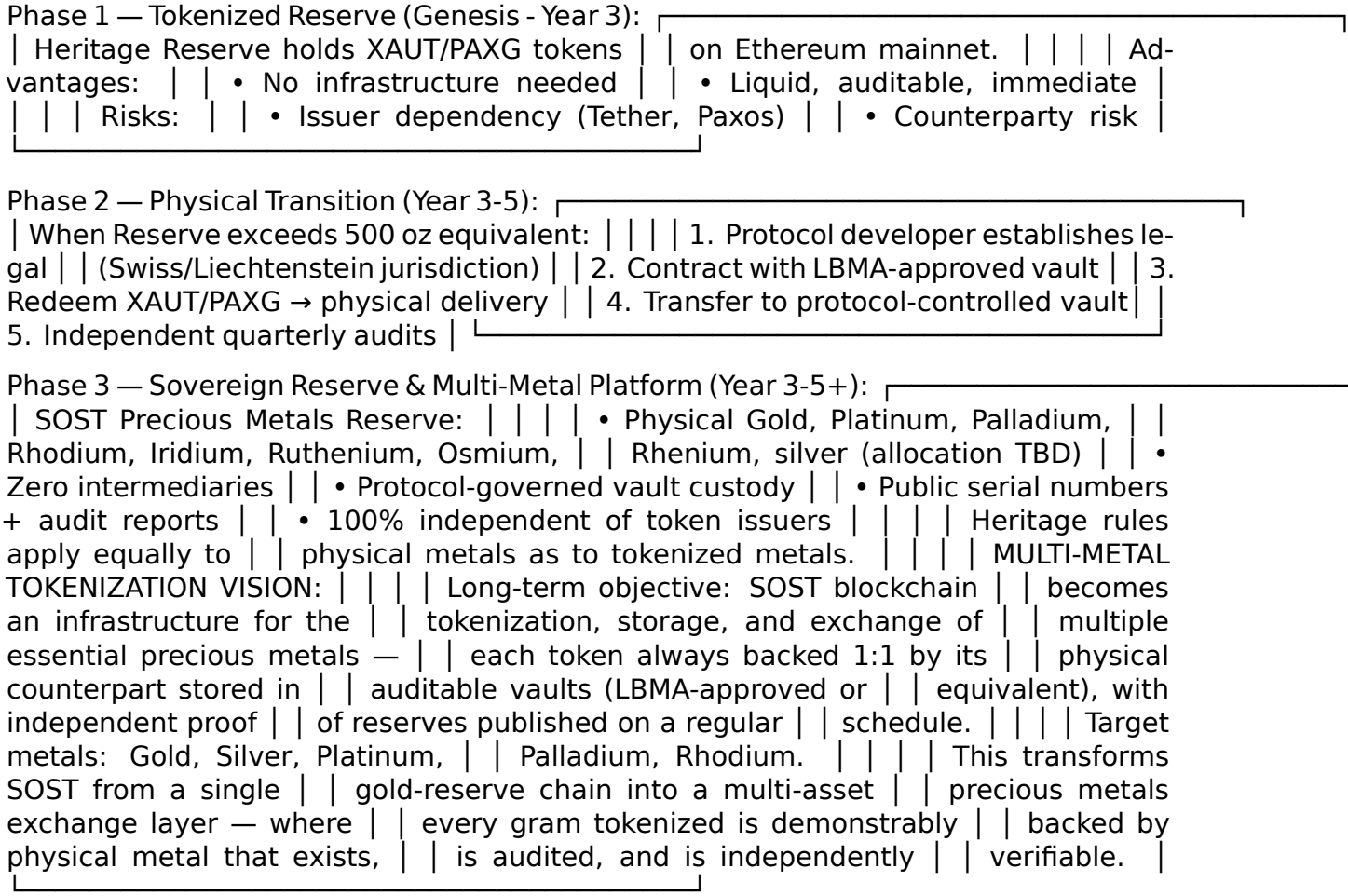
State-based timeline - **STATE 0 — Normal:** Blocks are produced normally. - **STATE 1 — Watch (30 days, zero blocks):** Publish a Liveness Alert; start recovery actions (debugging, client releases, miner outreach, incident reporting). No liquidation actions

occur. - **STATE 2 — Critical (60 days, zero blocks):** Publish a Critical Alert; prepare a full status report (root cause, repository state, known vulnerabilities, recovery attempts). No liquidation actions occur. - **STATE 3 — Dissolution Trigger (90 days, zero blocks):** Publish a Notice of Dissolution and begin a 60-day public notice window. - **STATE 4 — Execution window (after day 150):** After the notice window expires, execute final operational steps for wind-down.

Illustrative example - Day 0: last block mined. - Day 30: Watch Alert (transparency + recovery begins; no liquidation). - Day 60: Critical Alert (full incident report prepared). - Day 90: Dissolution Trigger (public notice starts; 60-day notice). - Day 150: wind-down execution (network lifecycle / operational shutdown).

Important constraint Dissolution enables orderly access to reserve assets for wind-down and distribution. If the chain dies, the 90-day dead-man switch triggers, followed by a 60-day public notice period. During Phase I, the protocol developer declares Formal Dissolution and executes the wind-down. After V6 activation (block 10,000), dissolution and reserve distribution are governed by the protocol's automated mechanisms.

5.5 Reserve Evolution Path



6. Proof of Personal Custody (PoPC)

No consensus changes. All PoPC logic remains operational/application-layer except that the PoPC Pool receives 25% coinbase by consensus.

```
IMPLEMENTATION STATUS:

  ✔ Implemented in SOST node software (C++)
  ✔ Documented publicly (whitepaper, website, repository)
  ✔ RPC interfaces ready (popc_register, escrow_register, etc.)
  ✘ Not yet active on mainnet
  ✘ Ethereum escrow contracts: pending deployment

  Activation is protocol-scheduled after block 10,000 and subject to
  network readiness (see Section 6.13 — Activation Conditions).
```

6.1 What PoPC Is — And Is Not

PoPC is not a tokenization of gold. It does not promise yield, stability, or redeemability. It is a voluntary time-bound commitment protocol with automatic economic consequences. A participant promises to maintain custody of precious metals for a defined period, backs that promise with a SOST bond, and is rewarded if they keep their word. If they break it, their bond is slashed automatically — no human judge involved.

6.2 Why It Is Unique

Protocol	What it actually does
MakerDAO	Deposit ETH (volatile crypto) → receive DAI. Collateral is crypto. Goal: price stability.
Compound/Aave	Deposit crypto → borrow crypto. All digital. No physical world connection.
Any staking protocol	Lock token X → earn token X. Circular. No external asset backing.
SOST PoPC	Lock SOST bond → prove custody of real gold (XAUT/PAXG). Audited by PoW entropy. No custodian. No bridge.

6.3 ConvergenceX Entropy for Audits

No party — not the protocol, not any server — decides when a user gets audited. The schedule is derived deterministically from ConvergenceX block entropy:

```
AUDIT SEED (per block):
  seed = SHA256(block_id || commit || checkpoints_root)

  This uses the full entropy triple from the ConvergenceX proof.
```

```
  All three components depend on the complete 100,000-round computation.

AUDIT TRIGGER (per contract, per period):
  r = PRF(seed, contractId, periodIndex)
  if r < p(reputation_stars):
      AUDIT REQUIRED

AUDIT PROBABILITY BY REPUTATION:
  0 stars (new):     p = 0.30  (30% of periods audited)
  1 star:            p = 0.20
  3 stars:           p = 0.10
  5 stars (veteran): p = 0.05  (5% of periods audited)

VERIFICATION (script, no humans):
  1. User signs message with Ethereum wallet
  2. ecrecover() confirms control of the declared wallet (no trust needed)
  3. Verifier uses multiple independent RPC endpoints with a
     deterministic retry schedule; results are logged and attested:
     token.balanceOf(user_wallet) >= committed_amount
  4. EOA VERIFICATION: the declared wallet MUST be an Externally Owned
     Account (extcodesize == 0). Smart contract wallets, multisig wallets,
     and proxy contracts are not eligible for Model A. EOA status is
     verified at contract creation AND at every subsequent audit. If code
     is detected at the wallet address during any audit (e.g., via CREATE2
     deployment after commitment), the contract is automatically slashed.
  5. CONTINUOUS CUSTODY CHECK: the audit verifies not only the balance at
     the audit block, but also samples historical balances at deterministic
     checkpoints (derived from ConvergenceX entropy) across the period since
     the last audit. If the balance dropped below the committed amount at
     any sampled checkpoint, this constitutes a custody violation and
     triggers a slash — even if the balance was restored before the audit.
  6. If all checks pass: checkpoint passed
  7. If any check fails: SLASH
  6. If no response within 48h: SLASH. The 48h GRACE period is the sole
     window. Redundant RPC checks confirm availability before slash
     execution. No additional grace periods beyond 48h.
```

6.4 Model A — Commitment with SOST Bond (Autocustody)

```
Model A: For crypto natives, miners, traders

1. User buys SOST on CEX
2. Declares Ethereum wallet holding XAUT/PAXG
3. Locks SOST bond (dynamic, based on ratio SOST/gold)
4. Commits to maintain custody for 1/3/6/9/12 months
5. ConvergenceX entropy schedules random audits
6. Script verifies XAUT/PAXG balance via Ethereum RPC
7. On completion: recovers bond + reward from PoPC Pool
```

```
8. On failure: bond slashed automatically

In Model A, the gold NEVER leaves the user's wallet.
Your gold never leaves your wallet. Only the posted SOST bond
is at risk. The protocol has no access
to, custody of, or control over the user's gold tokens.

Trust required: MEDIUM (keeps gold in own wallet, only SOST bond at risk)
```

6.5 Dynamic Bond — Model A

All fiat-denominated values use USD as the reference currency for documentation. The protocol is currency-agnostic — bond sizing depends on the SOST/gold ratio, not on any specific fiat currency.

The bond is always denominated in fiat value, converted to SOST at market price (7-day TWAP). As SOST appreciates relative to gold, the bond percentage increases gradually to maintain deterrence. The bond is designed to create deterrence and alignment, not to fully collateralize the committed asset. Sub-collateralization is intentional and compensated by layered deterrence (bond + reputation + blacklist + size limits).

```
DYNAMIC BOND CALCULATION:

  ratio = sost_price / gold_oz_price

  ratio <  0.0001              →  bond = 12%  of gold value
  0.0001 <= ratio < 0.001     →  bond = 15%
  0.001  <= ratio < 0.01      →  bond = 20%
  0.01   <= ratio < 0.1       →  bond = 25%
  0.1    <= ratio < 0.2       →  bond = 26%
  0.2    <= ratio < 0.3       →  bond = 27%
  0.3    <= ratio < 0.4       →  bond = 28%
  0.4    <= ratio < 0.5       →  bond = 29%
  ratio >= 0.5                →  bond = 30%  (maximum)

  bond_sost = bond_fiat / sost_price_TWAP_7d

RATIO THRESHOLDS (price-independent):

  The system uses RATIOS, not fixed prices. The ratio automatically
  adjusts to any gold or SOST price. Examples:
```

Gold price	SOST price	Ratio	Bond %
$2,700	$1	0.00037	15%
$2,700	$100	0.037	25%
$2,700	$1,350	0.50	30%
$5,000	$185	0.037	25%

$5,000	$2,500	0.50	30%

Note: The ratio determines the bond %, regardless of absolute prices. If gold doubles in price, the SOST threshold for each bond tier also doubles proportionally. All examples use USD; the system is ratio-based and currency-agnostic.

EXAMPLES (assuming 1oz gold = $2,700 at time of contract):

Example 1: SOST = $1 (early stage)
ratio = $1 / $2,700 = 0.00037
0.00037 < 0.001 → bond = 15%
bond_usd = 15% × $2,700 = $405
bond_sost = $405 / $1 = 405 SOST

Example 2: SOST = $100 (growth stage)
ratio = $100 / $2,700 = 0.037
0.037 < 0.1 → bond = 25%
bond_usd = 25% × $2,700 = $675
bond_sost = $675 / $100 = 6.75 SOST

Example 3: SOST = $500 (mature stage)
ratio = $500 / $2,700 = 0.185
0.185 < 0.2 → bond = 26%
bond_usd = 26% × $2,700 = $702
bond_sost = $702 / $500 = 1.40 SOST

Example 4: SOST = $1,000 (high value)
ratio = $1,000 / $2,700 = 0.37
0.37 < 0.4 → bond = 28%
bond_usd = 28% × $2,700 = $756
bond_sost = $756 / $1,000 = 0.756 SOST

Example 5: SOST = $2,000 (premium)
ratio = $2,000 / $2,700 = 0.74
0.74 >= 0.5 → bond = 30%
bond_usd = 30% × $2,700 = $810
bond_sost = $810 / $2,000 = 0.405 SOST

DESIGN RATIONALE:
- Gradual scale (25% → 26% → 27% → 28% → 29% → 30%) avoids sudden jumps
- Maximum bond (30%) only reached when ratio >= 0.5 (SOST worth half an oz)
- The "pain" of losing the bond remains proportional at any price level
- Low SOST price → many SOST deposited (but low fiat value)
- High SOST price → few SOST deposited (but high fiat value)
- System is RATIO-based, not price-based: works at any gold/SOST price

```
REPUTATION LIMITS (max oz per contract):
  0 stars: max 0.5 oz   |  3 stars: max 3 oz
  1 star:  max 1 oz     |  5 stars: max 10 oz

REPUTATION TIERS (4 discrete levels):
```

Level	0 stars (new)	1 star (established)	3 stars (trusted)	5 stars (veteran)
Max oz	0.5 oz	1 oz	3 oz	10 oz
Audit	30%	20%	10%	5%

```
  Progression: 0→1 after 1 successful contract, 1→3 after 3 cumulative
  successful contracts, 3→5 after 5 cumulative successful contracts.
  Any slash resets reputation to 0 stars.

REPUTATION SCOPE:
  Reputation is wallet-level. Each completed contract contributes to the
  wallet's reputation score. Stars represent the wallet's cumulative track
   record, not individual contract performance.
```

6.6 Price Bulletin and Bond Consent (Normative)

Bond sizing requires a market reference for SOST and gold prices. To avoid on-chain oracle dependency and discretionary pricing, PoPC uses a **Price Bulletin** model with explicit user consent:

```
PRICE BULLETIN (off-chain, publicly auditable):

  Published daily by the protocol developer (Phase I) / automated oracle (post-
V6):
    bulletin = {
      day:              YYYY-MM-DD,
      sost_usd_twap_7d: P_sost,
      gold_usd_twap_7d: P_gold,
      sources:          [exchange feeds / API references — includes all
                        exchanges where SOST has verifiable liquidity at
                         publication time, with public API endpoints. No
                         fixed minimum or maximum number of exchanges;
                        the list adapts as SOST is listed on or delisted
                         from trading venues. Each source must use a
                         publicly accessible and reproducible API],
      method:           "TWAP_7d",
      bulletin_hash:    SHA256(canonical_json(bulletin)),
     developer_sig:    EIP-712_sign(developer_key, bulletin_typed_data)
    }
```

```
  Publication: website + GitHub + IPFS mirror (redundant, immutable).
  The bulletin is informational. It does not directly control any contract.

BOND TERMS (client-side calculation + user consent):

  Given user commitment (gold_amt, duration, model):
    gold_value_fiat = gold_amt × P_gold
    ratio           = P_sost / P_gold
    bond_pct        = constitutional_table(ratio)
    bond_fiat       = bond_pct × gold_value_fiat
    bond_sost       = bond_fiat / P_sost

  The client constructs a Bond Terms package:
    terms = {
      model, user_wallet, gold_token, gold_amt, duration,
      bulletin_hash, P_sost, P_gold, ratio, bond_pct, bond_sost,
      nonce, expiry
    }
    terms_hash = keccak256(encode(terms))
    user_sig   = ECDSA_sign(user_key, terms_hash)

ON-CHAIN FINALIZATION (no oracle, no daily updates):

  The PoPC contract accepts (terms + developer_sig + user_sig) and verifies:
    1. EIP-712_recover(developer_sig, terms_typed_data) == developer_key
    2. ecrecover(user_sig, terms_hash) == user_wallet
    3. terms.expiry >= block.timestamp  (freshness / anti-replay)
    4. bond_pct matches constitutional table for provided ratio
    5. bond_sost transferred from user and locked

  Once accepted:
    - Contract stores bond_sost and commitment parameters.
    - Contract does NOT store live prices.
    - Contract does NOT support price updates.
    - No protocol action required after finalization.

PRICE VOLATILITY DURING CONTRACT:

  Prices are FROZEN at contract creation (bulletin of entry day).
  Multi-month price oscillation is absorbed by design:
    - Model A: bond is locked SOST; gold stays in user wallet.
      Volatility does not affect the contract mechanics.
    - Model B: SOST payout is calculated and delivered upfront
      at entry price. No repricing during the term.
  This eliminates oracle risk and discretionary pricing entirely.

DEVELOPER SIGNATURE (EIP-712 typed data — Phase I):
```

```
  Terms (including P_sost, P_gold, bond_pct, and reward) are signed by an
  authorized developer key using EIP-712 typed data signatures. The smart
  contract verifies the developer signature before accepting any commitment.
  Users cannot submit self-signed terms. After V6 activation (block 10,000),
    this signing role transitions to an automated oracle with on-
chain verification.

SECURITY PROPERTIES:
  - No on-chain oracle: pricing is not consensus-critical.
  - No discretionary pricing: the protocol developer cannot set a user's bond.
    - Dual consent: every contract requires both a developer EIP-
712 signature
    on the terms and a user signature. Neither party can act unilaterally.
  - Auditability: any party can reproduce bond sizing from bulletin + terms.
  - Permissionless verification: bulletin data + sources are public.
  - Developer-attested (Phase I): the PoPC contract verifies the developer's EIP-
712
    signature on terms before accepting any commitment. Independent parties
    may audit bulletin data, but on-chain terms require developer attestation.
     Post-V6, attestation transitions to automated oracle.
  - Malicious bulletin protection: because both the developer must sign the
     terms and the user must explicitly co-sign, neither party can impose
    pricing unilaterally. A malicious or incorrect bulletin simply results in
    participants refusing to co-sign contracts based on it. No funds are at
     risk from a bad bulletin.

(Subject to progressive decentralization per C16.)
```

6.7 Model A Rewards

Note: Reward rates are operational reference parameters, not guaranteed. They may be adjusted, budgeted per epoch, or paused for security without changing consensus rules. They do not constitute interest, promised yield, or a financial obligation of the protocol.

Rewards are denominated in SOST (percentage of bond). This ensures the PoPC Pool can always pay regardless of SOST price. Longer commitments receive proportionally higher reward rates to incentivize long-term participation. Bond is fully recoverable upon successful completion.

Duration	Reward (% of bond)	Reference annualized rate (non-guaranteed)
1 month	1% of bond in SOST	12% annualized
3 months	4% of bond in SOST	16% annualized
6 months	9% of bond in SOST	18% annualized
9 months	14% of bond in SOST	18.7% annualized
12 months	20% of bond in SOST	20% annualized

Rates are base maximums at Tier 1 (first 25 active contracts). A dynamic participation tier system reduces effective rates as the network grows:

DYNAMIC PARTICIPATION TIERS:

Tier	Active Contr.	Multiplier	A 12mo	B 12mo
EARLY ADOPTER	0 – 25	100%	20%	8%
GROWTH	26 – 50	75%	15%	6%
ESTABLISHED	51 – 100	50%	10%	4%
MATURE	101 – 200	30%	6%	2.4%
SCALE	201 – 500	15%	3%	1.2%
MASS	501 – 1,000	8%	1.6%	0.64%

HARD CAPS: Max reward per contract: 1,000 SOST (absolute ceiling) Max active contracts: 1,000 (no new registrations beyond this) Max gold per contract: 200 oz (anti-whale hard cap; >10 oz = reduced rates) Reward floor: 10 SOST (Model A) / 5 SOST (Model B) Protocol fee: 3% Model A (risk-taker discount) / 8% Model B (zero-risk premium) Pool protection: PUR quadratic curve $(1\text{-}PUR)^2$ + solvency check at registration

```
EXAMPLE: Model A — 12 month commitment

  Assumptions at contract creation:
    - Gold price: $2,700/oz (from Price Bulletin)
    - SOST price: $100 (from Price Bulletin)
    - User commits: 1 oz XAUT in their own wallet

  STEP 1: Calculate ratio and bond
    ratio = $100 / $2,700 = 0.037
    0.037 < 0.1 → bond = 25%
    bond_usd = 25% × $2,700 = $675
    bond_sost = $675 / $100 = 6.75 SOST

  STEP 2: User deposits 6.75 SOST as bond

  STEP 3: User maintains gold custody for 12 months
    - Random audits verify XAUT balance
    - Gold NEVER leaves user's wallet

  STEP 4: On successful completion
    - Recovers: 6.75 SOST (original bond — 100% returned)
    - Reward: 22% × 6.75 = 1.485 SOST
    - TOTAL: 8.235 SOST

  PROFIT (at original prices):
    - Deposited: 6.75 SOST
    - Received: 8.235 SOST
    - Net gain: 1.485 SOST (20% return on bond)
    - Gold: Still owns 1 oz XAUT

EXAMPLE: Model A — Slash (failure)
```

```
  Same setup as above, but user sells their XAUT during contract.

  Audit detects: balanceOf(user_wallet) < 1 oz

  SLASH TRIGGERED:
    - User loses: 6.75 SOST (entire bond)
    - Distribution:
      • 3.375 SOST → PoPC Pool (funds future rewards)
      • 3.375 SOST → Gold Funding Vault (buys more gold)
    - User's reputation: destroyed
    - User's address: blacklisted
```

6.7.1 Protocol Fee — Model A

A protocol fee of 3% (Model A) or 8% (Model B) is applied to the reward. The fee is calcu
creation and locked; disbursement occurs at successful completion. The user's bond
is returned in full regardless.

```
  base_reward    = bond_sost × reward_table[duration]
 protocol_fee  = base_reward × fee_rate  // 0.03 (Model A) or 0.08 (Model B)
  user_reward    = base_reward - protocol_fee

  Fee is locked at creation (immutable for the contract's lifetime).
  Disbursement at completion (single atomic operation):
    user receives:       bond_sost + user_reward
    protocol fee address receives: protocol_fee

EXAMPLE (12-month, same assumptions as Section 6.7):
  Base reward (22%):     6.75 × 0.22 = 1.485 SOST
  Protocol fee (3%):     1.485 × 0.03 = 0.045 SOST
  User receives:         6.75 + 1.411 = 8.161 SOST
  Protocol fee address receives: 0.074 SOST
  User effective rate:   20.9% (vs 22% nominal, non-guaranteed)

Fee destination: protocol fee address (publicly auditable).
Fee purpose: infrastructure, watcher operation, bulletin publication,
exchange execution, security audits, development.

Fee changes require 30-day advance public notice and apply only to
NEW contracts. Existing contracts retain the fee rate locked at creation.
```

6.8 Model B — Timelocked Escrow, No Audits

Model B: For holders of XAUT/PAXG — precious metal investors

1. User deposits XAUT/PAXG into an immutable escrow timelock contract (non-custodial, deployed on EVM).
2. The escrow contract has NO admin key, NO upgrade proxy, NO pause function. During the term, it cannot transfer deposited gold tokens to any third party —

including the protocol.

3. User receives SOST immediately from PoPC Pool: SOST_reward = gold_value × rate × (duration/12) / sost_price The SOST reward is delivered to the user's wallet at deposit time. This is the compensation for surrendering gold liquidity.
4. At expiry: ONLY the original depositor can withdraw full XAUT/PAXG.
5. SOST received are the user's to keep immediately. No return required. The user may sell, hold, or use the SOST freely from day 1.
6. No audits, no slash risk, and no bond requirement. Gold is enforced by escrow, not by trust.

NO EARLY EXIT (Model B — gold escrow): Users cannot unlock escrowed gold before maturity. The gold stays locked for the full term. However, the SOST reward is delivered immediately at deposit — the user has full liquidity of the SOST from day 1. This is the fundamental trade: gold liquidity for SOST liquidity. "Unvested reward forfeiture" clauses do not apply to Model B because there is nothing to forfeit — SOST is already paid.

ESCROW ARCHITECTURE: - Contract type: immutable (no proxy, no UUPS, no admin) - Functions: deposit(token, amount, unlockTime) + withdraw(depositId) - withdraw() callable ONLY by original depositor - withdraw() reverts if block.timestamp < unlockTime - No Protocol key, no multisig, no emergency withdrawal - NO EXTENSION: the escrow term is immutable once created. There is no renew(), extend(), or modify() function. To commit again after maturity, the user must create a new contract at current terms. - Source code: published and verified on explorer. Independent audit planned before mainnet activation of Model B.

FOUNDATION ROLE IN MODEL B: - The protocol does NOT custody, control, or access escrowed gold. - The protocol operates the watcher that detects deposits and triggers immediate SOST payout from PoPC Pool to the depositor. - Escrowed gold tokens are NOT protocol assets. They are recorded as off-balance-sheet commitments for transparency purposes only. - The SOST distribution is an incentive for participation in the PoPC program and does not constitute interest, yield, lending return, or a financial obligation.

ESCROW DEPLOYMENT: - The escrow contract address is immutable and publicly announced before PoPC activation. Source code is published and verified on the block explorer prior to any deposits being accepted.

WATCHER TRUST MODEL: - The watcher is an operational component, not a consensus mechanism. - The watcher CAN delay SOST payouts (liveness risk). - The watcher CANNOT steal, move, or access escrowed gold (safety). - The watcher CANNOT prevent withdrawal at expiry (escrow is autonomous). - If the watcher fails to pay, the user retains full gold + escrow proof. - AUTONOMOUS CLAIM FALLBACK: if the watcher fails to deliver the SOST reward within 72 hours of deposit confirmation, the depositor may invoke claimReward(depositId) on a publicly deployed claims contract. The claims contract verifies the escrow deposit on-chain (event log + deposit parameters) and disburses the NET reward (after protocol fee) directly from the PoPC Pool, bypassing the watcher entirely. Fee routing: protocol_fee is routed to the protocol fee address in the same atomic transaction as the user payout. The watcher is a convenience for instant payouts; the autonomous claim is the trustless fallback that guarantees de-

livery regardless of developer liveness. - IDEMPOTENCY: the claims contract enforces one claim per depositId. Duplicate invocations revert. All claims are recorded on-chain and included in the public audit record. Any discrepancy is a constitutional violation. - Watcher source code and payout logs are published for independent verification. Any discrepancy is a constitutional violation (G-rules).

Trust required: MEDIUM (non-custodial escrow; risks: smart contract, token issuer XAUT/PAXG, EVM chain, and SOST market price).

(Subject to progressive decentralization per C16.)

6.9 Model B Rewards by Duration

Note: Reward rates are operational reference parameters, not guaranteed. They may be a

Reward-rate parameters (non-guaranteed), expressed as annualized reference rates for

Duration	Reward rate (reference)	Total Reward
1 month	0.4% of gold value	paid immediately in SOST
3 months	1.5% of gold value	paid immediately in SOST
6 months	3.5% of gold value	paid immediately in SOST
9 months	5.5% of gold value	paid immediately in SOST
12 months	8% of gold value	paid immediately in SOST

Same dynamic participation tiers apply as Model A (100%/75%/50%/30%/15%/8%).
Hard cap: 1,000 SOST max per contract.

CALCULATION:

total_reward_pct = rate × (duration_months / 12)

SOST_reward = gold_value × total_reward_pct / sost_price

EXAMPLE: Model B — 12 month commitment

Assumptions at contract creation: - Gold price: $2,700/oz (from Price Bulletin) - SOST price: $100 (from Price Bulletin) - User deposits: 1 oz XAUT into escrow - Duration: 12 months - Reward rate (reference): 7%

STEP 1: Calculate reward total_reward_pct = 7% × (12/12) = 7% reward_usd = $2,700 × 7% = $189 reward_sost = $189 / $100 = 1.89 SOST

STEP 2: User receives 1.89 SOST IMMEDIATELY - Can sell, hold, or use from day 1 - No conditions attached

STEP 3: Gold locked for 12 months - Cannot be withdrawn early - No audits required (escrow enforces)

STEP 4: After 12 months - User withdraws full 1 oz XAUT - Gold returned intact

SUMMARY: - Locked: 1 oz gold for 12 months - Received: 1.89 SOST immediately - Returned: 1 oz gold at maturity - Net gain: 1.89 SOST (7% on gold value at entry)

EXAMPLE: Model B — 3 month commitment

Assumptions at contract creation: - Gold price: $2,700/oz (from Price Bulletin) - SOST price: $100 (from Price Bulletin) - User deposits: 1 oz XAUT into escrow - Duration: 3 months - Reward rate (reference): 1%

CALCULATION: total_reward_pct = 1% × (3/12) = 0.25% reward_usd = $2,700 × 0.25% = $6.75 reward_sost = $6.75 / $100 = 0.0675 SOST

SUMMARY: - Locked: 1 oz gold for 3 months - Received: 0.0675 SOST immediately - Returned: 1 oz gold at maturity

MODEL A vs MODEL B COMPARISON:

Aspect	Model A	Model B
What you lock	SOST (bond)	Gold (escrow)
Your gold	In YOUR wallet	In escrow
Bond required	Yes (12-30%)	No
Reward timing	At end if pass	Immediately
Risk	Lose bond	None (no slash)
Rate (12 month)	22%	7%
Best for	Crypto natives	Gold holders

6.9.1 Protocol Fee — Model B

A protocol fee of 8% is collected upfront at commitment creation. Model B has a higher fee because participants bear no slash risk and the protocol developer assumes greater operational burden (watcher, immediate payout, escrow monitoring).

base_reward = gold_value × rate × (duration/12) / sost_price protocol_fee = base_reward × 0.08 user_reward = base_reward - protocol_fee

EXAMPLE (12-month, 1 oz XAUT, same assumptions as Section 6.9): Base reward (7%): 1.89 SOST Protocol fee (8%): 0.151 SOST User receives: 1.739 SOST immediately Protocol fee address receives: 0.151 SOST User effective rate: 6.3% (vs 7% nominal, non-guaranteed)

Same operational rules apply: fee destination auditable, 30-day notice for changes, existing contracts locked at creation fee rate.

6.10 Slash Mechanics

SLASH TRIGGER (Model A only): - XAUT/PAXG balance < committed amount at audit time, OR - No response to audit after 48h GRACE period (sole window; confirmed RPC availability)

SLASH SPLIT (automatic, no human override): 50% → PoPC Pool (funds future rewards) 50% → Gold Funding Vault (buys more gold collateral)

No supply is burned. With a maximum supply of ~4.67M SOST, every token has long-term utility. Slashed bonds are recycled into the two constitutional vaults, strengthening both the reward ecosystem and the gold reserve. Fraud does not destroy value — it redistributes it to honest participants and the reserve.

Every fraud detected strengthens the protocol: half funds future honest participants, half buys more gold. No supply is destroyed — it is recycled where it creates the most value.

6.11 [Reserved — See Appendix M]

Section 6.11 previously described an "Adaptive Mode" where PoPC would accept SOST as a custody asset when the SOST/gold ratio exceeded 1.5. This mode has been removed from the normative specification because it contradicts the core purpose of PoPC as a proof of gold custody. Accepting SOST as a custody asset would create circular staking dynamics incompatible with the protocol's design. See Appendix M for the original text, retained for historical completeness.

6.12 PoPC Pool Sustainability

```
POOL AVAILABLE (PoPC Pool receives 25% of every block):
  Year 1:  103,160 SOST
  Year 2:  103,160 SOST
  Year 5:   ~75,000 SOST
  Year 10:  ~52,000 SOST
  30 years: 1,167,300 SOST total

SCENARIO: 200 active contracts, mix of durations:
  Annual rewards paid: ~6,000 SOST (with conservative rates)
  Pool used: 6,000 / 103,160 = 5.8% of annual intake
  Accrual: 97,160 SOST surplus per year → reserves grow

SCENARIO: 1,000 contracts (optimistic):
  Annual rewards: ~30,000 SOST
  Pool used: 29% of annual intake
  Still sustainable indefinitely

SLASH INCOME extends pool beyond 30 years:
  5% failure rate, 200 contracts, 200 SOST avg bond:
  Pool receives: 10 × 200 × 50% = 1,000 SOST/year extra
  Gold Funding Vault receives: 10 × 200 × 50% = 1,000 SOST/year extra

REWARD BUDGET + SOLVENCY SAFEGUARDS (non-consensus, operational safety valves):

  Per-wallet cap (per epoch):
    Limits the maximum rewards any single wallet can receive within
    a given epoch. Prevents whale concentration of pool resources.

  Global cap (per epoch):
    Limits total rewards disbursed by the PoPC Pool within a given
    epoch (reward budget). Ensures long-term pool sustainability.

  Pool solvency rule:
    A new PoPC contract can only be opened if the PoPC Pool has
    sufficient balance to cover the full reward amount at creation.
   - Model A: fee calculated at creation; reward + fee disbursed at completion
    - Model B: reward is paid immediately from pool at deposit
```

```
    If the pool is underfunded, new contracts are temporarily paused
    until replenished (via block inflows or slash income).

  No debt rule:
    The protocol never creates unfunded obligations; contracts cannot
    be opened unless the reward can be covered at inception.

  Existing contracts:
    Active contracts continue unaffected until expiry regardless of
    pool state. Pauses affect only new contract openings and do not
    alter consensus rules or existing matured withdrawal rights.

ANTI-WHALE CAPS (Model A):

  Per-wallet epoch cap: 50 oz maximum per wallet per epoch
  Global cap: 10% of PoPC Pool balance per epoch

  The per-wallet epoch cap is an absolute ceiling. The reputation
  per-contract limit (0.5/1/3/10 oz) applies per individual contract.
  A 5-star wallet may open multiple contracts up to the epoch cap:
    e.g., 5 × 10 oz = 50 oz across 5 contracts in one epoch.
  A 0-star wallet is limited to 0.5 oz per contract AND 50 oz per
  epoch — effectively 0.5 oz since reputation constrains first.

  Rationale: Model A includes slash risk, which naturally
  limits demand and returns 50% of slashed bonds to pool.

ANTI-WHALE CAPS (Model B):

  Per-wallet epoch cap: 10 oz maximum per wallet per epoch
  Global cap: 5% of PoPC Pool balance per epoch

  Same interaction: reputation per-contract limits apply first,
  then the epoch cap limits total activity. A 5-star wallet can
  commit up to 10 oz/epoch (e.g., 1 × 10 oz), while a 0-star
  wallet is limited to 0.5 oz/contract (reputation-constrained).

  Rationale: Model B has no slash risk, making it more
  attractive. Tighter caps protect pool sustainability.

EPOCH DEFINITION:

  1 epoch = 1 month = 4,320 blocks (at 10-minute target)

  Caps reset at the start of each epoch.
  Unused cap allocation does not roll over.

SUSTAINABILITY EXAMPLE (Year 1):
```

```
  Pool balance: 50,000 SOST

  Model B monthly cap: 5% × 50,000 = 2,500 SOST
  Model A monthly cap: 10% × 50,000 = 5,000 SOST
  Maximum combined outflow: 7,500 SOST/month

  Monthly pool income: ~8,597 SOST (103,160 ÷ 12, from block rewards)

  Net monthly growth: 8,597 - 7,500 = +1,097 SOST

  System: sustainable indefinitely, even at maximum demand
```

6.13 Activation Conditions

PoPC will be activated once ALL of the following technical and network conditions are met. Reaching block 10,000 is a necessary condition, but activation will only proceed once the system is considered stable and all required components are fully operational.

```
ACTIVATION CONDITIONS (all must be satisfied):

  1. Block height ≥ 10,000
     Activation is tied to a protocol milestone and will not occur before this threshold

  2. Sustained network stability
     Consistent block production, stable sync across nodes, no critical consensus
       or networking issues.

  3. Sufficient node and miner participation
     Minimum level of independent nodes and active miners to ensure reliable
       operation under real network conditions.

  4. Model A contract validation
     Registration, audit, reward, and slashing flows fully validated under live
       network testing conditions.

  5. Ethereum escrow deployment (Model B)
     Final escrow contracts deployed, verified, and publicly auditable before
       Model B activation.

  6. Explorer & wallet integration readiness
       Public interfaces correctly reflect all PoPC states (pending, active,
       completed, slashed).

ACTIVATION NOTE:
  PoPC activation is protocol-scheduled but condition-dependent. The protocol develope
  will announce activation with sufficient advance notice once all conditions
   are verified. This is not a date-based commitment — it is a readiness-
```

```
based
  milestone.
```

6.14 Year 2-3: Physical Gold Extension

In Year 2, PoPC extends to physical gold (bars, coins, ingots). Users commit to custody of physical precious metals without revealing identity, location, or exact quantity. Audit challenges are derived from ConvergenceX entropy and executed via a mobile app.

```
PHYSICAL AUDIT FLOW (Year 2+):
  1. ConvergenceX block entropy selects challenge package:
     e.g. weight range + ring test + geometric measurement
  2. User performs challenges using mobile app
  3. App derives: stateHash = SHA256(extracted_features)
                  evidenceHash = SHA256(raw_evidence)
  4. Raw evidence is ephemeral (never stored anywhere)
  5. Hashes published on-chain
  6. Verifier committee (k=3, entropy-selected) signs attestation

PRIVACY GUARANTEE:
  No gold amount on-chain
  No location on-chain
  No identity on-chain
  Only cryptographic commitments and signatures
```

6.14 Year 3+: ZK Proof Extension

In Year 3, physical verification is planned to upgrade to Zero-Knowledge proofs. A ZK proof would certify that the correct challenge algorithm executed correctly — without revealing any underlying data. Trusted verifiers become unnecessary. ConvergenceX's structured, iterative, mathematical nature maps naturally to ZK circuits.

The scope of physical custody verification extends beyond gold to other precious metals including platinum, palladium, and rhodium, subject to the development of reliable challenge algorithms for each metal type.

```
ZK PHYSICAL CUSTODY — DEVELOPMENT DISCLOSURE

This feature constitutes a long-term R&D commitment, not a deliverable
for Year 1. Implementation is contingent upon the following conditions:

  1. ZK proof systems must demonstrably guarantee participant privacy
     (no leakage of identity, location, or asset quantity).
  2. Challenge algorithms must achieve sufficient reliability to
     prevent both false positives and false negatives at scale.
  3. The cryptographic primitives used must be formally audited
     and considered production-ready by independent reviewers.

If current or future technology cannot satisfy these requirements
```

```
to the standard necessary for protecting participants who commit
physical precious metals (gold, platinum, palladium, rhodium,
and other metals as added), this phase will be postponed
indefinitely until all privacy and security guarantees can be
fully met.

The protocol will not deploy physical ZK verification in a
degraded or partial state. Participants' physical security is
a non-negotiable prerequisite — not a trade-off.
```

6.9 PoPC Contract Lifecycle — Implementation Reference

This section documents the implemented PoPC contract lifecycle as of v0.3.2.

MODEL A — AUTOCUSTODY (BOND + AUDIT):

Step 1: CALCULATE BOND - User provides: Ethereum address, gold amount (oz), commitment duration - System computes: bond_pct = compute_bond_pct(sost_price/gold_price ratio) - Current ratio yields 25% bond (SOST=$1, gold=$10,000/oz) - Bond amount = gold_value_in_sost × bond_pct

Step 2: REGISTER COMMITMENT - RPC: popc_register(sost_address, eth_address, gold_token, gold_mg, months) - System calculates dynamic reward based on Pool Utilization Ratio (PUR) - Reward rate = base_rate × (1 - PUR)^2 (quadratic decay) - Anti-whale tiers: 0-10oz=100%, 10-50oz=75%, 50-200oz=50%, >200oz=rejected - Reward is RESERVED in the PoPC Pool (guaranteed if commitment succeeds) - Returns: commitment_id, bond_required, expected_reward

Step 3: LOCK BOND - User creates BOND_LOCK (0x10) transaction: sost-cli create-bond - Bond is time-locked on-chain (cannot be spent until commitment period ends) - This is a consensus-level lock enforced by rule S11

Step 4: MAINTAIN CUSTODY - User keeps XAUT/PAXG in their Ethereum wallet for the full period - Random audits derived from: SHA256(block_id || commit || checkpoints_root) - Audit probability: 30% (new), 20% (established), 10% (trusted), 5% (veteran) - Automated Etherscan checker verifies balances

Step 5: COMPLETION OR SLASH - Success: popc_release → bond returned + reward from pool - Failure: popc_slash → bond confiscated (50% pool, 50% gold vault)

Maximum reward rates (Model A — % of bond): 1 month: 1% | 3 months: 4% | 6 months: 9% 9 months: 14% | 12 months: 20%

MODEL B — ESCROW (DEPOSIT + IMMEDIATE REWARD):

Step 1: DEPOSIT GOLD IN ESCROW - User sends XAUT/PAXG to immutable Ethereum escrow contract - Contract: no admin key, no upgrade proxy, no pause function - Only the original depositor can withdraw after timelock expires

Step 2: REGISTER ON SOST - RPC: escrow_register(sost_address, escrow_contract, gold_token, gold_mg, months) - System verifies escrow deposit on Ethereum - Calculates dynamic reward (same PUR curve, halved rates) - SOST reward paid IMMEDIATELY (no waiting)

Step 3: WAIT FOR LOCK EXPIRY - No audits needed (gold is provably locked in escrow) - No bond required (no slash risk)

Step 4: WITHDRAW GOLD - After timelock: user calls withdraw() on Ethereum contract - 100% of gold returned to user's wallet - Register completion on SOST: escrow_complete

Maximum reward rates (Model B — % of gold value, halved from A): 1 month: 0.5% | 3 months: 2% | 6 months: 4.5% 9 months: 7.5% | 12 months: 11%

DYNAMIC REWARD SYSTEM (PUR):

All rewards are dynamically adjusted by the Pool Utilization Ratio: PUR = committed_rewards / pool_balance factor = (1 - PUR)^2 (quadratic decay) effective_rate = base_rate × factor

When PUR is low (few participants): rewards are near maximum. When PUR is high (many participants): rewards decrease automatically. At PUR=100%: no new registrations accepted (pool fully committed).

This is a self-regulating system — no manual intervention needed.

IMPLEMENTATION: - src/popc.cpp: PoPCRegistry, compute_bond_pct, compute_reward_pct, audit entropy - src/popc_tx_builder.cpp: TX builders for release and reward - src/popc_model_b.cpp: EscrowRegistry, calculate_escrow_reward - include/sost/popc.h: Constants, PUR functions, whale tiers - RPC commands: popc_register, popc_status, popc_check, popc_release, popc_slash - RPC commands: escrow_register, escrow_status, escrow_verify, escrow_complete - Tests: 27 test suites, 616 individual tests (all passing)

---

7. Governance Model

7.1 No Ongoing Parameter Governance

No entity — including the protocol developer, or any future organization — can modify consensus rules after genesis. SOST has no DAO, no voting token, and no multisig with authority over consensus or monetary parameters. All consensus rules — emission schedule, coinbase split, difficulty algorithm, supply cap, constitutional constraints — are immutable at genesis. No mechanism exists to alter them after block 0. Only narrow emergency authorization signaling exists, requiring ≥95% miner supermajority under the V6 5-defense Gold Vault model (Section 5.2, Appendix G).

Emergency Catastrophe authorization does not modify consensus rules or monetary policy; it only gates movements of external reserve assets held on Ethereum.

7.2 Operational Governance

During Phase I (before block 10,000), the protocol developer executes operational actions under constitutional constraints and public attestations. After V6 activation (block 10,000), custody transitions from the developer to the protocol itself — there is no intermediate multisig stage.

Operational scope (exhaustive):

- **Gold Funding Vault conversion:** During Phase I, the protocol developer executes SOST→XAUT/PAXG purchases per the automated pipeline (Section 5.3). After V6, execution is automated with on-chain attestation. Every batch is attested on-chain.
- **50/50 split adjustment:** During Phase I, the protocol developer may change the XAUt/PAXG acquisition split for future purchases. After V6 activation, adjustments require protocol-level governance. Adjustments apply only to future purchases and must be disclosed in batch reports.
- **Purchase pause:** During Phase I, the protocol developer may pause purchases without consensus, disclosed via batch reports. After V6, pausing requires protocol-level governance.
- **Future purchase redirection:** During Phase I, the protocol developer may redirect future purchases from one issuer to another without consensus (e.g., if issuer risk increases). After V6, redirection requires protocol-level governance.
- **PoPC contract management:** During Phase I, the protocol developer may pause new PoPC contracts in case of critical bugs. After V6, this is governed by the protocol. Existing contracts continue unaffected.
- **Dissolution declaration:** During Phase I, the protocol developer declares dissolution only after the dead-man switch triggers (90 days of zero blocks), with 60-day public timelock. After V6, dissolution is triggered automatically by the protocol.

Moving previously purchased reserve assets requires Emergency Catastrophe authorization under the V6 5-defense Gold Vault model (≥95% miner supermajority signaling + all five constitutional defenses). No other mechanism exists. The vault is a constitutional reserve engine, not a general treasury.

No operational action can modify consensus rules, alter the emission schedule, or redirect coinbase allocations. Reserve assets may only be accessed under Emergency Catastrophe (≥95% miner supermajority + 5-defense model), formally declared Dissolution (Section 5.4), or planned Reserve Lifecycle transitions (Section 5.5).

DECENTRALIZATION COMMITMENT (C16): Every operational function described in this section — vault execution, purchase management, PoPC verification, and reporting — is designed for progressive automation and decentralization. The protocol developer holds these roles as a transitional necessity during Phase I (before block 10,000). At V6 activation, custody transitions directly from developer to protocol — no intermediate multisig. As each process is automated, tested, and independently verified, manual control is permanently relinquished. This trajectory is irreversible: decentralization only moves forward. See Section 7.3 for the full commitment.

7.3 Progressive Decentralization

The protocol is committed to progressive decentralization and full automation of all operational processes. At V6 activation (block 10,000), custody transitions directly from the protocol developer to the protocol — no intermediate multisig stage. Manual operations during Phase I — PoPC balance verification, reward payouts, vault batch

conversions, and reporting — are transitional by design, not permanent. Each process has a documented automation path: PoPC verification moves to on-chain smart contracts, reward payouts to the sost-popc-daemon, and vault conversions to a deterministic execution stack with public attestations.

This is a constitutional commitment (C16), not a discretionary goal. As each operational process is automated and independently verified, the protocol developer permanently relinquishes manual control over that process. The trajectory is irreversible: decentralization only moves forward.

8. Economic Model

8.1 Demand Sources for SOST

- **Mining:** miners earn SOST and decide to hold or sell.
- **Model A bonds:** participants must buy SOST on CEX to create bonds — direct buy pressure.
- **Model B distribution:** SOST distributed to gold holders — organic distribution to non-crypto users.
- **Metals Reserve (tokenized, allowlisted) sales:** The protocol sells SOST to buy gold — provides sell-side liquidity and price discovery.
- **Speculation:** market prices SOST partly on observable gold reserve backing.

8.2 Deflationary Mechanics

- **Smooth exponential decay:** 9.03% annual reduction in new issuance — no cliff events.
- **Slash recycling:** 100% of every slashed bond is redistributed to PoPC Pool (50%) and Gold Funding Vault (50%), strengthening both systems without destroying scarce supply.
- **Reputation limits:** large commitments require proven track record — slow unlock of supply pressure.
- **Long contracts:** 9 and 12-month commitments remove SOST from circulation for extended periods.

8.3 Reserve Ratio Metric (observable, not a peg)

As the Metals Reserve grows, the system produces a public, verifiable metric:

```
Reserve Ratio (observable metric):

  reserve_ratio = reserve_value_usd / circulating_supply
```

This is NOT a peg, NOT a guarantee, and does NOT grant redemption rights. It is simply a transparent indicator of the reserve accumulated by the protocol. Markets may price this information as they see fit.

8.4 Autoequilibrium: When SOST > Gold

If SOST price significantly exceeds gold price, three automatic corrective forces activate:

- **Gold Funding Vault force:** With expensive SOST, each vault sale buys more gold. Gold demand increases. Gold price rises. Ratio compresses.
- **Model B force:** With low SOST yield in unit terms, fewer users deposit gold. Pool accumulates. Supply pressure falls.
- **Model A force:** With SOST worth more than gold, bonding SOST to custody gold loses economic sense. Participation drops naturally.

No governance required. The economics self-regulate through participant incentives.

9. Security Analysis

9.1 51% Attack Resistance

Each ConvergenceX attempt requires ~8 GB RAM (4 GB dataset + 4 GB scratchpad) and 100,000 sequential rounds. Even an attacker with unlimited CPUs cannot speed up a single attempt — parallelism only adds more independent attempts. The sequential dependency chain is the fundamental defense: an attacker with 1,000 cores runs 1,000 parallel attempts, not 1,000× faster on a single attempt.

As mining becomes profitable, network hashrate grows naturally, continuously raising the attack threshold. The cASERT difficulty adjustment ensures that block intervals remain stable regardless of hashrate level.

9.2 Bond Deterrence Analysis

The bond does not make fraud impossible — it makes it costly. For a user with urgent liquidity needs, selling committed gold while losing a 30% bond may still be rational. The system compensates with layered deterrence:

- **Bond (30% max):** immediate financial cost.
- **Reputation destruction:** permanent record, future contracts blocked at all levels.
- **Wallet blacklist:** PoPC policy may exclude known fraudulent wallets **at the application layer**. This is not consensus and does not affect the base chain.
- **Size limits by reputation:** new users can only commit small amounts (0.5 oz max at 0 stars).

The combination of these four layers makes large-scale systematic fraud economically irrational, even if individual emergency exits remain possible.

9.3 Audit Manipulation Resistance

Audit timing is derived from ConvergenceX block entropy — specifically the entropy triple (block_id, commit, checkpoints_root). No party can predict or influence the audit schedule because all three components depend on the complete 100,000-round PoW computation that has not yet occurred.

A user who moves their gold immediately after passing an audit faces another random audit within the next period — with probability 5%-30% depending on reputation. The expected window for undetected fraud is short, and the cost of detection (full bond slash) is high.

9.4 Oracle Manipulation Resistance

No external oracle is required for consensus-critical validation. All consensus operations (block validation, emission, difficulty) use only chain-internal data.

Bond calculations use a 7-day TWAP from exchanges with verifiable liquidity (consistent with the Price Bulletin model in Section 6.6) — resistant to single-block manipulation. The gold ratio check uses the same TWAP. Price data is advisory for PoPC bond sizing and settlement, not consensus-critical.

9.5 cASERT Security Properties

The asymmetric policy (Section 3.12) prevents cASERT from being weaponized:

- An attacker with high hashrate mining blocks quickly pushes the signal negative (chain fast).
- Negative signal → cASERT stays in NORMAL mode. No relaxation.
- The attacker gains nothing — stability requirements remain at maximum.

cASERT only relaxes when the chain is genuinely slow (hashrate dropped), which is when relaxation is needed to maintain liveness. This is the correct security trade-off: relaxed stability is only available when the chain needs help, never when an attacker is dominating.

9.6 Emergency Procedures

Scenario	Response
Critical bug	During Phase I, the protocol developer pauses new PoPC contracts. After V6, pausing is governed by the protocol. Existing contracts continue unaffected.
Exchange hack	The Heritage Reserve on Ethereum operates as a sealed vault with no operational interaction. The Gold Funding Vault on SOST chain interacts with execution venues via the conversion pipeline but is not a hot wallet — during Phase I, all operations require developer authorization; after V6, operations are protocol-governed.

Scenario	Response
XAUT/PAXG issuer risk	Emergency Catastrophe: rotate to allowlisted metals (E1) OR liquidate-buy-redistribute (E2). Requires ≥95% miner supermajority signaling + all five constitutional defenses under the V6 5-defense Gold Vault model.
Issuer risk (future purchases only)	During Phase I, the protocol developer redirects future purchases without consensus. After V6, redirection requires protocol-level governance. Previously purchased assets remain sealed.
Chain death (90 days)	Dead-man switch activates. 60-day public Dissolution notice begins. During Phase I, the protocol developer executes orderly wind-down. After V6, dissolution and reserve distribution are protocol-governed (Section 5.4).

9.7 Key Hygiene (operational)

Institutional keys (developer / Vault ops) must be kept offline (cold) and never reside on public nodes. Public nodes run with low-balance hot wallets and no reserve keys. Any operational address rotation will be published with full traceability and proofs.

10. The Constitution

SOST has no ongoing parameter governance. No DAO. No voting token. No multisig that can change protocol rules. The following are immutable at genesis:

```
SOST Constitutional Rules — Encoded at Genesis, Unchangeable

C1: Total supply is upper-bounded by SUPPLY_MAX_STOCKS by construction of the emissio
     (total supply shall never exceed 4,669,201 SOST).
   The only issuance mechanism is subsidy(height). Any block that mints more is invali
C2: Coinbase split is immutable: q=reward//4, gold=q, popc=q, miner=reward-
gold-popc.
   Exact integer conservation guaranteed: miner + gold + popc == reward always.
C3: Gold Funding Vault coinbase destination is immutable at genesis. Heritage Reserve
   reserve assets may only be accessed under Emergency Catastrophe (≥95% miner superma
   V6 5-defense Gold Vault model), formally declared Dissolution (Section 5.4), or pla
     Reserve Lifecycle transitions (Section 5.5).
C4: Metals Reserve (tokenized, allowlisted) shall not be sold except under Emergency C
buy-redistribute)
     or formally declared Dissolution.
C5: Dissolution requires 90 days of zero blocks + 60-day public timelock.
```

```
C6:  Dissolution is one-way and irreversible.
C7:  Slash is automatic. No human can override once triggered.
C8:  Slash split is 50/50 (pool/gold). No supply is burned. Immutable.
C9:  Bond dynamic table: <0.0001→12%, <0.001→15%, <0.01→20%, <0.1→25%, <0.2→26%, <0.3-
C10: No admin key, pause function, or upgrade proxy exists for the PoW chain.
C11: ConvergenceX parameters are fixed at genesis and apply to all epochs.
   Deterministic epoch transitions are derived from consensus constants defined at ge
C12: All consensus arithmetic is integer-only. No floating-point in any consensus path
C13: Model B escrow contracts must be non-upgradeable, have no admin key, and allow
     withdrawal only by the original depositor after expiry.
C14: Default 50/50 XAUt/PAXG acquisition split; during Phase I, adjustable operational
   by the protocol developer for future purchases only. After V6, adjustments require
   protocol-level governance. Previously purchased assets are not rebalanced.
C15: Emergency Catastrophe requires the V6 5-defense Gold Vault model: (1) purpose res
     (2) dual destination whitelists (hard fork to modify), (3) hard per-
spend cap (2% of balance),
   (4) asymmetric rate limiting, (5) ≥95% miner supermajority (W=144 blocks, YES≥137)
   Only two actions permitted: E1 (ROTATE) and E2 (LIQUIDATE→BUY→REDISTRIBUTE).
    No other mechanism to move reserve assets. Vault SOST is never burned.
C16: Progressive decentralization is irreversible. At V6 activation (block 10,000), cu
     transitions directly from the protocol developer to the protocol —
no intermediate multisig.
   All operational roles held by the developer are transitional. As each process is au
   and independently verified, manual control is permanently relinquished.
```

10.5 ConvergenceX Licensing

ConvergenceX is open source technology. The source code is publicly readable, auditable, and open to contribution without restriction.

ConvergenceX and the SOST Protocol implementation are released under the MIT License. The software may be used, copied, modified, integrated, distributed and deployed without restriction and without any licence fee, security deposit, registration or additional permission.

This applies to all participants — including exchanges, custodians, wallet providers, block explorers, miners and infrastructure operators — for both non-commercial and commercial deployment.

The only obligation is the standard MIT requirement to retain the copyright and permission notice. The SOST and ConvergenceX names and associated branding may be subject to separate trademark rights. The protocol makes no representation about future SOST value. This is not an investment product.

10.5 Pre-Market Phase and Internal Valuation Model (PoPC)

SOST is currently operating in a pre-market phase.

An internal valuation system, called Proof of Personal Custody (PoPC), has been im-

plemented to simulate value dynamics and stress-test the economic model. Approximately $6,000 USD equivalent has been committed within PoPC as a reference layer for internal valuation experiments.

IMPORTANT: - PoPC DOES NOT represent a real market price - PoPC DOES NOT imply liquidity - PoPC DOES NOT guarantee future valuation

PoPC is: - A theoretical and experimental valuation framework - A system to test convergence-based economics - A reference mechanism for early-stage analysis

Real market price will ONLY be determined when: - SOST is listed on exchanges - Real liquidity exists - Open market trading begins

Until then, all displayed values are strictly non-binding and should be interpreted as experimental indicators only.

How Value Will Emerge

SOST does not impose a price. Value will emerge through: 1. Mining participation 2. Network usage 3. Market interaction 4. Liquidity formation

Initial market conditions may be: low liquidity, high volatility, manual order seeding. This is expected and natural. SOST follows a pure price discovery model, similar to early Bitcoin stages.

“SOST does not simulate a market — it prepares for one.”

11. Roadmap

Date	Milestone
Mar 15, 2026	GENESIS BLOCK — 18:00:00 UTC. Solo mining (C++). Chain begins.
Mar 30, 2026	PUBLIC ANNOUNCEMENT — BTCTalk ANN. C++ miner released. GitHub public.
Mar-Jun 2026	PRE-MARKET PHASE — Infrastructure testing, external miner onboarding, bug fixing. PoPC internal valuation active. No exchange listing.
Jun 16, 2026	PUBLIC LAUNCH — Exchange outreach begins. Initial market formation. Real price discovery starts.
Q2 2027	Gold purchases begin (8 months post-public-launch per G4, subject to sufficient SOST market liquidity). First XAUT/PAXG acquired. Heritage Reserve deployed on Ethereum mainnet. Dashboard published.

Date	Milestone
Q1 2027	PoPC Model A + B launch on Ethereum mainnet. First custody contracts active.
Aug 2027	Physical gold extension. Mobile app for audit challenges. Verifier network.
2028	ZK R&D begins. Physical proof system research.
2029-2030	ZK physical custody proofs. Full privacy-preserving audit system.
2030+	**Multi-Metal Tokenization Platform (Vision).**

SOST blockchain aims to serve as infrastructure for tokenizing,storing, and exchanging multiple precious metals (gold, silver, platinum, palladium, rhodium, iridium, ruthenium, osmium, and others as market conditions permit). The goal is for every tokenized asset to be backed 1:1 by physical reserves in auditable vaults with independent proof of reserves. | | Indefinite | Heritage Reserve. Gold sealed by default. Emergency 5-defense model if needed. Chain runs on fees. |

NOTE: The protocol developer reserves the right to accelerate or delay any committed timeline if it is determined that the protocol can be implemented with maximum guarantees of technical reliability and security, or if additional development time is required to meet those standards.

12. Technical Specifications

Parameter	Value
Name / Symbol	SOST
Genesis	2026-03-15 18:00:00 UTC (block 0)
Algorithm	ConvergenceX (~8GB RAM mining: 4GB dataset + 4GB scratchpad; ~500MB node; 100k rounds, sequential, stability certificate)
Block time	600 seconds target
Supply cap enforcement	Subsidy-by-height (hard cap by construction)
Max supply	~4,669,201 SOST (466,920,160,910,299 stocks)
Min unit	1 stock = 0.00000001 SOST
Epoch	131,553 blocks (~2.502 years, Feigenbaum alpha)
~95% mined	~12 epochs (~30 years)
Initial reward	7.85100863 SOST/block
Coinbase split	q=reward//4; gold=q, popc=q, miner=reward-gold-popc

Parameter	Value
Annual decay	~9.03% (smooth, no halvings)
Difficulty	cASERT bitsQ Q16.16, per-block; V1: 48h half-life, 6.25% cap; V2 (block 1450+): 24h half-life, 12.5% cap; V5 (block 5175+): avg288-based with dynamic cap (±15s dead band, 0%/0.5%/1.0%/2.0%/~3.03% (prev/33); median removed at block 5,270, avg288 sole signal)
cASERT equalizer	Emergency-only, consensus-critical, 43 profiles. Pre-V12 (#5,750..#7,349): 21 active E7-H13, 22 reserved H14-H35. V12 (#7,350+): 28 active E7-H20 (new ceiling), 15 reserved H21-H35. H35 max reserved, H10-H35 margin=115, B0 baseline. Direct lag map since block 5,323 (HISTORICAL slew ±1 profile/block, V6 5,000-5,322). V13 (#12,000+): all 43 active, ceiling H35. Current relief: V12 triangular cascade (7 steps, max 28 levels); HISTORICAL E7@605s (5,750-6,549)
Stability check	k probes × g steps, dual criteria (local non-explosion + global contraction)
Checkpoints	16 per attempt, Merkle tree, SPV-verifiable
Sync modes	FULL / ADAPTIVE (default) / BASIC bootstrap
Consensus arithmetic	100% integer (zero floating-point)
Reserve chain	Ethereum mainnet
Reserve mode	Heritage (sealed by default)
Emergency	≥95% miner signaling (W=144) + V6 5-defense Gold Vault model
Emergency actions	E1: ROTATE / E2: LIQUIDATE→BUY→REDISTRIBUTE
Default gold split	50/50 XAUt/PAXG (adjustable operationally, future purchases only)
Premine	0 SOST — zero
ICO / VCs	None
Consensus governance	None — immutable at genesis
Operational governance	Phase I: developer-executed; Phase II (V6, block 10,000): protocol-governed. Constitutionally constrained, publicly auditable

12.5 Sovereign Value Stack

SOST is building a cooperative value system based on self-custodied tokenized gold and precious metals, with a signed position market, private economic coordination, and verifiable settlement. The system is designed around identified positions and contractual rights — not generic fungible pool tokens.

12.5.1 E2E Encrypted Coordination Layer

All trade coordination uses true end-to-end encryption. The cryptographic stack:

Identity & Signing: ED25519 (permanent identity, signs everything) Key Agreement: X25519 (per-deal ephemeral key exchange) Key Derivation: HKDF-SHA256 (labels: “sost-deal-key-a” / “sost-deal-key-b”) Payload Encryption: ChaCha20-Poly1305 AEAD (authenticated encryption) Replay Protection: sequence numbers + sliding window + nonce dedup per session

The relay is blind transport — it can store, forward, and route messages but CANNOT read their content. Only the two parties of a deal can decrypt the payload.

Prekey architecture (Phase XIV): - Identity Key (ED25519): permanent, signs everything - Signed Prekey (X25519): rotated every 7 days, signed by identity key - One-Time Prekeys (X25519): consumed once per session, replenished in batches - Prekey bundles publishable to relay for offline session establishment

Offline delivery: - Sender can initiate encrypted session while recipient is offline - Relay queues encrypted messages (store-and-forward, 7-day TTL) - Recipient fetches pending messages when back online - Delivery receipts: delivered / acknowledged / processed

The E2E layer supports: - Signed + encrypted trade offers (GOLD, POSITION_FULL, POSITION_REWARD_RIGHT) - Signed + encrypted acceptances with deterministic deal_id derivation - Encrypted cancellations and settlement notices - Nonce-based replay rejection with sliding window - Per-deal private channels with encrypted message persistence - Async session bootstrap via prekey bundles - Header minimization (only routing metadata visible, all economics encrypted)

Not yet implemented: double ratchet, post-compromise security, deniability. These are planned for a future phase.

Current status: 231 TypeScript tests passing across 31 test files.

12.5.2 SOST Position Market

The SOST market is designed as a signed OTC/RFQ venue for identified positions and rights, not as a traditional AMM or public token exchange.

What can be traded: Phase 1 (current alpha): - Model B full position transfer (escrow-backed, clean ownership change) - Model B reward-right split (sell future reward, keep gold-backed principal) Phase 2: - Model A reward rights (custody obligation stays with original holder) - Model A controlled novation (with revalidation and operator approval) Phase 3 (requires legal review): - XAUT/PAXG entry/exit rail - Broader market access

How a trade works: 1. Seller creates signed offer (ED25519, canonical hash, nonce, expiry, position_id) 2. Buyer responds with signed accept 3. System derives deal_id = SHA256(offer_id:accept_id)[:16] 4. Settlement engine verifies: ownership, position status, signature, nonce, no double-spend 5. Executes: full position transfer OR reward-right split 6. Emits signed settlement_notice 7. Records complete audit trail

The market exists because the same position does not have the same value for everyone: sellers want immediate liquidity; buyers want discounted future value. This is analogous to selling a bond before maturity or separating coupon from principal.

12.5.2b Ownership Rules (Constitutional)

Every Model B position has two separate rights:

1. Right to the Principal (principal_owner) The right to recover the gold locked in the escrow at maturity.
2. Right to the Reward (reward_owner) The right to receive the SOST reward at maturity. Can be sold separately without affecting the principal.

Constitutional rule: "The right to the principal belongs to the principal_owner. The right to the reward belongs to the reward_owner. The escrow pays to the currentBeneficiary, which must always reflect the principal_owner of the position."

Full position sale: Before: principal_owner=Angel, reward_owner=Angel, eth_beneficiary=Angel After: principal_owner=Pedro, reward_owner=Pedro, eth_beneficiary=Pedro At maturity: gold → Pedro, reward SOST → Pedro

Reward-right sale only: Before: principal_owner=Angel, reward_owner=Angel, eth_beneficiary=Angel After: principal_owner=Angel, reward_owner=Pedro, eth_beneficiary=Angel At maturity: gold → Angel, reward SOST → Pedro

Automation roadmap: - Settlement Engine: operational (verifies, executes, updates ownership) - E2E Relay: operational (encrypted coordination, blind transport) - Position Registry: operational (tracks principal_owner, reward_owner) - Maturity Tracker: operational (monitors unlock times) - SOSTEscrow v1: deployed on Sepolia (pays original depositor only) - SOSTEscrow v2: planned (adds currentBeneficiary — pays whoever owns principal) - Auto-Withdraw Daemon: planned (calls withdraw() at maturity automatically) - Reward Engine: planned (credits reward SOST to reward_owner at maturity)

SOSTEscrowV2 deployed with dynamic beneficiary and settlementOperator: - currentBeneficiary per deposit (initialized to depositor) - settlementOperator (immutable, set at construction) can update beneficiary - settlementOperator CANNOT withdraw funds — only reassign beneficiary - This enables automated full-sale settlement without seller cooperation - Withdraw restricted to currentBeneficiary only

Full automation roadmap: - Settlement operator pattern: operational (Phase XVII) - Beneficiary handoff after full sale: operational - Auto-withdraw daemon: operational - Reward settlement daemon: operational - Reconciliation tools: operational - 550 tests (Python 272 + TypeScript 231 + Solidity 47), 0 failures

12.5.3 Settlement Engine

The settlement stack consists of: - Deal state machine (11 states: CREATED through SETTLED/REFUNDED/EXPIRED/DISPUTED) - Ethereum watcher (polls Sepolia/mainnet for escrow deposit events, 6 confirmations) - SOST watcher (polls node RPC for balance/lock events) - Settlement daemon (coordinates state transitions, triggers refunds on expiry) - Refund engine (handles ETH-only, SOST-only, and both-locked timeout cases) - Position settlement (executes transfers and reward splits) - Audit log (JSONL format, complete event trail per deal)

Happy path: CREATED → NEGOTIATED → AWAITING_ETH_LOCK → AWAITING_SOST_LOCK → BOTH_LOCKED → SETTLING → SETTLED Refund path: any lock state → EXPIRED or REFUND_PENDING → REFUNDED

Current status: fully tested with 213 Python tests (unit + integration + adversarial), 19 Solidity tests (Foundry), happy path and refund path demonstrated end-to-end.

12.5.4 SOSTEscrow Contract (Model B)

SOSTEscrow.sol is deployed on Ethereum Sepolia testnet: Address: 0x01Eaab645DA10E79c5Bae1C3 Mock XAUT: 0x38Ca34c6B7b3772B44212d6c2597Fd91a6f944D0 Mock PAXG: 0x754A7D020D559EDD60848450c563303262cAdec7

Contract properties (constitutional, immutable): - NO admin key - NO upgrade proxy (no UUPS, no transparent proxy, no beacon) - NO pause function - NO emergency withdrawal - ONLY original depositor can withdraw, ONLY after timelock expiry - nonReentrant on deposit() and withdraw() - Accepted tokens: XAUT (6 decimals) and PAXG (18 decimals) - Lock duration: 28 days minimum, 366 days maximum - Minimum deposit: 0.001 oz

First Model B position registered: Position ID: 8afed8fcd27553a7 Amount: 0.5 oz XAUT (500,000 units) Duration: 28 days Reward rate: 0.4% Status: ACTIVE ETH deposit TX: 0x4d3a6beff787b8fe24f37bb1e8945f823e2b29244a65c7853f89767dc329a8c6

12.5.5 Position Registry and Lifecycle

Every position is an identified registry object with: - position_id (SHA256-derived, 16 hex chars) - owner (SOST address) - contract_type (MODEL_A_CUSTODY or MODEL_B_ESCROW) - backing_type (ETH_TOKENIZED_GOLD, AUTOCUSTODY_GOLD, PHYSICAL_CUSTODY_CERT) - token_symbol (XAUT, PAXG, PHYSICAL) - reference_amount (gold amount in units) - reward schedule and claimed/remaining tracking - status lifecycle: ACTIVE → MATURED → REDEEMED (or SLASHED/EXPIRED) - right_type: FULL_POSITION, REWARD_RIGHT, or PRINCIPAL_CLAIM

Position pricing uses: - Gold backing value at current SOST/gold rate - Time-discounted remaining reward (5% annual for Model B, 12% for Model A) - 3% illiquidity discount - Net value = gold_value + discounted_reward - illiquidity

12.5.6 Architecture

sost-core — chain base, consensus, PoW, cASERT, constitutional addresses sost-gold-exchange — positions, watchers, deals, settlement, pricing, operator tools sost-comms — trade protocol schemas, ED25519 signing, relay node, deal channels

ETH is used only for onboarding/exit. SOST handles all registration, trading, and settlement.

12.5.7 Alpha Status

Current phase: limited alpha (operator-assisted, Sepolia testnet, mock tokens). Test coverage: 467 tests (Python 217 + TypeScript 231 + Solidity 19), 0 failures. SOSTEscrow deployed on Sepolia with first on-chain gold deposit (0.55 oz total locked). First Model B position registered and tracked. Full position trade and reward-right trade demonstrated with signed messages. Position lifecycle tracking operational (maturity ~May 20, 2026). True E2E encryption operational: X25519 + ChaCha20-Poly1305 + blind relay. Prekey architecture with offline delivery: async session bootstrap, store-and-forward. cASERT ceiling: H13 pre-V12 (#5,750..#7,349) / H20 V12 (#7,350+). Profile floor enforcement (block 5,560+). Relief valve: E7@605s (block 5,750+). Profile descent bug fixed: slew now proportional to stall time.

12.5.8 Strategic Direction

The SOST position market begins as a private, signed market for identified positions and rights — not a public mass token exchange. Trade contracts, not pools. Identify positions, not mass tokens. Coordinate privately, settle verifiably. Any broader public market phase is subject to legal and operational review.

12.5.9 SOST AI Engine

The SOST AI Engine is an internal autonomous research and validation system that supports decisions across Materials Engine, a separate project, SOST protocol wording, and Useful Compute task design. It is intentionally off-chain, off-consensus and off-DEX: it does not modify the SOST chain, does not run inside the DEX page, does not sign transactions and does not have access to wallet keys, beneficiary state, or signed deal messages.

CORE PRINCIPLE.

No conclusion is accepted because a model said it. Every claim is classified by evidence level — local_code_verified, local_data_verified, local_doc_supported, multi_source_supported, external_official_supported, model_consensus_only, speculative, contradicted, insufficient_evidence, or do_not_publish. Public outputs are internal-only by default and require explicit human review before being promoted into public copy.

PIPELINE.

mass local hypothesis generation (10s to 100,000 candidates, offline) -> deduplication by stable hash + project-specific pair keys -> local-evidence search across docs/, reports/, outputs/ -> multi-validator pass with strictest-verdict merging -> deterministic weighted ranking (configurable weights) -> top-N retention per campaign template -> AI Council review of top-K only (validator-veto, not majority vote) -> structured applicability profile (utility, false-positive risk, validation pathway, publishability) -> report + dashboard + append-only persistence -> human review before any public claim or DFT promotion

MILESTONES (PHASE 3).

M1 — Evidence Core & Public Claim Guard: Ten-level evidence classification, claim extractor, public-claim guard with safer-rewrite suggestions, eight seeded eval cases derived from real past mistakes (UC rewards postponed, avg288 vs avg1000, gold-redemption wording, a separate project mineral-guarantee, DFT-validated overclaim, etc.).

M2 — Validators (Materials, a separate project, SOST): Read-only validators that cross-check claims against the actual local corpus. Unified ValidatorResult carries verdict, evidence_level, confidence, publishability, evidence_items, missing_evidence, risks and next_steps. Orchestrator merges multiple validators with the strictest verdict winning.

M3 — Hypothesis Factory + AI Council + Learning Loop: Mass local hypothesis generation across binary, ternary, quaternary and doped compositions (materials), AOI x commodity (a separate project), and risky-wording / Heavy-task-design ideas (SOST / UC). Deterministic ranking. AI Council with validator-veto (not majority vote). Eight canonical campaign templates. Append-only outcome-driven rule-based learning loop. Static HTML dashboard generator.

M4 — Applicability & Utility Engine: Each hypothesis is enriched with a structured ApplicabilityProfile answering: what it could be useful for, why theoretically, what evidence is missing, what false-positive risks exist, what the next validation step is, and whether it is publishable. Materials family classifier (oxide / sulfide / nitride / phosphide / carbide / silicide / halide / metallic), element-risk detection (PGM, toxic, rare-earth, cheap-only) and application-aware validation pathways.

AI COUNCIL.

The Council is not a majority vote. It is a panel with three always-on members:

- ValidatorMember (wraps M2 validators; veto power)
- LocalKnowledgeMember (greps the local corpus)
- MockAIMember (deterministic offline stand-in)

Plus four optional members that self-skip when network is disabled or paid models are not enabled:

- OllamaMember (local LLM, network-off by default)
- OpenRouterMember (free-tier hosted LLM, network-off by default)
- HuggingFaceMember (free-tier inference, network-off by default)
- PaidJudgeInterface (paid-off by default; –allow-paid required)

Validator “contradicted” verdicts are a hard veto regardless of any other opinions.

PROVIDER BUDGET.

Four canonical modes:

tiny max 100 hypotheses, 0 provider calls, 0 paid calls normal max 1,000 hypotheses, 20 provider calls (network must be explicitly enabled) deep max 100,000 hypotheses, 100 provider calls (network must be explicitly enabled) paid_judge paid calls require both –network and –allow-paid plus a configured paid token

The default for any campaign is network OFF, paid OFF.

HARD GUARANTEES.

The engine:

- never publishes autonomous conclusions to the public web
- never activates Useful Compute rewards
- never modifies SOST consensus, miner code or node code
- never writes to Useful Compute queues
- never signs transactions or moves funds
- never reads wallet files or private keys
- never scrapes websites or bypasses rate limits
- never calls paid models without an explicit operator flag
- never makes a network call by default
- never treats model-only consensus as verified evidence

Public-facing description: sost-ai-engine.html (web).

13. Conclusion

SOST is built on three ideas that individually exist, but have never been combined:

Proof-of-Work that produces a mathematical certificate. ConvergenceX replaces the hash lottery with gradient descent convergence, memory-hard scratchpad dependency, checkpoint Merkle trees, and formal stability basin verification. The result is not just a hash below a target — it is a verifiable proof that meaningful mathematical work occurred, producing useful entropy as a byproduct.

Automatic, constitutional accumulation of gold reserve. Every mined block allocates 25% to a Gold Funding Vault. Accumulated SOST is converted to tokenized gold exposure (XAUT/PAXG) through an automated pipeline with public attestation. The reserve is Heritage sealed — perpetual and untouched by default. After V6 activation (block 10,000), custody transitions directly from the protocol developer to the protocol under the 5-defense Gold Vault model, requiring ≥95% miner supermajority for any non-routine reserve movement. The Gold Vault must be able to change form, but not purpose. The accumulation is a structural property of the protocol, not a promise made by people.

Voluntary custody commitments audited by PoW-derived entropy. PoPC uses the entropy triple from ConvergenceX blocks to schedule random audits of gold custody — no oracle, no trusted third party, no governance. The audit schedule is a deterministic function of the proof-of-work itself.

The genesis block is self-sufficient: it requires only a miner and a timestamp. Community growth and gold accumulation follow naturally from the protocol's own mechanics. SOST launches from code, not from capital.

18:00:00 UTC, March 15, 2026.

Legal Disclaimer

This document is for informational purposes only. SOST is not an investment product. Participation in PoPC is entirely voluntary and does not constitute a financial instrument or security. The Metals Reserve (tokenized, allowlisted) does not represent a claim on gold by SOST holders. Past simulations do not predict future performance. The protocol makes no representations about the future price, utility, or availability of SOST. Nothing herein constitutes an offer, solicitation, or promise of returns; PoPC reward-rate parameters may be modified at the operational layer and may be paused for security without affecting consensus. Participants should seek independent legal and financial advice before participating in any cryptocurrency system.

Operational Limits & Pauses: PoPC reward-rate parameters are non-guaranteed and may be subject to operational per-epoch budgets, per-wallet caps, and temporary pauses if the PoPC Pool is underfunded or risk controls trigger. The protocol does not create debt or unfunded obligations: no new contracts may be opened unless the reward can be reserved at inception. Any pauses affect only new contract openings and do not alter consensus rules or existing matured withdrawal rights. Nothing herein constitutes a promise of returns or an entitlement to rewards under all conditions.

NO BROKER / NO TRADING / NO CUSTODY: SOST does not provide brokerage, does not execute trades on behalf of users, does not offer arbitrage, and does not custody user assets. Any Vault conversion (SOST → allowlisted tokenized metals) is a transparent, auditable operational procedure and does not constitute active management or promised returns. SOST is a protocol; third-party venues (exchanges, issuers, custodians) may impose their own KYC/AML requirements.

Protocol Custody Commitment: Tokenized gold reserves are held under Protocol Custody Commitment (Model A) on publicly disclosed, verifiable Ethereum addresses. Custody remains fully under protocol control; no redeemability, backing, or price guarantee is implied. Reserve status and commitment periods are published on the SOST explorer. PoPC reward distribution and fee structures will be defined and published prior to Phase 2 activation (target: April 2027). The protocol's long-term objective is to provide full protocol backing by contributing tokenized gold reserves to protocol virtual balances via a PoPC smart contract, subject to verifiable proof mechanisms as determined by the protocol.

---

10. Native Financial Primitives — Technical Roadmap

SOST does not support smart contracts — there is no virtual machine, no user-deployed bytecode, and no Turing-complete execution. Instead, SOST implements purpose-built transaction types that provide specific financial primitives directly in the consensus

layer. These are deterministic, auditable, and cannot be exploited through contract bugs because they are not programmable — they are protocol rules.

This approach follows the model established by Ravencoin (native assets on UTXO without VM) and is consistent with Bitcoin's OP_CHECKLOCKTIMEVERIFY — consensus rules, not smart contracts.

10.1 Reserved Transaction Types

The following output types are reserved in the codebase (include/sost/transaction.h):

OUT_BOND_LOCK (0x10) — Time-locked bond for PoPC Model A OUT_ESCROW_LOCK (0x11) — Time-locked escrow for PoPC Model B OUT_BURN (0x20) — Reserved, NOT activated. SOST supply is immutable by construction. No token destruction mechanism exists or is planned.

Future types (not yet reserved in code): OUT_TOKEN_ISSUE — Create native metal-backed tokens OUT_TOKEN_TRANSFER — Transfer native metal-backed tokens

10.2 Activation Roadmap

Phase 1 — Bond Lock + Escrow (Q1-Q2 2027, ~12 weeks): Activate BOND_LOCK and ESCROW_LOCK transaction types via block height activation. Enables native PoPC bonds and escrow on the SOST chain without Ethereum dependency. Changes limited to consensus validation (R11, S9, S11) and wallet CLI.

Phase 2 — Native Metal Tokens (Q4 2027-Q1 2028, ~6 months): Implement TOKEN_REGISTER, TOKEN_ISSUE, TOKEN_TRANSFER. Developer-only issuance initially (Phase I). Enables native XAUT-SOST, PAXG-SOST, SLVR-SOST tokens backed by verified metal holdings. Each TOKEN_ISSUE includes a Capsule attestation with the Ethereum TX hash proving metal acquisition. Token registration fees are paid to the Gold Funding Vault, not burned.

Phase 3 — Fully Native PoPC (Q2 2028): Migrate all PoPC operations from Ethereum to the SOST chain. Bond locks, escrow, verification, and reward distribution fully on-chain. No Ethereum dependency for any core protocol operation.

Target: 18 months to full sovereignty.

All activation heights will be announced at least 30 days in advance. Each phase requires stability of the previous phase on mainnet for a minimum of 3 months before activation.

The supply of SOST is immutable by construction. No minting, burning, or destruction mechanism exists in the protocol. All slashing operations redistribute funds — they never destroy them.

---

11. EXPERIMENTAL: SOST Materials Discovery Engine

11.1 Overview

The SOST Materials Discovery Engine is an experimental project that extends the SOST ecosystem beyond financial infrastructure into scientific discovery. It is a computational platform for predicting, generating, and evaluating novel material compositions — with a focus on materials derived from abundant, low-cost, or recycled precursors.

This project has a natural relationship with the SOST protocol: SOST is built on the premise that physical metals (gold, silver, platinum) have intrinsic value derived from their atomic properties. The Materials Engine explores this same domain — the physics and chemistry of elements, their combinations, and the emergent properties of their structures.

11.2 Architecture

The engine operates in layers:

Layer 1 — Data: A curated database of ~150,000 known materials sourced from open repositories (Materials Project, JARVIS, AFLOW). Each entry includes composition, crystal structure, and computed properties (hardness, density, conductivity, bandgap, thermal stability, corrosion resistance, cost estimate, toxicity, and elemental abundance).

Layer 2 — Inverse Search: Users specify desired properties and constraints. The engine returns ranked candidates from known materials, scored by match percentage.

Layer 3 — Property Prediction: Machine learning models (gradient boosting, neural networks) trained on known materials predict properties of hypothetical compositions not yet in the database. Each prediction includes a confidence interval and uncertainty estimate.

Layer 4 — Generative Discovery: An evolutionary algorithm generates random material compositions within user-defined constraints (element availability, cost, toxicity). A genetic algorithm iterates over generations, mutating and combining successful candidates. The predictor estimates properties for each candidate. Top candidates are ranked by fitness against the user's requirements.

Layer 5 — Continuous Learning: The system improves over time. User queries, community feedback, and published experimental results are incorporated into the training data. The models retrain periodically, expanding coverage and improving accuracy.

11.3 Access via SOST Escrow

Access to the Materials Engine requires a SOST escrow deposit — the same mechanism used by PoPC Model B. Users deposit SOST into an immutable timelock contract for 1 to 6 months. During the escrow period, the user has full access to the engine. At expiry, 100% of the deposit is returned. The protocol charges a protocol fee (3% Model A, 8% Model B) on the reward amount.

This creates organic demand for SOST: researchers, engineers, and materials scientists need SOST tokens to access the service. The escrow model ensures tokens are

locked during use, reducing circulating supply. The fee funds ongoing development and infrastructure.

Access tiers (preliminary): - Tier 1 (10 SOST, 1 month): Search + prediction - Tier 2 (50 SOST, 3 months): + generative discovery - Tier 3 (200 SOST, 6 months): + DFT validation + export

11.4 Relationship to SOST Protocol

The Materials Engine does not modify the SOST blockchain or its consensus rules. It is an application-layer service that reads the blockchain to verify escrow deposits. The SOST protocol provides:

- Escrow infrastructure (BOND_LOCK / Model B contracts)
- Payment rail (SOST token)
- Immutable access verification (on-chain escrow state)
- Economic alignment (locked tokens reduce sell pressure)

The blockchain provides the trust layer. The Materials Engine provides the utility layer. Together, they create a feedback loop: scientific utility drives SOST demand, SOST demand strengthens the gold reserve, the gold reserve underpins the token's value.

11.5 Output Specification

For each candidate material, the engine produces a standardized report:

- Candidate ID (SOST-MAT-XXXXX)
- Composition (formula + elemental percentages)
- Predicted structure (crystal system, space group)
- Property estimates with confidence intervals: hardness, bulk modulus, shear modulus, bandgap, density, melting point, thermal conductivity, electrical conductivity, corrosion resistance, magnetic properties
- Stability assessment (formation energy, hull distance)
- Synthesizability estimate (temperature, pressure, method)
- Cost estimate (based on elemental prices)
- Toxicity classification
- Elemental abundance check
- Similar known materials (nearest neighbors)
- Suggested applications
- Risk assessment (prediction confidence)

11.6 Disclaimer

EXPERIMENTAL PROJECT: The SOST Materials Discovery Engine is a research initiative, not a guaranteed deliverable. Implementation will proceed only if the following conditions are met:

(a) The underlying machine learning models demonstrate verified accuracy against independent test datasets with published benchmarks.

(b) The prediction pipeline produces no systematic errors that could mislead users into pursuing dangerous or impossible synthesis paths.

(c) The escrow integration passes independent security audit with zero critical vulnerabilities.

(d) The system's computational requirements are sustainable within the protocol's operational budget.

(e) The technology does not introduce any attack vector against the SOST blockchain or its users.

The protocol commits to full transparency: if the project does not meet these criteria, it will be discontinued or restructured rather than deployed in an unsafe or misleading state. No timeline is guaranteed. The Materials Engine is a vision, not a promise.

This project does not affect the SOST blockchain, its consensus rules, emission schedule, or any constitutional parameter. It is an independent application-layer service that can be added or removed without any protocol change.

Appendix A: Canonical Constants (Normative)

All values below are consensus-critical. Any deviation constitutes a consensus failure.

```
CONVERGENCEX PARAMETERS:
  CX_N                          = 32
  CX_ROUNDS_MAINNET             = 100,000
  CX_SCRATCH_MB_MAINNET         = 4,096
  CX_LR_SHIFT                   = 18
  CX_LAM                        = 100
  CX_CHECKPOINT_INTERVAL        = 6,250
  CX_STABILITY_PERTURBATION_SCALE = 1  // B0 baseline scale
  CX_STABILITY_K                = 4    // B0 baseline k
  CX_STABILITY_MARGIN           = 185  // B0 baseline margin
  CX_STABILITY_GRADIENT_STEPS   = 4    // B0 baseline steps

MONETARY PARAMETERS:
  GENESIS_TIME                 = 1,773,597,600  // 2026-03-15 18:00:00 UTC
  TARGET_SPACING                = 600
  BLOCKS_PER_EPOCH              = 131,553
  R0_STOCKS                    = 785,100,863
  SUPPLY_MAX_STOCKS            = 466,920,160,910,299
  Q_NUM                         = 7,788,007,830,714,049
  Q_DEN                         = 10,000,000,000,000,000

DIFFICULTY PARAMETERS:
  Q16_ONE                       = 65,536
  BITSQ_HALF_LIFE (V1)          = 172,800   // 48h — blocks < 1450
  BITSQ_HALF_LIFE_V2 (V2)       = 86,400    // 24h — blocks >= 1450
```

```
 MAX_FUTURE_DRIFT         = 600     // HISTORICAL (blocks < 6,550); 60 s 6,550-
11,999; current 30 s (block 12,000+)

cASERT EQUALIZER PARAMETERS (consensus-critical, 43 profiles):
 CASERT_H_MIN              = -7   // E7 is lowest profile (block 5,750+)
  CASERT_H_MAX                          =  35      //  scale  2  for  H5-
H35, H35 hardest; active since V13 (#12,000+)
 CASERT_MAX_ACTIVE_PROFILE_PRE_V12 = 13   // pre-V12 ceiling (#5,750..#7,349)
  CASERT_MAX_ACTIVE_PROFILE_V12     = 20    // V12 ceiling (#7,350+)
 // 43 profiles: E7(-7) through H35(+35), each with {scale, steps, k, margin}
  // B0(0) baseline: scale=1, steps=4, k=4, margin=165
 // Relief valve: HISTORICAL E7 at 605s (5,750-6,549); current V12 triangular cascade
  // See profile table in Section 3.12

cASERT DECAY ANTI-STALL (zone-based, targeting B0):
  cASERT_DECAY_H13_H7_SECS    = 600    // 10 min/level for H13-H7
  cASERT_DECAY_H6_H4_SECS     = 900    // 15 min/level for H6-H4
  cASERT_DECAY_H3_H1_SECS     = 1,200  // 20 min/level for H3-H1
   cASERT_DECAY_EASE_SECS           =  1,800   //  30  min/level  for  E1-
E7 (after 6h+ at B0)

EMERGENCY PARAMETERS:
  EMERGENCY_WINDOW_BLOCKS      = 144       // ~24 hours at 600s/block
  EMERGENCY_THRESHOLD_NUM      = 95        // 95%
  EMERGENCY_THRESHOLD_DEN      = 100
  EMERGENCY_YES_REQUIRED       = 137       // ceil(0.95 × 144)
```

Appendix B: Serialization (Normative)

All consensus-critical serialization uses **little-endian** byte order unless explicitly noted.

```
FIELD SERIALIZATION:

  height:       int64,  8 bytes LE
  time:         int32,  4 bytes LE (Unix timestamp)
  powDiffQ:     int32,  4 bytes LE (Q16.16 compact)
  nonce:        uint32, 4 bytes LE
  extra_nonce:  uint32, 4 bytes LE
  subsidy:      int64,  8 bytes LE (stocks)

  header_core:  72 bytes =
   prev_hash(32) || merkle_root(32) || timestamp(u32 LE) || powDiffQ(u32 LE)

 block_key:   SHA256(prev_hash(32 bytes, raw) || "BLOCK_KEY"(9 bytes, ASCII))
                = 32 bytes

 seed:      SHA256(MAGIC || "SEED" || header_core(72) || block_key(32) || nonce(u32 L
```

```
                = 32 bytes

  commit:     SHA256(MAGIC || "COMMIT" || header_core(72) || seed(32) || state(32) || x
                     checkpoints_root(32) || stability_metric(u64 LE))
                = 32 bytes

   checkpoint leaf:
                SHA256("CP"(2 bytes) || state_hash(32) || x_hash(32) ||
                       round(u32 LE) || residual(u64 LE))
                = 32 bytes

   full_header:  MAGIC(10 bytes) || "HDR2"(4 bytes) ||
                 header_core(72 bytes) || checkpoints_root(32 bytes) ||
                 nonce(u32 LE) || extra_nonce(u32 LE)
                 = 126 bytes total
                 NOTE: full_header does NOT contain commit.

  block_id:    SHA256(full_header(126 bytes) || "ID"(2 bytes) || commit(32 bytes))
                = 32 bytes
           NOTE: commit is concatenated AFTER "ID", not inside full_header.

  Target comparison: commit vs target, both 32-byte BIG-ENDIAN, lexicographic.
   Valid if: commit <= target (big-endian comparison).

MAGIC CONSTANT (network-dependent):
  MAGIC = "CXPOW3" (6 bytes, ASCII) || SHA256("SOST/CONVERGENCEX/" || NETWORK)[:4]
   Total: 10 bytes.

   Precomputed values:
     mainnet:  0x4358504f5733 c6e88538
     testnet:  0x4358504f5733 39014c33
     dev:      0x4358504f5733 f950f94b

   Where NETWORK is the ASCII string "mainnet", "testnet", or "dev".

PARSING CONVENTIONS (normative):
    int32_le(buf):  interpret 4 bytes as signed 32-bit integer, little-
endian.
   u32_le(buf):    interpret 4 bytes as unsigned 32-bit integer, little-
endian.
   u64_le(buf):    interpret 8 bytes as unsigned 64-bit integer, little-
endian.
  asr_i32(x, s):  arithmetic right shift on signed 32-bit (see Section 3.6).
   All multi-byte integer fields in hash inputs are little-endian unless
   explicitly noted otherwise (target comparison is big-endian).
```

Appendix C: PRNG Specification (Normative)

```
PRNG(seed, n_bytes):
  output = empty
  counter = 0
  While |output| < n_bytes:
    chunk = SHA256(seed || counter_u32_le)
    output = output || chunk
    counter += 1
  Return output[0 : n_bytes]
```

Appendix D: Test Vectors (Normative)

Canonical test vectors are published alongside the reference implementation. A conforming implementation MUST reproduce all test vectors bit-for-bit. The generation script (`generate_test_vectors.py`) is included in the repository.

D.1 Required Vector Set

```
TEST VECTORS (minimum conformance set):

  V1: block_key derivation
    Input:  prev_hash = 0x00...00 (32 bytes of zeros)
    Output: block_key = SHA256(prev_hash || "BLOCK_KEY")
    Expected: (hex, published in repo)

  V2: header_core construction
    Input:  prev_hash = 0xAA×32, merkle_root = 0xBB×32,
            timestamp = 1773597600, powDiffQ = GENESIS_BITSQ
    Output: 72 bytes (hex, published in repo)

  V3: seed derivation
   Input:  header_core from V2, block_key from V1, nonce = 0, extra_nonce = 0
   Output: seed = SHA256(MAGIC || "SEED" || header_core || block_key || 0x00000000 || 0
    Expected: (hex, published in repo)
   NOTE: header_core MUST be included per conformance requirement (Appendix B).
   NOTE: MAGIC uses the corrected network-dependent derivation (Appendix B).

  V4: ConvergenceX deterministic execution (test profile)
   Input:  header_core from V2, block_key from V1, nonce = 0, extra_nonce = 0,
           rounds = 1000, scratch_mb = 8
    Output: commit, checkpoints_root, stability_metric, is_stable
    Expected: (hex values, published in repo)

  V5: pow_meets_target boundary test
    Input:  Various (bitsQ, commit) pairs
    Expected: boolean results for boundary conditions
```

```
  V6: Subsidy computation (by height only)
    height = 0              → 785,100,863
    height = 131,553        → floor(R0 * q)
    height = 2*131,553      → floor(R0 * q^2)

  V6b: Hard cap by construction (bound check)
    Assert using rational fixed-point math:
      B * R0 / (1 - q) <= SUPPLY_MAX_STOCKS
    where B = BLOCKS_PER_EPOCH

  V7: Coinbase split conservation
    Input:  reward = 785,100,863
   Expected: gold = 196,275,215, popc = 196,275,215, miner = 392,550,433
              sum = 785,100,863

  V8: BitsQ direction (V14 canonical)
    Input:  bitsQ = GENESIS_BITSQ
   Expected: lower bitsQ accepts strictly more commits (monotonic, larger target)

GENERATION:
  The canonical test vector script generates all vectors from the
  reference implementation and outputs hex-encoded JSON.
  A conforming implementation MUST match every vector bit-for-bit.
```

Appendix G: Emergency Reserve Authorization (V6 5-Defense Model) (Normative)

This appendix formally defines the Emergency Catastrophe procedure referenced in G9 and C15. The Gold Vault must be able to change form, but not purpose. After V6 activation (block 10,000), the Heritage Reserve contract on Ethereum is governed by a Zodiac + Reality.eth module architecture — no single key or entity controls the vault. During Phase I (before block 10,000), the protocol developer holds operational custody as a transitional necessity.

G.1 EmergencyProposal (SOST chain)

An emergency proposal is initiated by broadcasting a special transaction or coinbase annotation on the SOST chain.

```
EmergencyProposal:
 proposalId:    bytes32   // unique identifier (hash of proposal content)
 actionType:    uint8    // 1 = E1 (ROTATE), 2 = E2 (LIQUIDATE→BUY→REDISTRIBUTE)
 paramsHash:    bytes32   // hash of action parameters (target tokens, amounts, etc.)
  startHeight:     uint64     // first block of signaling window
 expiresAtHeight: uint64   // last block of signaling window (startHeight + W -
1)
```

G.2 PoW Miner Signaling

Miners signal support for an emergency proposal by including a tag in their coinbase transaction:

```
SIGNALING TAG (in coinbase):
  SOST_EMERG|<proposalId_hex>

YES BLOCK:
  A block counts as YES if its coinbase contains the exact tag for the active proposal.

SIGNALING WINDOW:
  Range: [startHeight, startHeight + W - 1]  where W = 144 (default, ~24 hours)

APPROVAL CONDITION:
  YES = number of blocks in window containing the signaling tag
  TOTAL = W
  Approved if: YES >= ceil(0.95 × W)
  With W = 144: YES >= 137

RECORD:
  EmergencyApproved(proposalId, paramsHash, startHeight, endHeight)
  Recorded on-chain (event, RPC, or recognized state).
```

G.3 V6 5-Defense Vault Execution (Ethereum — Zodiac + Reality.eth)

After on-chain approval is confirmed on SOST with ≥95% supermajority, the Heritage Reserve contract on Ethereum enforces the 5-defense model via a Zodiac module with Reality.eth oracle integration:

```
V6 5-DEFENSE EXECUTION:
  Defense 1 (Purpose Restriction):
    actionType must be 1 (ROTATE) or 2 (LIQUIDATE→BUY→REDISTRIBUTE).
    No general-purpose spending. Vault SOST is never burned.

  Defense 2 (Dual Destination Whitelist):
    Target addresses are hardcoded in the contract. Adding a new valid
    destination requires a hard fork of the Heritage Reserve contract —
    no vote or key can modify the whitelist.

  Defense 3 (Hard Per-Spend Cap):
    No single transaction may move more than 2% of the vault balance
    at the time of execution.

  Defense 4 (Asymmetric Rate Limiting):
    GV1 (routine rotation): max 5% of balance per rolling 30-day window.
    GV3 (non-routine migration): 30-day cooldown between operations.

  Defense 5 (95% Supermajority):
```

```
    Non-routine reserve migration (GV3) requires EmergencyApproved
    confirmation from the SOST chain with YES >= 137 (95% of W=144).

  proposalId:    bytes32
  paramsHash:    bytes32
 actionType:    uint8      // 1 = ROTATE, 2 = LIQUIDATE→BUY→REDISTRIBUTE
  expiry:        uint256      // Ethereum block.timestamp deadline
  nonce:         uint256      // replay protection
 params:     bytes     // action-specific parameters (target tokens, amounts)
```

G.4 Ethereum Contract Verification

The Heritage Reserve contract on Ethereum mainnet verifies all five defenses:

```
VERIFICATION STEPS:
  1. Verify proposal approval via Reality.eth oracle (SOST chain ≥95% signaling confirm
  2. Verify nonce (sequential, prevents replay)
  3. Verify expiry >= block.timestamp
  4. Verify actionType is 1 or 2 (Defense 1: purpose restriction)
  5. Verify target addresses are on hardcoded whitelist (Defense 2: destination whiteli
  6. Verify amount <= 2% of vault balance (Defense 3: per-spend cap)
  7. Verify rate limits not exceeded (Defense 4: asymmetric rate limiting)
  8. Execute E1 (transfer to allowlisted token) or E2 (sell + buy SOST + redistribute to
```

G.5 Mandatory Audit Trail

Every emergency execution MUST produce a complete, publicly verifiable audit trail:

```
AUDIT REQUIREMENTS:
  1. Proof of signaling: list of blocks with YES signal, block hashes, explorer links
  2. Proof of approval: EmergencyApproved record on SOST chain
  3. Ethereum transaction hashes for all Heritage Reserve operations
  4. Batch report: amounts moved, tokens involved, execution timestamps
  5. If E2 (redistribute): proof of SOST redistribution (TX hashes to PoPC Pool and Gold
  6. All data published on project website, GitHub, and IPFS mirror
```

HISTORICAL APPENDICES — NOT NORMATIVE

The following appendices (F, H, I) document the evolution of the protocol specification. They are retained for transparency and historical reference. The normative specification is contained in the main body of this document (Sections 1-12) and normative appendices (A-D, G). In case of conflict between a historical appendix and the main body, the main body prevails.

Appendix F: Errata v3.3 (Normative Conformance)

Changes from v3.2 to v3.3, applied to close conformance gaps identified by CTO audit:

```
ERRATA v3.3

TEST VECTOR CONFORMANCE:

26) V3 seed derivation: header_core added
    V3 now requires header_core from V2 as input to seed derivation.

ENDIANNESS / PARSING (normative):

27) b[i] parsing: int32 → int32_le (explicit little-endian)
28) w0, w1 parsing: state[0:4] → u32_le(state[0:4]) (explicit)
29) x0 initialization: >> 4 → per-element asr_i32(int32_le(...), 4)
30) Appendix B: added PARSING CONVENTIONS normative block

BLOCK_ID / FULL_HEADER:

31) full_header explicitly defined
32) block_id: commit concatenated AFTER "ID" separator

EPOCH ANCHOR GRINDING:

33) anchor_timestamp removed from epoch_key

MODEL B WATCHER TRUST MODEL:

34) Watcher trust model added (CAN delay, CANNOT steal/block)

MODEL B CONSISTENCY (CTO review):

35) Model B reward: CONFIRMED immediate payout
36) Model B: added NO EARLY EXIT clause (gold escrow only)
37) Model B watcher: clarified "triggers immediate SOST payout"
38) Section 6.9: table confirms "received immediately" + reward rate proration
39) Section 6.12: REWARD BUDGET + SOLVENCY SAFEGUARDS added
40) Legal Disclaimer: added Operational Limits & Pauses paragraph
41) Section 5.1: TWO-VAULT ARCHITECTURE (Gold Funding Vault (Gold-
first) + Heritage Reserve)
42) Section 4.3: COINBASE TRANSACTION FORMAT (normative)
43) Section 3.13: BASIC MODE TIP REJECTION (normative box)
```

Appendix H: Errata v3.4 (Heritage Reserve & Emergency Dual Gate)

Changes from v3.3 to v3.4:

```
ERRATA v3.4
```

RESERVE MODEL:

44) Timelock 30 years eliminated → Heritage sealed by default
    Heritage Reserve: sealed by default, perpetual
    and untouched while SOST exists. No fixed expiry date.

45) Emergency Catastrophe dual gate added (G9, C15, Appendix G)
    Reserve movements require both: ≥75% miner signaling in a 24-hour
    window (W=144 blocks, YES≥108) AND Protocol Execution Order.
    Only two actions: E1 (ROTATE) and E2 (LIQUIDATE→BUY→REDISTRIBUTE).

46) Default 50/50 XAUt/PAXG split (G7, C14)
    Protocol may adjust split operationally for future purchases only.
    Previously purchased assets not rebalanced except under emergency.

47) Staging assets: USDC + USDT (G6)
    Both USDC and USDT permitted as temporary staging (≤72 hours).

48) Ethereum mainnet only
    All references to "Ethereum/Arbitrum" updated to "Ethereum mainnet".
    XAUt and PAXG are ERC-20 tokens on Ethereum L1.

49) Section 5.4 rewritten: "What Happens After 30 Years" → "Heritage Reserve and Dissol
    Heritage is default perpetual state. If chain dies without prior
   emergency authorization, reserve remains sealed (no signaling possible).

50) Section 7.1: Emergency does not modify consensus
    Added clarification that Emergency Catastrophe authorization gates
   only external reserve movements, not consensus rules or monetary policy.

51) Section 7.2: Operational governance expanded
    Explicit bullets: 50/50 split adjustment, purchase pause, issuer
    redirection — all operational, future-purchases-only. Moving
    previously purchased assets requires Emergency Catastrophe.

52) Section 9.6: Emergency table updated
    XAUT issuer risk → Emergency Catastrophe dual gate.
    Dissolution → reserve stays Heritage if chain dead.

53) Constitution: C3 rewritten (Heritage + dual gate), C14/C15 added.

54) Roadmap: "Year 30+ Heritage locked forever" → "Heritage indefinite;
    emergency dual-gate if needed."

55) Tech Specs: Reserve chain, Heritage mode, Emergency parameters added.

56) Appendix A: Emergency parameters added (W=144, threshold=75%, YES≥108).

```
SERIALIZATION (consensus-critical fix):

57) MAGIC constant corrected in Appendix B
    Previous: MAGIC = "SOST_CX_v1" (10 bytes, ASCII)
   Corrected: MAGIC = "CXPOW3" (6 bytes) || SHA256("SOST/CONVERGENCEX/" || NETWORK)[:4
    Total: 10 bytes. Precomputed values for mainnet/testnet/dev provided.
    This matches the Python reference and C++ implementation exactly.
    The v3.3 text contained an outdated placeholder that was never used
    in the actual consensus code.

58) full_header size corrected
    Previous: 128 bytes (included "CXPOW3" prefix + MAGIC separately)
    Corrected: 126 bytes (MAGIC(10) || "HDR2"(4) || header_core(72) ||
    checkpoints_root(32) || nonce(4) || extra_nonce(4))
    MAGIC already contains "CXPOW3" as its first 6 bytes; the separate
    "CXPOW3" prefix was a documentation error, not present in code.

59) Appendix G added (Emergency Authorization & Execution Order)
    Normative specification of EmergencyProposal, PoW signaling,
    Protocol Execution Order, Ethereum contract verification,
    and mandatory audit trail.
```

Appendix I: Errata v3.5 (Security, Fees & Slash Reform)

Changes from v3.4 to v3.5:

SLASH REFORM: 60) Slash split changed: 33/33/33 (burn/pool/gold) → 50/50 (pool/gold). No supply is burned. With ~4.67M max supply, every token has utility. Slashed bonds are recycled into PoPC Pool and Gold Funding Vault. 61) Constitution C8 updated to reflect 50/50 slash split. 62) Section 8.2 updated: “Slash burn” → “Slash recycling”.

PROTOCOL FEES: 63) Section 6.7.1 added: Model A protocol fee (3% of reward, calculated at creation, disbursed at completion). [corrected from 5%] 64) Section 6.9.1 added: Model B protocol fee (8% of reward, deducted from immediate payout). [corrected from 10%] Fees fund protocol operations. 30-day notice for changes. Existing contracts locked at creation fee rate.

SECURITY IMPROVEMENTS: 65) Section 6.3: EOA-only verification for Model A wallets. Smart contract wallets rejected. CREATE2 post-deployment detection triggers slash. 66) Section 6.3: Continuous custody check via historical balance sampling at deterministic checkpoints between audits. 67) Section 6.8: Watcher autonomous claim fallback. If watcher fails to pay within 72h, depositor can claim directly via on-chain contract. 68) Section 6.6: Price Bulletin sources now dynamic — includes all exchanges with verifiable SOST liquidity, no fixed min/max count.

DOCUMENT GOVERNANCE: 69) Living Document Notice added. Non-consensus parameters may evolve. Consensus rules remain immutable at genesis.

cASERT MODIFICATIONS: 70) cASERT rewritten: 43 discrete profiles E7(-7) through H35(+35), each with unique {scale, steps, k, margin}. CASERT_H_MIN=-7, CASERT_H_MAX=35 (scale 2 above H4, H35 hardest). 71) Anti-stall: zone-based decay targeting B0. H10-H7: 600s/level, H6-H4: 900s/level, H3-H1: 1200s/level. Easing after 6h+ at B0 (1800s/level), down to E7 in the current implementation.

Appendix M: Removed — Adaptive Mode (Historical, Non-Normative)

This appendix preserves the original Adaptive Mode specification (formerly Section 6.11) for theoretical completeness. This mode is NOT planned for activation. It is documented for historical reference only.

Potential future extension — requires governance evaluation.

The Adaptive Mode proposed that when the SOST/gold ratio exceeded 1.5 for 30 consecutive days, PoPC would also accept SOST as a custody asset. This was removed from the normative specification in v4.0 because accepting SOST as a custody asset creates circular staking dynamics that contradict PoPC's core purpose as a proof of external gold custody.

```
ADAPTIVE MODE (REMOVED — historical reference only):
  ratio = sost_price / gold_oz_price
  If ratio > 1.5 for 30 consecutive days → mode would have activated
  Users would demonstrate custody of SOST rather than gold
  Same bond, audit, and reward mechanics would apply
```

3.14 Chain Selection and Fork Resolution

SOST uses best-chain-by-cumulative-work selection, not longest-chain. Each block contributes work = floor(2^256 / (target + 1)). The chain with the highest total accumulated work wins. This prevents attacks using many easy blocks to outpace a shorter chain with more real work. This is the same approach used by Bitcoin.

Fork resolution is atomic: if a chain with more cumulative work is found, the node disconnects current blocks from the active tip back to the fork point, connects the new chain block by block with full validation (L1-L4 + ConvergenceX Transcript V2 verification), and recovers orphaned non-coinbase transactions to the mempool. If any block in the new chain fails validation, the entire reorganization is aborted and the original chain is fully restored.

Orphan blocks (parent not yet known) are stored separately from fork candidates (parent known but not on active tip) and re-processed when their parent arrives via P2P sync.

MAX_REORG_DEPTH = 500 blocks (~3.5 days). Combined with 1000-block coinbase maturity and ConvergenceX memory-hard requirements, this provides robust protection against deep reorganization attacks.

SOST Protocol Whitepaper v4.0 — Normative Specification Last updated: March 2026 Previous versions: v3.7 (deprecated)

LICENSE: SOST source code is published under the MIT License. The code is fully open-source and free to use, modify, and distribute. See LICENSE file for full terms.

==========
APPENDIX: NON-CONSENSUS PLATFORM EXTENSIONS (March 2026) ==========

A. DYNAMIC FEE MARKET (Mempool Policy, No Consensus Change) - Fee-rate ordering: rational arithmetic (fee*other.size vs other.fee*size) - BuildBlockTemplate: descending fee-rate selection - estimatefee RPC: advisory from 10-block lookback + mempool analysis - Minimum relay: 1 stock/byte (consensus floor, static before block 10,000) - Dynamic relay floor (block 10,000+): scales with mempool pressure (1x at <100 tx, 2x at 100-500, 5x at 500-2000, 10x at 2000-4000, 25x at 4000+, ceiling 50 stocks/byte) Policy-only — does not change block validity. Anti-spam protection. - RBF (Replace-By-Fee): full RBF enabled — replacement requires higher fee per byte - CPFP (Child-Pays-for-Parent): package-aware block template selection

B. NON-CONSENSUS SECURITY ARCHITECTURE - Build hardening: 6 flags (-fstack-protector-strong, -fstack-clash-protection, -fPIE, -pie, -Wl,-z,relro,-z,now, -D_FORTIFY_SOURCE=2) - Wallet encryption: AES-256-GCM + scrypt (N=32768, r=8, p=1), OPENSSL_cleanse after every use - cASERT profile: exposed via RPC getinfo (casert_profile, casert_lag)

B.1 HD WALLET (BIP39) - 12-word seed phrase backup using standard BIP39 English wordlist (2048 words) - Entropy: 128 bits via OpenSSL RAND_bytes (CSPRNG) - Mnemonic: entropy + SHA256 checksum → 12 words - Seed derivation: PBKDF2-HMAC-SHA512, 2048 iterations, salt="mnemonic" - Seed used directly as secp256k1 private key (compatible with web wallet) - CLI commands: sost-cli hd create, sost-cli hd restore - Cross-platform: same seed phrase produces same address in web wallet and CLI - 22 tests passing (BIP39 test vectors, roundtrip, address format)

B.2 SOST-PSBT (OFFLINE SIGNING) - SOST-specific offline signing format — NOT BIP174 - Transport: JSON + base64 with "sost-psbt" magic prefix (0xff terminated) - Supports P2PKH inputs and REDEEMSCRIPT_HASH_MULTISIG inputs - Multiple partial signatures per input (for M-of-N multisig) - Flow: psbt_create → psbt_sign (offline) → psbt_combine → psbt_finalize → broadcast - Private keys NEVER appear in serialized PSBT - Balance validation: sum(inputs) = sum(outputs) + fee + change - Overflow-safe: __int128 arithmetic for amount checking - 24 tests passing (create, sign, combine, finalize, encode/decode, security)

B.3 TRANSACTION SAFETY LAYER - Trusted address book: 4 trust levels (trusted, known, new, blocked) - blocked addresses hard-stop sends - trusted addresses show label in confirmation - JSON persistence at ~/.sost/trusted_addresses.json - New-address cooldown: warning on first send to unknown address - High-value alert for >10 SOST to unknown - –skip-warning flag for automation - Treasury safety profile: configurable via ~/.sost/wallet_policy.json - daily_send_limit, per_tx_limit, vault_mode - vault_mode: all sends require destination in address book - require_addressbook_for_large: large TXs need trusted destination - Pre-send confirmation: full TX summary with yes/no prompt, –yes/-y to skip - All above consensus — no fork required

B.4 DYNAMIC FEE MARKET - RBF (Replace-by-Fee): full RBF mempool policy, enabled by default - Replacement requires strictly higher fee-rate (integer arithmetic) - Must

pay old_total_fee + tx_size × 1 stock/byte minimum - SetRBFEnabled(false) to disable per-node - CPFP (Child-Pays-for-Parent): package fee-rate aware block template builder - BuildBlockTemplateCPFP: parent-child dependency graph - Topological ordering: parents always before children in block - Neither changes consensus — relay/mempool/miner policy only

C. SEPARATE PROJECT MINERAL INTELLIGENCE PLATFORM - 11+ AOIs across 5 continents, zone-specific ML models - AUC 0.72-0.86 (5 supervised zones), heuristic scanner for arbitrary AOIs - Direct GNN inference (CGCNN forward pass) on crystal structures - 162 mineral targets with exact coordinates exported - Key finding: deposit type > commodity for ML training

D. NATIVE MULTISIG (CONSENSUS CHANGE — PRE-LAUNCH) - Minimal script engine: OP_0, OP_1-16, OP_DUP, OP_HASH160, OP_EQUAL, OP_EQUALVERIFY, OP_CHECKSIG, OP_CHECKMULTISIG, OP_PUSHDATA1/2 - RedeemScript-hash outputs (P2SH-like): OP_HASH160 OP_EQUAL - M-of-N CHECKMULTISIG: ordered signature matching, no backtracking - sost3 address type: “sost3” + hex(HASH160(redeemScript)) = 45 chars - Activation: MULTISIG_ACTIVATION_HEIGHT = 2000, OUT_SCRIPT_HASH = 0x30 - Max keys: MAX_MULTISIG_KEYS = 15 - Treasury protection: 2-of-3 for Gold Vault and PoPC Pool - scriptSig format: OP_0 ... - Backwards compatible: P2PKH (sost1) outputs unchanged, old blocks still valid - 24 tests passing (script engine, P2SH, create, verify, backwards compat)

E. AUTONOMOUS MATERIALS DISCOVERY ENGINE - 76,193 material corpus, CGCNN + ALIGNN-Lite models - Autonomous campaign engine with validation queue (6 tiers) - 8 campaign profiles: III-V, oxide, group IV, stable novel, valuable, strategic, battery, exploratory - Structure lift pipeline with direct GNN inference on novel candidates - Phase V.C: 65.5% direct FE rate, 21.7% novel direct GNN, 34.5% proxy dependency - Dual-output explainer (technical + plain language) - 1,073 tests passing across 47 files

F. SEPARATE PROJECT MINERAL INTELLIGENCE MODULE - Research-grade mineral prospectivity mapping, separate from consensus - 11 AOIs globally, 162+ targets, zone-specific models (AUC 0.72-0.86) - Long-term thermal proxy pipeline: 20-year Landsat climatology * Moderate but real improvement at Kalgoorlie (+0.013 AUC) * Signal robust against geology-matched background * This is a thermal proxy family, NOT direct subsurface detection * Full cross-site replication at Chuquicamata: 4 features consistent * Multi-zone verdict: PRODUCTION_WORTHY (4/6) * Thermal most useful when satellite baseline is moderate - EMIT hyperspectral: ACTIVE, clay/hydroxyl signal at Chuquicamata (d>0.8, small sample) - V3 ML residual experiment: NEGATIVE (honest result, documented) - 10 V3 ideas evaluated, 3 VIABLE, next: spatial gradient geology proxies - EXPERIMENTAL: no production claims, research initiative only

G. NETWORK SECURITY LAYER - P2P encryption: X25519 key exchange + ChaCha20-Poly1305 (default ON) - Ban system: 100 penalty points = 24h ban, 64 max inbound peers, 4 per IP - RPC authentication: Basic Auth (–rpc-user/–rpc-pass), localhost binding - Checkpoints: hardcoded at source, MAX_REORG_DEPTH = 500 blocks (~3.5 days) - Build hardening: stack protector, stack clash, PIE/ASLR, full RELRO, FORTIFY_SOURCE - Coinbase maturity: 1000 blocks (~7 days) before miner rewards are spendable

H. CAPSULE PROTOCOL V1 - Binary metadata in transaction outputs (12-byte header + up to 243-byte body) - Activates at mainnet block height 5000 - Enables on-chain

attestation, proof-of-discovery, and structured metadata - Type byte (1B) + version (1B) + length (2B) + reserved (8B) + body (variable) - Validated at consensus level: malformed capsules make the transaction invalid

I. cASERT V2 CONSENSUS UPGRADE (BLOCK 1450) - Fork height: 1450 (no regenesis, history preserved) - Halflife: 48h → 24h (faster difficulty response) - Delta cap: 6.25% → 12.5% symmetric (faster per-block adjustment) - Target spacing unchanged: 600s - Anti-stall, profiles, PoW unchanged - Rationale: audit showed slow upward adjustment with V1 parameters

J. SEPARATE PROJECT MINERAL INTELLIGENCE MODULE (NON-CONSENSUS ECOSYSTEM NOTE) - Satellite-based mineral prospectivity mapping using free data and AI - NOT part of consensus — research/exploration module within SOST ecosystem - Phase 8 (March 2026): Multi-source exploration intelligence platform - 8 supervised zones: Kalgoorlie (0.882 AUC), Chuquicamata (0.862), Peru (0.758), Arizona (0.718), Zambia (0.763), Pilbara (FAILED), Pebble Alaska (20/40, COMPLETED), Rosia Montana (QUEUED — epithermal Au-Ag) - Validated families: satellite baseline (universal), thermal 20yr (universal modest), EMIT alteration (porphyry Cu specific), PCA embeddings (Kalgoorlie specific), magnetics (small positive), neighborhood context (promising for mineral ID) - Rejected: spatial gradients, ML residuals, EMIT at orogenic Au, cross-zone transfer - Canonical objective: 23.9/40 (60%). Mineral 4.3/10, Depth 4.5/10, Coordinates 7.0/10, Certainty 8.1/10 - MINERAL = commodity / mineral system / deposit type — metals + non-metallic + strategic elements - Validated commodities: Porphyry Cu (0.882), Orogenic Au (0.879), IOCG (0.841), SEDEX (0.781), Sediment Cu (0.760), Lithium brines (0.891), Alaska porphyry Cu-Au-Mo (queued) - Type-aware auto-selection: tests all available families, selects best per zone and commodity - Framing: “zone-aware, type-aware, multi-commodity mineral intelligence” - Does NOT claim direct subsurface detection — surface proxy inference only - All data free (Sentinel-2, Landsat, EMIT, DEM, MRDS, OZMIN, Mindat.org, EarthMRI, GA geophysics, GA gravity) - Gravity integrated (460m Bouguer) — outperforms spectral at subsurface deposits - Magnetics consolidated at 3 zones (+0.048 to +0.153 over S2) - Full methodology and results at: website/sost-separate-project.html

K. SEPARATE PROJECT CANONICAL OBJECTIVE STATUS (NON-CONSENSUS RESEARCH NOTE) - Canonical objective: “There is [mineral] at [depth] at [coordinates] with [certainty%]” - Current score: 23.9/40 (60%). MINERAL expanded to include non-metallic commodities (lithium, graphite, REE, diamonds). - MINERAL: 4.0/10 — Deposit type discrimination (porphyry Cu, orogenic Au, sediment Cu) - DEPTH: 4.1/10 — Depth estimation (surface proxy — deposit-scale geophysics blocked) - COORDINATES: 7.0/10 — Spatial precision at 30m per pixel - CERTAINTY: 7.7/10 — Calibrated probability (isotonic, Brier validated) - Phase 9 information fusion results (March 2026): - Kalgoorlie: 0.8654 → 0.8770 (+0.012 AUC), calibrated Brier 0.0999 - Zambia: 0.7366 → 0.7584 (+0.022 AUC), calibrated Brier 0.1547 - Neighborhood context multi-zone validated (generalized to Zambia) - Method: neighborhood context + hydrology + magnetics fusion + isotonic calibration - Phase 10: Chuquicamata full fusion (sat+geo+EMIT+neighborhood+hydrology) = 0.882 AUC (+0.093) Biggest improvement ever. Confirms multi-source fusion as the correct architecture. - Phase 11: Kalgoorlie full fusion 0.879 AUC (+0.013). Gravity BLOCKED (manual download needed). Best calibrated Brier ever: 0.096. Multi-source fusion confirmed at 2

independent zones. - Phase 12: Zambia fusion +0.024 AUC. Multi-source fusion confirmed at 3 independent zones. Canonical V3: 22.9/40. Manual data acquisition layer created for blocked datasets. - Phase 13: Data closure. Canonical score methodology frozen at v4: 22.8/40 (57%). All manual data dropzones created but awaiting operator download. - Phase 14: Peru fusion NEGATIVE (-0.063). Fusion helps at 3/4 zones but not universal. Minimum baseline ~0.73 AUC needed to benefit from neighborhood/hydrology fusion. - Phase 15: Baseline-aware gating. Peru diagnostic: fusion hurts weak baselines (<0.73 AUC). 8 adaptive rules. 27 families tracked. Architecture: type+zone+baseline aware. - Phase 16: Macrostrat API activated (4 zones). Peru geology-first +0.168 AUC (needs bias correction). - Phase 17: Geology bias fix. Zambia lithology genuine +0.054 (LOW leakage). Peru still biased. - Phase 18: Geology validated at 3 zones (lithology content > has_data at all 3). Parity needs improvement. - Phase 19: Geology promoted to VALIDATED SELECTIVE. Depth push: all deposit-scale sources BLOCKED. 11 blocked data items documented. The next bottleneck is depth, not architecture. - Ceiling with satellite data only: ~22-24/40 (fundamental limit) - Ceiling with satellite + free geophysics: ~28-30/40 - Ceiling with satellite + geophysics + drill holes: ~36/40 - Path forward: geology integration + gravity + neighborhood context + calibration - a separate project is a probabilistic, proxy-based exploration support tool - It does NOT claim direct subsurface imaging or guaranteed mineral identification

— a separate project Phase 22 Note (2026-03-26) — Frontier validation of spectral unmixing and NDVI trend analysis was attempted but blocked by raw data availability (stacks contain derived indices, not raw reflectance). Google Earth Engine access confirmed, enabling raw data pipeline construction in Phase 23. Canonical score remains 22.8/40 (57%). The platform's bottleneck has shifted from ML architecture to data engineering.

— a separate project Phase 23 Note (2026-03-27) — Raw data pipelines constructed via Google Earth Engine: Sentinel-2 L2A reflectance (10 spectral bands, 4 zones) and multi-year Landsat NDVI (12 annual composites, 2013-2024). Both sample-tested with real satellite data. GEE operationalized within a separate project. Full raster export and frontier validation pending. Canonical score remains 22.8/40 (57%).

— a separate project Phase 24 Note (2026-03-27) — First real frontier validation on exported GEE rasters (no simulation). 4 zones exported: Chuquicamata (267 S2 images), Zambia (528), Kalgoorlie (180), Peru (55). Spectral unmixing real features computed on 500 real pixels per zone. NDVI trend: 12 years real Landsat data per zone. Zambia NDVI promising (mean 0.310, slope +0.0032/yr). Chuquicamata not applicable (hyperarid). Validation blocked by spatial alignment between GEE pixels and labels. Canonical score remains 22.8/40 (57%).

— a separate project Phase 25 Note (2026-03-27) — Spatial alignment resolved via GEE sampleRegions() at exact label coordinates. Zambia: S2 baseline 0.640, unmixing +0.001 (NEUTRAL), NDVI 0.772 (POSITIVE — best single family). Peru: S2 baseline 0.850, unmixing -0.003 (NEUTRAL), NDVI 0.724 (NEGATIVE). Kalgoorlie & Chuquicamata blocked by GEE memory limits. Spectral unmixing: NEUTRAL. NDVI trend: zone-specific (vegetated only). Canonical score remains 22.8/40 (57%) — no improvement exceeding +0.005 threshold.

— a separate project Phase 26 Note (2026-03-27) — Terrain depth pilot: 8 depth sources audited, 3 GEE-accessible (SRTM, SAR, terrain derivatives). 4 zones piloted. Peru depth-only AUC 0.907 (+0.057 vs S2 baseline — terrain features add real value). Zambia 0.642 (+0.002, neutral). Kalgoorlie 0.774, Chuquicamata 0.769. Terrain features are surface proxies, not subsurface depth. True depth unlock needs GA gravity, AEM, Earth MRI (all blocked). GEE terrain infrastructure operational at all 4 zones. Canonical score remains 22.8/40 (57%).

— a separate project Phase 27 Note (2026-03-27) — 9-feature subsurface-aware family built at all 4 zones via GEE (CSP/ERGo landform dataset). topo_diversity = top feature at 3/4 zones. Standalone AUCs strong: Peru 0.902, Kalgoorlie 0.859, Chuquicamata 0.846, Zambia 0.682. Combined with S2: REDUNDANT — Kalgoorlie +0.001 (NEUTRAL), Zambia -0.068 (NEGATIVE), Chuquicamata -0.021 (NEGATIVE), Peru -0.004 (NEUTRAL). Features are redundant with existing spectral. Terrain reclassified: SURFACE_STRUCTURE, not true depth. Real depth needs GA gravity, GSWA AEM, USGS Earth MRI (all manual portals). Canonical score remains 22.8/40 (57%).

— SOST Ecosystem: a separate project & Materials Engine —

a separate project — Mineral Intelligence Platform Canonical Objective: “There is [MINERAL] at [DEPTH] at [COORDINATES] with [X%] certainty.” Canonical Score: 25.1/40 (63%), Phase 48 (gravity fusion validated). - MINERAL 4.0/10, DEPTH 6.0/10, COORDINATES 7.0/10, CERTAINTY 8.1/10 Validated at 6 zones on 4 continents: Chuquicamata (0.882 AUC porphyry Cu), Kalgoorlie (0.922 AUC orogenic Au with gravity), Tennant Creek (0.841 AUC IOCG), Mt Isa (0.781 AUC SEDEX Cu-Pb-Zn), Zambia (0.760 AUC sediment Cu), Peru (0.850 AUC porphyry Cu). Multi-source fusion confirmed at all zones. Architecture: type-aware + zone-aware + baseline-aware. GEE operationalized. Gravity + magnetics integrated. 53+ phases completed. Validated commodities: porphyry Cu, orogenic Au, IOCG, SEDEX Cu-Pb-Zn, sediment Cu, lithium (0.891), graphite. Remote-first architecture validated: works anywhere on the planet with satellite data alone. Cost: $0 (all satellite data free, runs on CPU).

RESTRICTED — Implementation Details: Model architecture, feature engineering algorithms, training data, hyperparameters, gating engine rules, and frontier research experiments are restricted. Available under authorized access only.

Materials Discovery Engine — Autonomous Materials Intelligence 76,193 materials corpus. Predicts formation energy and band gap from crystal structure. 29 campaign profiles across 16 industrial domains. 50+ validated candidates. CGCNN (formation energy MAE ~0.15 eV/atom) + ALIGNN-Lite (band gap MAE 0.34 eV, R^2 0.707). CHGNet pre-DFT structural validation (17 candidates in 40s, 64h CPU saved). Autonomous discovery pipeline: SEED → GENERATE → FILTER → LIFT → GNN → SCORE → VALIDATE → HANDOFF. Full validation bridge with prediction-observation-reconciliation lifecycle. Cost: $0/month (CPU-only, no cloud required). Expansion roadmap: database 76K → 500K+ (Materials Project, OQMD, NOMAD), PGM replacement engine (Pt/Pd/Rh alternatives), multi-property GNN (12+ properties), MLIP validation layer (CHGNet/M3GNet/MACE), 8 mission profiles (Pt-free H2 catalyst, cheap OER, Li capture, desalination, rare-free PV, corrosion alloys).

RESTRICTED — Implementation Details: Neural network architecture, model weights, training methodology, hyperparameter tuning, and backend framework are restricted.

— LEGAL DISCLAIMER —

SOST is experimental, unaudited software. It is a completely new proof-of-work protocol — NOT a fork of Bitcoin, Litecoin, Monero, RandomX, Ethash, Equihash, CryptoNight, X11, Scrypt, or any other existing cryptocurrency or algorithm. The ConvergenceX proof-of-work algorithm, the cASERT difficulty adjustment, and the constitutional economic model are original designs that have not been independently verified by any third-party security auditing firm.

This software is provided "AS IS" without warranty of any kind. The developer (NeoB) makes no representations or warranties regarding the security, reliability, or fitness for purpose of this software. The protocol has not been tested under adversarial conditions by a large number of independent miners. Undiscovered vulnerabilities, bugs, or design flaws may exist.

Every participant is solely and entirely responsible for their own investment decisions. You may lose your entire investment. The developer assumes no liability for any losses. The gold reserve mechanism is not a redemption right, not a price guarantee, and not a stability mechanism. SOST is not a security, not a stablecoin, and not financial advice.

The decision to advance the public announcement to March 30, 2026 was made to enable community participation earlier than originally planned. The developer is continuously monitoring the network and actively searching for potential vulnerabilities. Maximum diligence and caution are recommended during the early operational phase.

Phase 28 Note (2026-03-28): a separate project is an advanced remote sensing and exploration intelligence platform based on multi-source fusion and zone-based validation. Phase 28 adds QGIS operational layer spec: practical tool map, reproducible workflows, and canonical tool mapping. QGIS strengthens COORDINATES (alignment QA) and CERTAINTY (visual inspection) but does NOT solve DEPTH or replace ML. Canonical score 22.8/40 (57%) unchanged.

8.4 Post-Quantum Cryptography Roadmap

SOST Protocol currently uses secp256k1 elliptic curve cryptography for transaction signing — the same battle-tested scheme used by Bitcoin since 2009. While no practical quantum threat exists today, SOST is designed with long-term constitutional integrity in mind.

Current Security Baseline

All SOST addresses (prefix: sost1) are derived from secp256k1 public keys using HASH160 (SHA256 + RIPEMD160). Transaction signatures use ECDSA with DER encoding — producing compact 64-byte signatures verified by libsecp256k1. This provides strong security against all known classical attacks.

The theoretical quantum threat (Shor's algorithm) requires approximately 4,000 stable logical qubits — a capability that current hardware does not possess and that credible estimates place at least 10-15 years away.

Post-Quantum Migration Plan

SOST has reserved the address prefix sost2 for post-quantum addresses and will follow a phased migration approach:

Phase 1 — Research (COMPLETED): Selected algorithm: CRYSTALS-Dilithium (NIST FIPS 204 winner). Signature size: 2,420 bytes. Signing time: ~0.5ms. Verification: ~0.3ms. Security level: NIST Level 3.

Phase 2 — Prototype (2027 Q3-Q4): Integration of liboqs library. Performance benchmarks. Block size analysis.

Phase 3 — Dual Signatures on Testnet (2028 Q1): Transactions carry both ECDSA and Dilithium signatures. sost2 addresses available for testing. Nodes accept both types.

Phase 4 — Mainnet Activation (2028 Q3): sost2 addresses activated on mainnet. Both sost1 and sost2 fully supported.

Phase 5 — ECDSA Deprecation (2030+): Constitutional hard fork via version signaling (95% threshold, 288-block window) when quantum threat materializes.

Design Principle: The migration is intentionally gradual. Forcing immediate adoption would increase TX size by 16x. Dual signatures allow sustainable pace migration without sacrificing performance.

Appendix K: Errata v3.8 (Sovereign Value Stack — Position Market, E2E, Settlement)

Changes from v3.7 to v3.8:

SOVEREIGN VALUE STACK:

56) Section 12.5: expanded from overview to full operational specification
57) Section 12.5.1: E2E layer — ED25519 real signatures, relay node, deal channels
58) Section 12.5.2: renamed “DEX” → “SOST Position Market” — OTC/RFQ model, not AMM
    - Phase 1: Model B full position + reward-right trades
    - Phase 2: Model A reward rights + controlled novation
    - Phase 3: broader market (legal review required)
59) Section 12.5.3: SOSTEscrow deployed on Sepolia with first on-chain gold deposit
    - Address: 0x01Eaab645DA10E79c5Bae1C38d884B4D1a68f113
    - First Model B position: 0.5 oz XAUT, 28-day lock, position 8afed8fcd27553a7
60) Section 12.5.4: PoPC contract infrastructure updated with lifecycle states
61) Section 12.5.5: Architecture — sost-comms now includes relay + deal channels
62) Section 12.5.6: Alpha Status added (375 tests, 0 failures)
63) Section 12.5.7: Strategic Direction — “trade contracts, not pools”

SETTLEMENT ENGINE:

64) Deal state machine: 11 states, full happy path + refund path documented
65) Settlement daemon, Ethereum watcher, SOST watcher, refund engine specified
66) Position pricing formula: gold_value + discounted_reward - illiquidity

POSITION MARKET:

67) Position as identified registry object (not fungible token)
68) Full position trade + reward-right split with signed messages
69) 213 Python + 139 TypeScript + 19 Solidity tests passing

cASERT:

70) H11 ceiling activation at block 5,480 (16 of 40 profiles active)
71) Block interval chart added to explorer with avg/median explanation

Last updated: April 2026

Appendix L: Errata v3.9 (E2E Encryption, Prekeys, Offline Delivery, cASERT Fixes)

Changes from v3.8 to v3.9:

E2E ENCRYPTION (Phase XIII):

72) X25519 key agreement added (separate from ED25519 signing)
73) ChaCha20-Poly1305 AEAD payload encryption
74) HKDF-SHA256 directional channel key derivation
75) Relay converted to blind transport (cannot read message content)
76) Encrypted envelopes with minimal header for routing
77) Replay guard with sliding window + nonce dedup per session
78) Rekey mechanism per deal channel

PREKEYS AND OFFLINE DELIVERY (Phase XIV):

79) Signed Prekey (X25519, rotated 7 days, ED25519-signed)
80) One-Time Prekeys (consumed once, batch replenishment)
81) Async session bootstrap (sender initiates while recipient offline)
82) Store-and-forward offline queue (7-day TTL)
83) Delivery receipts (delivered / acknowledged / processed)
84) Key lifecycle policy (rotation, replenishment, health evaluation)
85) Header minimization (shortened field names, economics inside ciphertext)
86) Prekey bundles publishable to relay for offline discovery

CASERT FIXES:

87) H11 profile descent bug fixed: slew now proportional to stall time (1 level per 10 min of stall, no artificial cap)
88) Profile floor enforcement at block 5,560 (declared profile must be within deterministic [floor, ceiling])
89) Block #5525 H1 validation gap identified and closed

TEST COVERAGE:

90) Total: 467 tests (Python 217 + TypeScript 231 + Solidity 19), 0 failures

Last updated: April 2026

Appendix M: Errata v3.10 (Beneficiary Handoff, Relief Valve, H12)

Changes from v3.9 to v3.10:

BENEFICIARY AUTOMATION (Phase XVI-XVII):

91) principal_owner / reward_owner / eth_beneficiary separation in position schema
92) SOSTEscrowV2 with settlementOperator for automated beneficiary handoff
93) Full sale: changes principal + reward + beneficiary. Reward sale: only reward.
94) settlementOperator can reassign beneficiary but CANNOT withdraw funds
95) Auto-withdraw daemon: executes withdraw at maturity automatically
96) Reward settlement daemon: credits reward to reward_owner at maturity
97) Maturity watcher: ACTIVE → NEARING_MATURITY → MATURED transitions
98) Beneficiary sync: on-chain update after full position trade
99) Reconciliation tools: beneficiary, withdraw, reward status checks
100) Full lifecycle: ACTIVE → MATURED → WITHDRAWN → REWARD_SETTLED → CLOSED

CASERT:

101) H12 ceiling activation at block 5,635 (17 of 40 profiles active, superseded by block 5,750)
102) Relief valve at block 5,635: profile drops to H1 when block > 10m 30s (superseded by E7@605s at block 5,750)
103) Profile descent fix: slew proportional to stall time (1 level per 10 min)
104) Profile floor enforcement at block 5,560

FINAL CALIBRATION (block 5,750):

106) H13 ceiling activation at block 5,750 (21 of 43 profiles active: E7-H13)
107) Relief valve upgraded: E7@605s (was H1@630s), deeper easing on stall
108) 3 new easing profiles: E7, E6, E5 prepended to table
109) E4-H2 margins recalibrated to 5-point gradient (185→155)
110) Lag-adjust interval reduced from 30s to ~6s
111) cASERT equalizer implementation and calibration complete. Additional hardening profiles will activate as hashrate grows.

TEST COVERAGE:

112) Total: 743 tests (C++ 193 + Python 272 + TypeScript 231 + Solidity 47), 0 failures

Last updated: April 2026